

VECTOR

APL99 Report & Pictures ...

- Ray Cannon on APL99 61
- Sharp APL vs APL2 coding style 87
- Budd on time-space tradeoffs 101
- Balako on APL and embedded IE 113
- Chastney on data domains 121
- Neitzel on a J structure compiler 129



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

www.vector.org.uk

Vol.16 No.2 October 1999

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette or via email. APL code can be accepted in workspaces from I-APL, APL+Win, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from Vector Production), and MS Word (any version).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 1999-2000

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	10	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385 Email: apl385@compuserve.com

Contents

	Page
Editorial	3
APL NEWS	
Quick Reference Diary	5
APL2000 User Conference — Abstracts	7
British APL Association News	13
News from Sustaining Members	14
APL Product Guide	19
	Gill Smith
The Education Vector	
Zark Newsletter Extracts	33
J-ottings 22: Something for our very youngest readers	41
	Norman Thomson
The Random Vector	
Bayesian Machine Learning in J	47
	A Charman-Williams
RECENT MEETINGS	
APL99 — Notes and Pictures	61
	Ray Cannon
GENERAL ARTICLES	
The Dance of the Quirk	82
Sharp APL vs APL2 Coding Style	87
Time-space Tradeoffs in Vector Algorithms	101
	Sylvia Camacho
	Dan Baronet
	Timothy Budd
TECHNICAL SECTION	
Hacker's Corner: Making Waves in Win32	110
APL+Internet Explorer	113
Data Domains	121
A Very Simple Structure Compiler for J	129
From Horadam to Atanosov	135
	Adrian Smith
	Alexander Balako
	Phil Chastney
	Martin Neitzel
	Joseph De Kerf
Index to Advertisers	143



A Sparse Array of Light

Look for the upcoming release of J version 4.04, which will include support for sparse arrays - an essential feature for OLAP applications.

Sparse arrays provide a compact and efficient storage form for very large data sets where most elements are zero or some other "sparse element". The sparse array representation does not store copies of the sparse element, thus enabling J to directly manipulate arrays that previously could not be represented entirely in the memory of the computer.

There are many practical applications for this facility, including financial modelling and analysis, engineering and scientific programming.

Check out our website and download your trial version today!



Distributed by:
Strand Software, Inc.
19235 Covington Ct.
Shorewood, MN 55331
(612) 470-7345
Fax (612) 470-9202
info@jsoftware.com

www.jsoftware.com

Editorial

by Stefano Lanzavecchia (stf@apl.it)

"Linux, the choice of a GNU generation"

"GNU is Not Unix"

Anonymous

The first thought brought to my mind by the two syllables "li" and "nu" is one of the characters out of the excellent cartoon by Charles Schultz, namely *Peanuts*. Linus Van Pelt, despite his remarkable sister, was one of my favourites: sharp, open-minded, passionate and considerate, talented, he inspires confidence. Much as I would like to spend time comparing Linus' merits and skills against Snoopy's, somehow I don't think this is the proper forum.

There is of course a second thought, which follows this first one, and this time it's for *Linux*, the equally excellent product originally from Linus Torvalds. A Finnish boy, probably bored by the cold and long winter and distressed by the short summer swarming with mosquitoes of his country, one day decided to play with the multitasking capabilities built into the 80386 CPU from Intel. A stream of As and Bs printed in an interleaved fashion on the screen by two concurrent tasks was his first step to fame. "A Minix better than Minix" was the goal he had set for himself, Minix being a single-user simplified version of the more complex Unix, which could be hosted even by a PC AT. The rest is history, mostly known to us all. Today Linux is a fully POSIX compliant Unix operating system with a solid base of contributed software ranging from academic languages to arcade videogames. Despite the existence of other free Unix systems, among which FreeBSD is very popular, Linux seems to stand in a class of its own. The number of direct and indirect users (how many web sites powered by a Linux box enter the World Wide Web every day?) grows continuously, along with the applications ported to it, the set of devices supported, and the new development, done to enhance its capabilities. The availability of the source code even for the core of the operating system, makes it the ideal environment for hackers to work in, but one does not need to be a hacker any more to feel comfortable with it: the install procedures and the administrators' tools have become so user-friendly that a minimal Unix training is enough to get a new user started. This means, for instance, that even if it's still considered a fundamental step to become a real Linux lover, the re-compilation of the kernel is no longer a required step to set up one's box.

Its cheap price (what could be cheaper than something that is free?) has turned Linux into one of the preferred environments for students in scientific universities: new languages are being developed, new tools, and languages originally created in other Unices are very quickly ported and made available to the community of researchers. The fanatics of functional programming can choose among many versions of *Haskell*, *ML*, *Miranda* and cousins; there are imperative languages covering all the thinkable categories (*C*, *C++*, *Java*, *Pascal*, *Perl*, *Python*, *Tcl*, *LISP*, *Scheme*, *Smalltalk*, even *Fortran* for the masochists or the old-fashioned doctors in physics); let's complete this

quick and incomplete list mentioning *Prolog* and the more domain specific languages (*Mathlab*, *Mathematica*, *Spice*, *CSound*) to finally notice a name missing particularly dear to us all.

Nobody is surprised to find no real APL. Up to now the only APL interpreter available commercially or otherwise is the simple OpenAPL, completely free, source code included in the small distribution package. Unfortunately OpenAPL hardly matches the ISO-APL specs and it's many years behind when compared with the interpreters available on other platforms, especially Windows: the total absence of nested or boxed arrays and the total lack of a development environment would discourage even the most courageous researchers. Once again, no surprise then if APL is disappearing from the universities. Big news comes from Scranton, though: Sharp APL has announced an almost complete port of their interpreter to Linux, to be freely distributed to all non-commercial users. This is indeed good news, but ... Here comes the ever-present "but": Sharp APL is a very interesting version of the language, yet it's different enough from IBM's APL2. The most popular implementations available on the PC platform are all derived from the latter and the downsizing of legacy code from the old mainframes tends to favour this flavour (sorry for the pun).

The attentive readers will have noticed that I am carelessly mixing two completely separate domains: the academic/research domain and the commercial domain. So, what if the APL that can be used to learn how to think in terms of arrays as opposed to scalars is not exactly the same as the one found more easily used in commercial applications? After all, the array-thinking paradigm is useful by itself and that can be taught and learned using Sharp's model. True. But then why not use J instead? The little bit I know of J is far more than I know of Sharp APL but still it's clear to me that the former is an evolution of the latter. And like all the cases of evolutions, it's kept all the good things about its ancestor but it's now gone very far ahead introducing many more useful abstract concepts and completing the implementation of many features simply sketched in Sharp APL. J is available on many platforms, including Linux. Universities can obtain it cheaply for non-commercial applications. And it's modern.

One of the problems caused by being late with the editorial is that only a limited space is left, so I am forced to stop here. If any of you readers have strong opinions about what I said, I am sure you will let me know and we will continue the discussion. If you were not interested, then there would be no point wasting more space. Either way, it's safe to stop here.

Quick Reference Diary 1999-2000

Date	Venue	Event
14-17 Nov 1999	Orlando, Florida	APL2000 User Conference see Abstracts on page 7
24-27 July 2000	Berlin	APL Berlin 2000

APL Berlin 2000

APL Berlin 2000 is an Array Programming Languages Conference, to be held in the Department of Computer Science at the Technical University of Berlin.

The Conference Chairmen are Dieter Lattermann
(dieter_lattermann@compuserve.com) and Conrad Hoesle-Kienzlen
(conrad_hoesle@csi.com).

The Conference has a website at:

<http://stat.cs.tu-berlin.de/APL-Berlin-2000>

mirrored at:

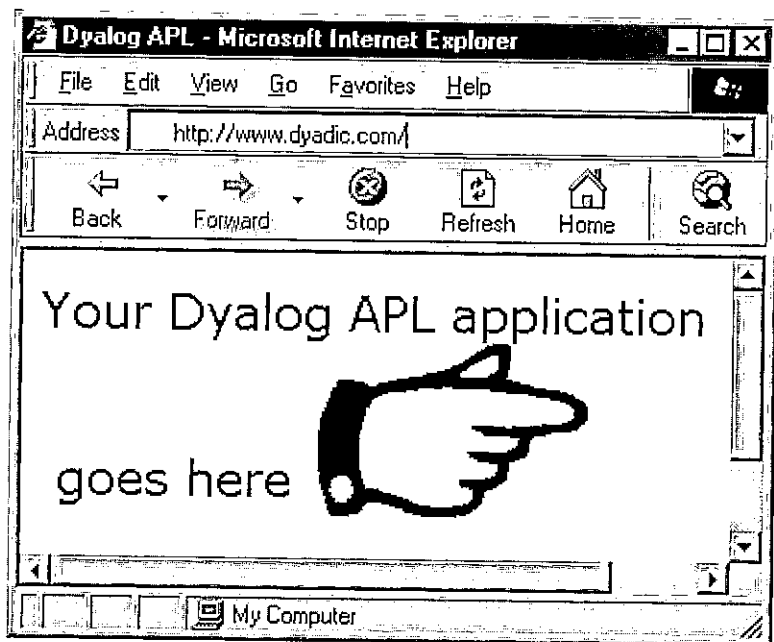
<http://www.lingo.com/APL-Berlin-2000>

Dates for Future Issues of VECTOR

	Vol.16 No.3	Vol.16 No.2	Vol.16 No.3
Copy date	4th December	10th March 00	19th May 00
Ad booking	11th December	17th March	26th May
Ad Copy	18th December	24th March	2nd June
Distribution	January 00	April 00	July 00

dyalog APL

The Definitive APL for Windows™



Using Dyalog APL, not only can you write a powerful, fully functional, **multi-threaded** Web Server, but you can also write ActiveX controls that automatically download and run in the Web browser.

Dyalog APL - the Internet - the Future

Dyadic Systems Limited, Riverside View, Basing Road, Old Basing, Basingstoke,
Hants. RG24 7AL, United Kingdom.

Tel: +44 1256 811125 Fax: +44 1256 811130 Email: sales@dyadic.com

Microsoft is a registered trademark and Windows and the Windows Logo are trademarks of Microsoft Corporation

APL 2000 User Conference

Orlando, November 14th-17th 1999

Abstracts

Plenary Presentations

Cognos Corporation — A New Day For APL, Jim Sinclair, Vice President, Cognos Inc.

SoftMed Systems — Building A Company On APL, Don Segal, President and CEO, SoftMed Systems.

Finac — The Use Of APL in the Colombian Banking System, Luis Caro, President, Finac Ltda.

Training Sessions

Introduction to APL, Ray Polivka. Learn APL via the "See and Do" approach. This course introduces a major portion of the APL programming language. Attendees will learn how to create functions and variables and work with many of the APL primitives available. No previous APL knowledge is required.

Introduction to APL GUI Programming, Rick Butterworth. This course is designed for APL Programmers new to Windows. Learn how to build Windows applications or enhance your existing applications. Includes objects, properties, methods, events, and message driven architecture.

APL On The Internet, Fred Waid. Learn how to access the World Wide Web from your APL application. Make your existing applications Web Aware and let your users access them from any browser over the Internet. Manipulate data from any web site and incorporate that information into your application.

ZarkWin Class, Gary Bergquist. ZarkWin is software for quickly building Windows input forms. Patterned after the popular Screen Interface Toolkit of APL+DOS, ZarkWin was developed by Zark Inc. to allow rapid interactive form design. Forms can be built, tested, and modified with no knowledge of `⌈WI`

Com Objects and APL+Win, Patrick Parks. An Introduction to using Component Object Model (COM) which allows various Windows applications (such as Word and APL+Win) to work together.

WCALL, Brian Chizever. This sessions teaches how to call the Win32 Application Program Interface (API) and any other function in any 32 bit DLL from APL+Win.

Excel, Eric Baelen. Create multipage Excel workbooks from APL complete with formatting and graphics. A suite of functions to do all of the common Excel functions is provided. The session will end with a repeat of "Using Excel as a Front End to APL Applications".

Using APL+Win 3.6, Eric Baelen. An in depth discussion of using the new features available in APL+Win version 3.6.

Taking the Migraine out of Migration, Walter G. Fil. A discussion of the considerations of moving a DOS or Mainframe application to Windows. A utility (Qwin) has been created to make the process much easier.

The Nature And Perils Of Floating Point Calculations In APL, Mark Osborne.

0.040000000000000008

8

Managing Long Running Calculations, Rick Butterworth. Learn to interrupt long running calculations with the options to display intermediate results, resume or cancel without lost work.

APL + CD, John Walker. Learn the basics of Internet communications as you build an application to get CD Album Titles, Artist Name and Track Names from the World Wide Web.

Characters, Codes, Glyphs, Fonts and Unicode, Bill Rutiser. An introduction to the representation, processing, exchange and display of non-ASCII text such as APL characters.

The 14 Myths of APL, Gary Bergquist. APL has been called a dead language. This fatalistic view lingers because of the general acceptance of 14 myths pertaining to APL. In this talk, the myths are presented and dispelled, laying bare the future prospects of APL.

Open Here, Fred Waid. A web site completely written in APL. Learn the tricks and techniques of its construction and how you can benefit from it.

An Outsider's View of the Inside of Nested Arrays, Roy Sykes. Knowing how APL stores nested arrays can help you use them more efficiently. Roy covers the internal representation of arrays and gives general guidelines as to the tradeoffs in using them.

APL+Win and MS Access, Jeremy Main. Use the Microsoft Jet Database Engine from APL+Win to create, update and edit a .MDB file including changing table definitions, bulk update, and managing transactions.

Making It To Paper, Marvin Renich. A set of utilities to control various aspects of printing from Listview, Treeview or RichEdit controls. Control orientation, copies, paper tray, etc.

What Makes A Good Web Site? Carl House and Adrian Smith. Carl uses Causeway's NewLeaf and HTM2000 workspaces for compiling approximately 4000 web pages from databases and other source materials. Carl will discuss design criteria for a good web site, how to promote it, and how to measure its effectiveness. Adrian will introduce the HTM functions, which are freely downloadable from the Causeway website, with supporting documentation taken from articles in Vector 13.4 onwards.

ADO, Warren Baelen. The ActiveX Data Objects (ADO) programming model is Microsoft's strategic, high-level interface to all kinds of data. Learn the basics of ADO and how to access it from APL.

Managing APL Code, Patrick Parks. Methods to use commercially available code management systems with APL based workspaces is discussed. Sample functions for one such system are provided.

UDC, Eric Lescasse. A look into the creation of a suite of User Defined Controls by Lescasse Consulting.

An Excel Front End for APL+Win Applications, Eric Baelen. Version 3.6 of APL+Win now supports events from Excel so it can be more easily used as a front end to an APL application. A sample application is presented.

Creating a User Interface: Evolution or Intelligent Design? Colyn Phillips. A look at a real-world case of creating the user interface to select options for a product's new feature.

Polymorphic APL Control (FId): the Ultimate UDC, Jeff Pedneau. Build all of your application's screens using UDCs, and wrapping them up inside a "container" control (FId), and a few buttons.

APL and Powerpoint, Tom Atkins. Use APL+Win to control a Powerpoint presentation. Add MSAgent characters and interactive forms to hold users' attention.

Getting the Most From APL and User Commands, Jim Weigang. User commands improve programmer productivity. Tools for linking command files together, displaying differences, tracking changes, and paging commands in a run-time application are described.

**Experienced APL Analyst/Programmer
now available for work in UK and EU.**

Experience in banking, insurance and pensions
using IBM mainframe and PC software.

See Mackay Kinloch Limited in the Product Guide.

Rates reduced in Edinburgh area.

APL-Journal — Germany

Contact: Martin Barghoorn

Web Page: www.rhombos.de/rb/apljourn.htm

In der Ausgabe lesen Sie unter anderem:

- M. Neitzel: J — Sprache, System und der Rest der Welt
- C. Hoesle-Kienzlen: OLE mit Dyalog APL und Excel
- IBM Systems Journal Auszug aus: A formal description of system/360
- Sergy Alpin: APL-Applets — Ein neuer Weg zum Kunden
- H. Winkler: Über Rekursion
- H. Zuse: Anmerkungen zum John von Neumann Rechner
- C. Hoesle-Kienzlen: APL, TCP/IP und andere Sprachen: Eine ideale Ehe
- M. Barghoorn: APL — A Parallel Language (Teil 1)

APL Germany e.V. — was ist das?

Der APL Germany e.V. ist ein gemeinnütziger Verein mit Sitz in Düsseldorf. Sein Zweck ist es, die Programmiersprache APL sowie darüberhinausgehend die Verbreitung des Verständnisses der Mensch-Maschine-Kommunikation zu fördern. Zu diesem Zweck veranstaltet APL-Germany e.V. beispielsweise Vorträge, Diskussionen und APL-Kurse, stellt Kontakte zu Computerfirmen her, die APL anbieten, und gibt die Zeitschrift "APL-Journal" heraus.

For the numerically minded.

Competitive Salaries

Cambridge/West London

When it comes to determining the future of technology, the world's biggest IT companies know where to turn. We provide strategic market analysis and forecasting services to major blue-chip companies using the most sophisticated techniques available. A remarkable reputation stems from the talents of individuals with a voracious enthusiasm for complex numerical analysis. Our people have the intellectual vigour and logical approach to deliver incisive solutions.

Working with your forecasting colleagues you will develop tools and algorithms to deliver multidimensional views of markets, enabling commercial heavyweights to anticipate market trends and tailor their product and service strategies. There will be occasional travel to Europe and to our offices in the USA.

This calls for exceptional individuals who are already highly skilled Windows 95/NT developers, with experience of at least two languages, including APL. Working with a small team you will enjoy analysing problems and developing solutions. You will be looking for a company where the atmosphere is professional but informal, with the emphasis on your achievements.

Please send your CV to:

Mark Best,

IPR Corporation Ltd, Heathrow Business Exchange,
23 Clayton Road, Hayes, Middlesex UB3 1AN.

We would appreciate hearing from you by November 15th.

British APL Association News

Nominations for the British APL Association Committee

Our Constitution requires an Annual General Meeting to be held during May at which the results of the election of Committee members is announced. Anyone may nominate any willing member for election to any Committee post. A postal ballot of the membership is held if there is more than one candidate for a post. To allow for an election nominations must be received by 18 December. Nominators and nominees should be paid-up members of the Association.

In most years we have difficulty in filling all the posts so will welcome new nominees. If you might be interested in serving, do please approach any of the committee informally. The amount of effort required depends on the post but can be adjusted to suit the holder. Most Committee business is transacted by e-mail and expenses are paid when a face to face meeting is necessary.

If you have a brilliant idea for an activity to be undertaken by the Association the Committee has the power to co-opt officers to undertake jobs not currently defined. If you are interested contact any member of the current committee who will be delighted to hear from you.

Help Wanted

At a recent meeting of the Vector working group it was agreed that we ought to put together a general purpose guide to APL. The guide should give the books available and their sources, reviews of the various interpreters, help in getting the APL characters displayed and printed and even how to get your keyboard labelled with the APL character set.

We would be grateful to hear from anyone interested in contributing to the effort or if you have comments on how worthwhile such effort would be, particularly if you could help to make it more worthwhile.

We would be particularly interested in your views if you have experience of teaching APL to schoolchildren; we are very keen to help anyone who has tried to do this and is willing to try again.

Please contact me, Anthony Camacho, 100612,1057@compuserve.com or at 11 Auburn Road, Bristol BS6 6LS, England or on 44 + (0) 117 973 0036 and I will pass on contact details to the other interested members of the working group.

Sustaining Members' News

Soliton Associates Ltd.

Soliton Associates Ltd. is pleased to announce the future availability of SHARP APL for Linux. SHARP APL for AIX and Solaris have been used by Soliton's corporate customers since 1994. SHARP APL for Linux is the result of a POSIX port of the AIX/Solaris products. SHARP APL for Linux will be available in October 1999 as a beta release and a production version is planned for the first quarter of 2000.

Personal licences for SHARP APL for Linux will be available free of charge.

A new platform

Linux extends the SHARP APL product line that includes OS/390, AIX and Solaris. Linux can run on personal computers and workstations. The initial investment for setting up Linux is rather small and SHARP APL for Linux is an excellent product for prototyping and market development. With the Linux platform, SHARP APL offers scalability and a growth path for customer applications.

Linux is extremely reliable and efficient. Soliton considers Linux as 'industrial strength' and well suited for SHARP APL. Linux growth will benefit our customers.

Commercial licences for SHARP APL for Linux offer different levels of support and regular upgrades. Personal licences do not have support, but problem reports will be accepted.

Promoting APL

In our effort to promote APL, SHARP APL for Linux can be used, at no cost, for personal use. The free version of the product is complete. The product will be FTP-able from any Linux distribution site and from Soliton's home page (www.soliton.com/linux). There is no required registration process for the personal version. The documentation in PDF format is also free. We may consider selling a CD or a printed manual in the future.

We wish to promote APL among college and university students. By having a Linux version, we wish to encourage resellers and consultants to include SHARP APL in their solutions.

Architecture

SHARP APL for Linux is a true multi-user system. Linux, like Unix, offers its users private disk areas and security. Consequently, SHARP APL for Linux offers private workspaces and files.

Because it runs in a server environment (Linux), you can have several users logged in at the same time running different applications unaware of each other. These users could be on different Linux machines or running W/98 with an X-Server, a JVM (Java VM), Telnet or a browser. As long as you are TCP connected to Linux and have a user-id, you can run APL.

The same multi-user architecture enables an application writer or user to have several APL sessions running concurrently with excellent performance. These sessions are called tasks. A shared variable processor enables tasks to share data with each other, either within the same user-id or system-wide. The shared variable processor is also the mechanism used to contact APs (auxiliary processors).

The workspace size is limited to 2GB but Linux will restrict this limit further depending on its swapping area and memory. Intel Linux kernel 2.0.36 and above is the only supported platform.

Components

SHARP APL for Linux comes with:

- A interpreter that runs as a process;
- A file system where files can be shared among users;
- A private or global shared variable processor;
- An interface to C-code through the Intrinsic Function (IF) facility;
- A set of IFs for Linux file operations, Linux commands;
- A set of IFs for Socket calls;
- A host AP;
- A full screen function editor;
- A set of public workspaces for date calculation, printing, import and export of software;
- A utility to de-fragment files;
- All documentation in PDF format.

Additionally, a set of utilities will be available (not included in the beta package):

- SCL or Socket Client Library to send, and read E-mails from APL; and to FTP files to and from APL;
- SSM or Socket Server Manager to run socket servers including an FTP daemon;
- SBI or Browser Interface to enable applications to have a browser front-end;
- SJI or Java Interface to enable applications to have a Java front-end or use Java services like JDBC.

GUI

The Graphical User Interface for application development is based on Java and the "SJI". This approach is consistent with the server architecture of SHARP APL for Linux where the interface to an application is separated from the application server. This enables the 'client' to an application to be separated by a network, possibly the Internet and the choice of Java offers portability across many platforms including Windows (95, 98, NT, 2000), Mac-OS and most Unix implementations.

In particular, an Intranet connected machine with a Java VM (Netscape or IE4 for example) will be able to run GUI applications on Linux removing the need to have a second desktop. They can share a Linux machine with other users and have the same functionality as if they physically had it on their desks.

The SJI simplifies and provides the Java front-end capability for the application developers. Many will choose to call Java from APL via the SJI. However, it is also possible for a Java program to 'call' APL by using the SSM API for Java.

Language

The SHARP APL language is based upon Ken Iverson's APL dictionary. It has enclosed and heterogeneous arrays. Functionally, it is equal to APL2 but the syntax differs slightly. It is not difficult to program SHARP APL when you know APL2. Events and error trapping are similar to Dyalog APL. APL objects can be packed together into packages that can be stored on file and manipulated. Most large applications page-in code from files that have packages of functions and variables. A rank operator and the notion of rank in all the APL primitives enable elegant programming. A file system is incorporated in the language.

IDE – Integrated Development Environment

A session manager coupled with a full screen function editor simplifies the interactions with APL during application development. This offers portability

and capability to develop APL remotely, separated from the server. An X-server on a Windows platform can be used with the same benefit as using a Linux desktop.

We are currently developing an IDE within the SJI framework. An experimental version should be available by year-end.

Support

The personal version of SHARP APL for Linux does not have any support. However, problem reports can be submitted to linux@soliton.com. We plan to offer comprehensive support plans to our corporate customers. Please contact Soliton for further details.

Contact Information

Please refer to <http://www.soliton.com/linux> for up to date information. For commercial licenses, please contact Laurie Howard (ljh@soliton.com) at +31 20 646 4475 (Europe) or Nancy Lamb (nancy@soliton.com) at (716) 256-6466 (North America).

For information about personal licenses for SHARP APL for Linux, please contact sax@soliton.com.

Dyadic Systems Ltd

At APL99, Dyadic announced that it was developing a high-level interface to ActiveX Data Objects (ADO). This new Microsoft technology promises rapid access to data of all sorts including data in relational databases. ADO is a successor to Microsoft's ODBC technology and works with existing ODBC drivers. This means that it can be used in place of SQAPL. During APL99, Dyadic ran two ADO workshops demonstrating the use of the interface.

Dyadic has recently announced the availability of the LAPACK and FFTW dynamic link libraries which have been customised for the Dyalog APL $\square$ NA interface.

LAPACK is a subroutine library for solving the most commonly occurring problems in numerical linear algebra. It is public-domain software, and can be used freely.

FFTW (Fast Fourier Transform for Windows) is free software, released under the GNU General Public License (GPL).

To complement these DLLs, Dyadic supplies a *MATH* workspace that provides a high-level interface to the underlying C functions which are generally acknowledged to be among the fastest available for Windows.

Other recent enhancements, provided to DSS (Dyadic Support Service) subscribers, include a number of improvements to the Grid object and additional features for the Tracer (debugger).

Dyadic is nearing completion of Dyalog APL Version 8.2 for UNIX (APL/M) on Sun Solaris and IBM AIX and expects to announce a Linux implementation in early 2000. Version 8.2 for UNIX and Linux is almost completely compatible with Version 8.2 for the PC and workspaces developed on a PC may simply be loaded and run under UNIX.

Causeway Graphical Systems Ltd

New product releases now available ...

- *CausewayPro* for Dyalog 8.2 adds support for the Calendar, Splitter and ToolControl objects. Causeway treats the new toolbars as one-sided splitter bars, and will automatically move or resize any other objects which are affected when the bar changes size. There are many small improvements in the Designer (you can now set common properties on multiple objects, copy and paste properties and so on), and there is a new namespace to help edit bitmap artwork for toolbars and imagelists.
- *RainPro* adds support for mapped images (GIF or PNG format) to allow fully interactive graphics on your web site. All charts now have the option of high-colour, using the '#FF' notation common to the web. Markers and fill patterns are now easy to define, and there is automatic support for trellis layouts to show slices through multi-dimensional data.
- *Newleaf* also supports high-colour, as well as background colour in table cells. Tables generated using Spread now have a 'fit to page' option which scales the output to fit to n pages wide, or to fit entirely on 1 page.

Users with licence numbers after 98041 may upgrade free; please download the software from the web site and contact us for the passwords and updated manuals.

We are delighted to welcome Jonathan Manktelow to Causeway; he has worked with the Causeway software for nearly 2 years, as well as being a highly professional Delphi and C++ developer. One of his first jobs will be to implement a good working subset of RainPro on Sharp APL for Linux; this will be available on the same terms as the interpreter.

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete Systems (Hardware & Software)
- APL and J Interpreters
- APL-based Packages
- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

*Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.
Please help us keep it up to date!*

All contributions and updates to the Vector Product Guide should be sent to Gill Smith, at Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385, Email: apl385@compuserve.com

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
Optima	IBM Compatible	poa	Complete networked or stand-alone solutions including configuration installation, maintenance and commissioning.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
The Bloomsbury Software Company	APL+PC Version 11	250	Upgrade to version 11 gives free runtime (£120 from any version)
	APL+Win v3.0	1350	A 32-bit Windows-hosted interpreter that runs under all Windows platforms including Windows 95. Note: Upgrade for £350 from version 1.8 or 2, £920 from version 1.0
	Migration to APL+Win	1060	from APL*PLUS/PC APL*PLUS/DOS any version. from earlier versions of APL*PLUS II
	APL+DOS	1250	APL*PLUS II DOS is renamed to APL+DOS.
	Migration to APL+DOS	760	from APL*PLUS/PC
	APL Link	200	Database Access
	APL Link Pro	500	
	APL*PLUS II for UNIX	poa	APL2000's 2nd generation APL for all major Sparc and Risc Unix workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.

Dyadic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle Interface available for IBM, Sun and Xenix versions.
DynArray	DICE for Windows	poa	Software development kit which includes an APL Interpreter as a DLL and the ability to run and link existing and new APL code to non APL code such as VB, C/C++, Java and integration with various Windows software applications and database packages such as MS Office.
IAC/Human Interfaces			
I-APL Ltd	I-APL/Mac	13	Macintosh version of I-APL
	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
IBM APL Products	I-APL/BBC Master	8	As above
	I-APL/Archimedes	8	As above
	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	APL2 PC (US Version)	\$630	Product No. 5799-PGG. PRPQ Number RJ0411. Order from 1-800-IBM-CALL
	APL2 PC (European Version)	£348	Product No. 5804-280. Part number 38F1753. From all IBM dealers, including MicroAPL.
	APL2 for OS/2 Entry Edition	\$185	Part No 89G1555.
	APL2 for OS/2 Advanced Edition	\$650	Part No 89G1697. Contains all facilities of the Entry Edition plus: DB2 Interface; co-operative processing TCP/IP Interface; tools for writing APIs; TIME facility
	APL2 for Windows version 1.0	\$1500	Product No. 5639-d46, part number 4229558.
	APL2 Runtime environment	\$250	Part No. 39L8419 - Packager and runtime CDs. One additional runtime install \$200, 5 additional \$900, 10 \$1,500.
	APL2 for Sun Solaris	\$1500	Product No. 5648-065.
	APL2 for AIX 6000	poa	Product No. 5765-012.
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Envt Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
Insight Systems	APL*PLUS/PC	poa	APL systems marketed and supported ...
	Dyalog APL	poa	from: Dyadic, Manugistics, IBM
	APL2	poa	under: Windows, OS2 and Unix
Iverson Software Inc. J on the Web online registration ...			
	J Educational Edition	\$95	
	J Standard Edition	\$295	
	J Professional	\$895	
	Books and accessories (discounts for reg users)		
	J Dictionary	\$50	
	J User Manual	\$50	
	J Phrases	\$50	
	J Primer	\$50	
	Concrete Math	\$40	
	Exploring Math	\$50	

	J User Conference Proceedings	\$35	
	Mugs, T-shirts, Mousepads	\$10 each	
J Austria	J	poa	Distributor for Austria and Switzerland
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also
	APL+Unix	poa	distributes in Switzerland and Belgium. Call for price quotes.
	APL+DOS	poa	
	APL+Win	poa	
	Dyalog APL/W	poa	French distributor for Dyalog
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APL68000 Level I	2000	First generation APL with numerous enhancements. Multi-user version (Unix, Mirage, MCS).
	APL68000 Level II	2500	Second generation APL. Nested arrays, user defined operators, selective specification etc. Multi-user version (Unix, Mirage, MCS)
	APL68000/X	1500-6000	Second-generation APL. Nested arrays, user defined operators, selective specification, etc. Multi-user AIX version with full OSF/Motif support.
	APL68000 Level I Mac, ST, Amiga	87	First generation APL. Single user, full windowing interface, software floating point support.
	Mac, Amiga	260	First generation APL. Single user, full windowing interface, hardware floating point.
	APL68000 Level II ST	170	Second generation APL. Full windowing interface, software floating point support.
	Amiga	260	Second generation APL. Full windowing interface. Hardware and software floating point support.
	Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
	APL*PLUS Rel 10	450	
	APL*PLUS II V 4.0	1395	
Oasis	Dyalog APL	poa	Dyalog Systems
	APL*PLUS	poa	Manugistics
	APL68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Omega	Zero	poa	A "small simple and fast" alternative to APL
Optima	Dyalog APL/W	995	Fully fledged Windows development environment.
RE Time Tracker Cy	APL+PC (APL*PLUS/PC)	poa	Complete APL+ and Statgraphics product range and links to various 3rd party products.
	APL+DOS (APL*PLUS II)		
	APL+Win (APL*PLUS III), APL+Link		
	APL+UNIX		
	APL*PLUS Sharefile		
Soliton Associates	SHARP APL for OS/390	poa	for IBM OS/390 mainframes
	SHARP APL for Unix	poa	for IBM AIX and Sun Solaris

	SHARP APL for Unix	poa	for Intel Unix
Strand Software	Canada		
	All APL*PLUS Products	poa	All APL*PLUS products including upgrades and educational.
	Dyadic and ISI products	poa	
	USA		
	Dyadic and ISI products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADAPTA Software	MPS	poa	Master Production Scheduling
	FBS	poa	Forecasting and Budgeting System
	DRP	poa	Distribution Requirements Planning
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
Adaytum Software	Adaytum Planning	poa	Full-featured Budgeting and Financial Planning system for medium to large enterprises.
APL Group (see Eventra)			
APL Software\Services	APL Utilities	poa	Software: mostly .AWS for DOS, utilities for most APL Interpreters. Public domain APL*Plus v10 with on-screen documentation and interactive tutorials. APL Conference Software, Books; APL user manuals for STSC, IBM, and Sharp. Request email catalog from dck.holt@juno.com.
Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
The Bloomsbury Software Company (for VSAPL)	Enhancements & Sharefile	poa	Component files, quad-functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	Compiler	poa	The First APL compiler!
(for APL2)	Sharefile/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage.
Causeway	CausewayPro for Dyalog/W	400/\$600	Causeway application development platform for Dyalog APL/W.
	RainPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability. Web publishing and clipboard support to the shareware product. Charts can be included in NewLeaf reports. Functionally compatible across Dyalog/W and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win (1.8 and above).
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.

CODEWORK	HELM	poa	Decision Support System for top management. Handles large multi-dimensional tables, data analysis, EIS presentations, generates HTML and Latex output. Platforms: MS-DOS, LAN, Windows 3.1/95/NT. Ideal for APL customisation (APL*PLUS II and Dyalog APL); more than 150 installed.
DynArray	DynaWeb Server	poa	A web server providing web based access to applications running on the DICE interpreter from DynArray, or on an IBM mainframe running APL2.
	DynaHarry	poa	A DSS system which offers the next generation capabilities for current APLDI, IC/E and IC/I users. It comes with ROLAP capabilities, multisystem access to a wide variety of databases and data warehouses.
	DynaLink	poa	An ODBC client interface for DICE and IBM APL2 programs.
Eventra	QualedI	\$1500-4000	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
IAC/Human Interfaces	SPARKS	poa	Educational simulation of electric circuit (for Apple Mac.)
	EPIDEMIC	poa	Educational simulation of spreading infection (for Apple Mac.)
	COINS	poa	Educational simulation (KS3) of coin-tossing experiment with simple stats (for Apple Mac.)
	FIBONA	poa	Educational simulation of Fibonacci's rabbits (for Apple Mac.)
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingerst!
IBM APL Products	A Graphical Statistical System (AGSS)	\$250	for DOS, Product Number 5764-009
		\$500	for Workstations (OS/2, Aix, Solaris), Product Number 6764-092
		\$2500	for CMS, Product Number 5764-011
INFOSTROY	APL*PLUS/Xbase Interface (II/385 Version 2)	\$198	Complete package written in C. Comparable with the data, index & memo files of FoxPro, dBASE, & Clipper. Multi-user support. No DBMS license required.
	(DLL Version 1)	\$198	The same in a DLL form! Gives your Windows applications all advantages of DLLs.
Insight Systems	IUTILS/XP	20-95	Cross-platform utility library including simple OS calls (DIR, COPY, DEL, RENAME) and DATE functions. For APL*PLUS II, APL2 and Dyalog APL under Windows, OS/2 and Unix.
	ASI	95	APL Spreadsheet Interface. "Device-independent" spreadsheet driver supporting Excel, 123 and Quattro-Pro for Dyalog APL/W
	WinCom	95	Asynchronous comms package for Dyalog APL/W
	S2D,22D,X2X	poa	Advanced APL syntax analysis and conversion packages from Sharp and APL2 to Dyalog, and between any two APLs
	SQAPL Client	poa	Interface from APL*PLUS II, APL2 and Dyalog (Windows, OS/2 or Unix) to most SQL databases over most networks.
	SQAPL Server	poa	Makes APL*PLUS II, APL2 or Dyalog APL (Unix) available as SequelLink servers. Can be called from SQAPL clients or other applications such as Excel, C++, Smalltalk, Visual Basic.
JAD Software	JAD SMS	poa	Software management system for Dyalog APL (7.2 or later) based on hierarchical databases; includes gui interface and stand-alone functions. Pricing will be per-user. Available fall 1999.
Lescasse Consulting	APL+Win Monthly Training Program	\$600	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.

	Advanced Windows Programming ...	\$95	200-page book plus companion disk on Interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.
	DLL parser for APL	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC Interface for APL+Win
	SQAPL Pro	poa	ODBC Interface for Dyalog APL/W
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Lingo Allegro	FRESCO Business Graphics	poa	Fast and easy business graphics DLL
	AP126/PC	poa	GDDM interface for Dyalog APL/W
	AP127/PC	poa	ODBC interface for Dyalog APL/W
	AP119/PC	poa	TCP/IP interface for Dyalog APL/W
	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	ISAP	poa	MS Windows DLL for calling Dyalog APL/W from web pages via MS Internet Information Server. Visit www.lingo.com for a demo.
RE Time Tracker Oy	UIT/W	poa	Comprehensive high-level Windows User Interface library for APL+Win and +tl v 5.1. Comprehensive spreadsheets, replicated fields, special field types, etc. 16 and 32 bit versions available.
	AJGRAPH	poa	Graphpak-compatible 2D graphics package for +Win and +DOS. Includes multi-window support, print and metafile support. No DLLs required.
	ECCO PRO with APL	poa	Leading group and personal information management system with comprehensive customising. Supplied with sample +Win workspace to interface to ECCO databases via DDE.
	NEWT TCP/IP SDK with APL	poa	Lead TCP/IP SDK with interfaces to all protocols. Supplied on 3 CD ROMS together with a sample +Win workspace.
	DB+	poa	Database Interface for APL+DOS under Windows. Allows combining character-based APL applications with ODBC-compliant databases such as Oracle and SQL-server..
Soliton Associates	LOGOS	poa	Application Development Environment
	VIEWPOINT	poa	Report generator with Interfaces to DB2 and OS/390 data
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL+PLUS PC & APL+PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) Including 2-disk set of utility functions (APL+PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Ajay Askoolum	Consultancy	poa	APL+Win development and migration of actuarial, financial, mathematical applications.
Andrews	Consultancy	poa	APL programming and analysis, Year-2000 legacy systems, algorithms, tree-processing.
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1989.
AUSCAN Software	Consultancy	poa	APL software development, training
Bloomsbury Software	Consultancy	300-750+VAT	
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
Paul Chapman	Consultancy	250-500	24-hour programmer: APL, Smalltalk, C; Windows front end design a speciality.
CODEWORK	Consultancy	poa	Development, maintenance, migration, documentation of APL applications. Speciality: Info systems for top executives, Internet applications.
Dincsoft Oy	Consultancy	poa	Specialised in very large databases.
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
DynArray	Consultancy	poa	DynArray offers consulting in the areas of DSS, Y2K and APL programs upgrade/conversion to modern Web enabled platforms.
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
First Derivative Analytics Ltd.	Consultancy	poa	Analysis, design, prototyping, development & testing of APL (especially financial) applications: Sharp, Dyalog APL/W.
General Software	Consultancy	from 200	Over 20 years experience with every version of APL, large mainframe systems and small PC based programmes.
Godin London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Hoekstra Systems Ltd	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
Michael Hughes	Consultancy	poa	Consultant with 10+ years experience with various APL Interpreters and C.
IAC/Human Interfaces	Consultancy	poa	APL and ergonomics consultancy services.
	Documentation	100-200	On-line assistance, product demos & mock-ups, manual writing; foreign language software localization.
	Training	poa	Using I-APL for courseware & distance learning materials; Mac programming in C, APL & HyperCard.
INFOSTROY	Consultancy	poa	Moving applications between platforms. Client/server development. Multilingual user interface.

Insight Systems	Consultancy	poa	Experts in APL conversions between any combination of: APL*PLUS, APL2, Dyalog APL and Sharp APL. We are also experienced right-sizers, comfortable with networks and relational databases (that also means when NOT to use SQL) and client/server development in APL, C and Visual Basic.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.
Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APL/W. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lingo Allegro	Consultancy	poa	General APL consulting, internet website development, migration and downsizing, performance tuning, education and training.
George MacLeod	Consultancy	poa	Design and programming of new APL applications. Enhancing and maintaining existing APL applications. Porting existing APL applications from one APL system to another. Supporting users of APL applications. Experienced on both mainframe, UNIX and PC APL Interpreters.
Mackay Kinloch Ltd	Consultancy	from £40/hr	Design, analysis and programming for banking, insurance and pensions, financial planning and modelling, corporate performance and legal reporting
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Millinta Inc	Consultancy	poa	Design, development, maintenance, conversion, documentation in all APLs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Y2K, Windows programming.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Oasis	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning; for all APL versions (10+ years experience)
Object Oriented Ltd	Consultancy	poa	General APL consulting, code recycling — mainframe to PC, performance tuning.
Omega Computing	Consultancy	poa	APL consultancy, programming, etc.
Optima	Consultancy	poa	A range of consultants specialising in all areas of pharmaceutical, industrial and financial systems with 5-15 yrs experience on both PC and mainframe.
RadSys Technologies	Consultancy	poa	Areas of expertise: financial systems, risk analysis systems, healthcare systems.
RE Time Tracker Oy	Consultancy	poa	APL application conversions, APL Windows interfaces, APL to API-level interfacing to any system under Windows, TCP/IP network and database connectivity.
Rex Swain	Consultancy	poa	Independent consultant, 20 years experience. Custom software development & training, PC and/or mainframe.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Shepp & Associates	Consultancy	poa	APL applications development and consulting, especially in the travel industry, especially on small computers. 25 years experience in APL programming.
Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
SovAPL	Consultancy	poa	Offshore APL development service.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.

Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
Stephen Wynn	Consultancy	poa	Most experience of financial planning, and mathematical areas: operational research, quality control, experimental design.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification.
Bloomsbury Software	Training	poa	Contact the company for details.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	Books	poa	I-APL stocks books written to go with the I-APL interpreter and some APL Press books. For a list write to 11 Auburn Road, Bristol BS6 6LS, ring 0117 973 0036 or email 100612.1057@compuserve.com.
Oasis	Training	poa	Introductory courses in APL Advanced courses for different APL versions
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Soliton Associates	MVSLINK	poa	Interface from Sharp APL (Unix & MVS) to non-APL data and software in the MVS environment.
	SSQL	poa	High-performance DB2 interface for Sharp APL (Unix and MVS).

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM SigAPL	International	APL QuoteQuad	Conferences; APL, white pages; web site	\$30
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Club Germany	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL	FF350 (private) FF2800 (Company)	
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Capital PCUG	Washington, D.C.	Monitor	Monthly meetings, occasional classes	free
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	Vector provided	Mini-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat)	10,000yen to join
NY SigAPL	New York, USA	Big Apple APL	Monthly meetings	\$35/\$25(ACM)
Rome/Italy SIG	Roma, Italy			
SE APL Users Grp	Atlanta, Georgia	SEAPL Newsletter	Quarterly meetings	\$10
SovAPL	Moscow, Russia	-	Seminars and Annual Meeting	
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG) Bern		Part of Qity SI-Info	SF60 (SI) + SF20 (SAUG)	
Toronto SIG	Toronto, Canada	Gimme Arrays!	Monthly Meetings, APL skills database, J SIG, Toronto Toolkit	\$25

ADDRESSES

ORGANISATION CONTACT

ACM SigAPL David Siegel
 ADAPTA Software GmbH Michael Baas

Adaptable Systems Lois & Richard Hill

Adaytum Software Douglas Rowley
 Adfee Bernard Smoor

Andrews Dr Anne D Wilson
 APL-385 Adrian Smith

APL Bay Area APLBUG Curtis Jones (Sec)

APL Club Austria Harald F. Nelson

APL Club Germany Dieter Lattermann

APL Group (see Eventra)

APL Software/Services Dick Holt

APL Solutions Inc Eric Landau

Ajay Askoolum Ajay Askoolum

Association Francophone pour
 la promotion d'APL Ludmila Lemagnen

AUSCAN Software Ltd Richard Procter

BACUS Joseph De Kerf

Beautiful Systems, Inc. Jim Goff

Bloomsbury Software Peter Day

Camacho Anthony Camacho

Ray Cannon 21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK.
 Tel: 01252-874697 Email: 100430.740@compuserve.com

Causeway Graphical Systems Ltd Adrian Smith
 The Maltings, Castlegate, MALTON, North Yorks YO17 7DP, UK.
 Tel: 01653-698760 Fax: 01653-697719
 Email: causeway@compuserve.com

Paul Chapman 51B Lams Conduit Street, London WC1N 3NB, UK.
 Tel: 0171-404 5401. Compuserve: 100343,3210

Cinerea AB Rolf Kornemark
 Box 61, S-193 00 Sigtuna, Sweden.
 Tel/Fax: +46 859 255 421 Email: rolf@cinerea.se

CODEWORK Mauro Guazzo
 Corso Cairoli 32, 10123 Torino, Italy.
 Tel: +39 11 885168 Fax: +39 11 812 2652 Email: codework@inrete.it

ComLog Software Jeff Pedneau
 18728 Bloomfield Road, Olney, MD 20832 USA
 Tel: +1 (301) 260-1435 Email: jeff@softmed.com

CPLUG Lynne Sturtz
 Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville,
 Maryland 20850-2421, USA. Tel: +1 (301) 762-9372 Fax: (301) 762-9375.

Danish User Group Per Gjerlov
 Email: gjerlov@ibm.net

Dinosoft Oy Pertti Kaliojärvi
 Lännrotinkatu 21C, 00120 Helsinki, FINLAND.
 Tel: +358 9 70028820 Fax: +358 9 70028824 Email: dinosoft@dinosoft.fi

Dutch APL Association Bernard Smoor (Sec)
 Postbus 1341, 3430BH Nieuwegein, Netherlands.
 Tel: +31 347 342 337 Fax: +31 347 342 342

ADDRESS, TELEPHONE, FAX, EMAIL etc.

ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
 Marienhoehe 88, 25451 Quickborn, Germany. Tel: +49 4108 60977
 Fax: +49 4106 67869 Email: info@adapta.de

49 First Street, Black Rock 3193, Australia.
 Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: adsys@ibm.net

13 Great George Street, BRISTOL BS1 5RR, UK. Tel: 0117-921 5555

Dorpsstraat 50, 4128 BZ Lexmond, Netherlands.
 Tel +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl

12 Thorny Hills, Kendal, Cumbria LA9 7AL, UK. Tel: 01539-731205

Brook House, Gilling East, York YO62 4JJ, UK, Tel: 01439-788385
 Email: 100331.644@compuserve.com

228 South 15th Street, San Jose, CA 95112-2150, USA
 Tel: +1 (408) 292-4060 Email: jonesca@vnet.ibm.com

c/o N-TECH, Siebenbrunnenfeldg. 4-6, A-1050 Wien, Austria.

Tel: +43 1 5458063 Fax: +43 1 5458063-17
 Rheinstraße 23, D-69190 Walldorf, Germany.
 Tel: +49 6227-63469 Compuserve: 100332,1461

3802 N Richmond St, Suite 271, Arlington, VA 22207 USA
 Tel: +1 (703) 528-7824; Fax: +1 (703) 528-7817; Email: dick.holt@juno.org

1107 Dale Drive, Silver Spring, MD 20910-1607 USA
 Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: elandau@cals.com

42 Hanworth Road, Redhill, Surrey RH1 5HT
 Tel: 01737-771643 Email: 106173.3347@compuserve.com

174 Boulevard de Charonne, F-75020 Paris, FRANCE
 Email: lemagnen@aol.com

PO Box 39, Mensfield, Ontario L0N 1M0 Canada
 Tel: +1-705-434-1239 Email: rjp@interlog.com

Rooiberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24

308 Old York Road, Suite 5, Jankintown, PA 19046, USA
 Tel: +1 (215) 886-2636; Fax: +1 (215) 886-4888

Bloomsbury House, 74-77 Great Russell St., London WC1B 3DA, UK.
 Tel: 0171-436 9481 Fax: 0171-436 0524
 Email: pd@bloomsbury-software.co.uk

11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036.
 email: 100612.1057@compuserve.com

21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK.
 Tel: 01252-874697 Email: 100430.740@compuserve.com

The Maltings, Castlegate, MALTON, North Yorks YO17 7DP, UK.
 Tel: 01653-698760 Fax: 01653-697719
 Email: causeway@compuserve.com

51B Lams Conduit Street, London WC1N 3NB, UK.
 Tel: 0171-404 5401. Compuserve: 100343,3210

Box 61, S-193 00 Sigtuna, Sweden.
 Tel/Fax: +46 859 255 421 Email: rolf@cinerea.se

Corso Cairoli 32, 10123 Torino, Italy.
 Tel: +39 11 885168 Fax: +39 11 812 2652 Email: codework@inrete.it

18728 Bloomfield Road, Olney, MD 20832 USA
 Tel: +1 (301) 260-1435 Email: jeff@softmed.com

Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville,
 Maryland 20850-2421, USA. Tel: +1 (301) 762-9372 Fax: (301) 762-9375.

Email: gjerlov@ibm.net

Lännrotinkatu 21C, 00120 Helsinki, FINLAND.
 Tel: +358 9 70028820 Fax: +358 9 70028824 Email: dinosoft@dinosoft.fi

Postbus 1341, 3430BH Nieuwegein, Netherlands.
 Tel: +31 347 342 337 Fax: +31 347 342 342

Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL, UK. Tel: 01256-811125 Fax: 01256-811130
DynArray Corporation	Dr James Brown	16360 Monterey Rd. Suite 260, Morgan Hill, CA 95037, USA Tel: +1 (408)-782-6648 Fax: +1 (408)-782-6827 Email: info@DynArray.com
Eventra	Stuart Sawabini	Meritt Crossing, 440 Wheelers Farms Road, Milford, CT 06460, USA. Tel: +1 (203) 882-9988 Fax: +1 (203) 882-9946 Email: ssawabini@eventra.com; eshaw@eventra.com
Evestic AB	Olle Evero	Bertellusvagen 12A, S-146 38 Tullinge, Sweden Tel&Fax: +46 778 4410 Email: olle.evero@mailbox.swipnet.se
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olli.paavola@pyr.fi
First Derivative Analytics Ltd.	Ken Chakahwata	114 Lemsford Lane, Welwyn Garden City, Herts AL8 6YP, UK Tel/Fax: 01707-339620. Email: KenChakahwata@compuserve.com
General Software Ltd	M.E. Martin	Little Wester House, Westerhill Road, LINTON, Kent ME17 4BS Tel: 01622 749328 Fax: 01622 749365 E-mail: martin@gssoft.freeserve.co.uk
Godin London Incorporated	Gaëtan Godin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 438-8381 Email: info@godin.on.ca
H.M.W.Trading Systems Ltd		Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA, UK. Tel: 0171-353 8900; Fax: 0171-353 3325; Email: 100020.2632@compuserve.com
Hoekstra Systems Ltd	Bob Hoekstra	Dominique, Salsbury Road, Woking, Surrey, GU22 7UR, UK. Tel: 01483-771028 Email: bob.hoekstra@khamlin.demon.co.uk
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL, UK. Tel: 01536-770998 Email: Michael@Hughes.uk.com
IAC/Human Interfaces	Ian A. Clark	IAC Human Interfaces, Unit R21, Auckland New Business Centre, St. Helen Auckland, Bishop Auckland, County Durham, DL14 9TX, UK. Tel: 01388-605544. Email: 100021.3073@compuserve.com
I-APL Ltd	Anthony Camacho	11 Auburn Road, Redland, Bristol BS8 6LS, UK. Tel: 0117-973 0036. Email: 100812.1057@compuserve.com
IBM APL Products	Nancy Wheeler	APL Products, IBM Santa Teresa, Dept M46/D12, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 453-APL2 (-2752) Fax: +1 (408) 453-4488 Email: APL2@vnet.ibm.com Cserve: GO IBMAPL2
INFOSTROY	Alexei Miroshnikov	3 S. Tulerin Lane, St. Petersburg 191186 Russia. Tel: +7 812 312-2673 Fax: +7 812 311-2184 Email: airm@infostroy.spb.su
Insight Systems ApS	Morten Kromberg	Nordre Strandvej 119C, DK-3150 Hellebæk, Denmark Tel: +45 49 76 20 20 Fax: +45 49 76 20 30 Email: info@insight.dk
Iverson Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9. Tel: +1 (416) 925-6098; Fax: +1 (416) 488-7559 Email: info@isoftware.com
JAD Software	David Crossley	175 East 96th St., Apt. 17G, New York, NY 10128 Country: USA Tel: +1 (212) 369-6713 Fax: +1 (212) 761-0124
Japan APL Assoc	Toshio Nishikawa	1-8-13 Masujima Buid.6F Higashi Gotanda Shinagawa-ku, Tokyo Japan 141-0022. Tel: +81 (03) 3280-0411 Fax: +81 (03) 3280-0418 Email: KY00361@niftyserve.or.jp
J Austria	Joachim Hoffmann	Bramley, Oldstead, YORK YO61 4BL. Tel: 01347-868121. Email: JoHo@ping.at
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts, EN8 8JY, UK. Tel: 01992-633807 Fax: 0121-359 0375 Email: phil_last@compuserve.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France. Tel: +33.1.48.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescassee.com
Lingo Allegro USA, Inc	Steven J Halasz	1105 Chicago Avenue, Suite 155, Oak Park, IL 60302, USA. Tel: +1 800 546 4621-1 Email: sjhalasz@interaccess.com
Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK. Tel: +44 (0)131 228 5235 Email: akinkoch@globalnet.co.uk
George MacLeod	George MacLeod	37 Newhouse Rd, Bovingdon, Herts, HP3 0EJ, UK. Tel: 01442-834015 Email: macleod@entropy-group.freeserve.co.uk
Mercia Software Ltd.	Gareth Brentnall	Holt Court North, Heneage Street West, Aston Science Park, Birmingham B7 4AX, UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	Richard Nabavi	South Bank Technopark, 90 London Road, LONDON SE1 6LN, UK. Tel: 0171-922 8866 Fax: 0171-928 1006 Email: MicroAPL@microapl.demon.co.uk
Milinta Inc.	Dan Baronet	4215 St-Andre, Montreal, CANADA H2J 2Z3. Tel: +1 (514) 529-1434

Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE, UK. Tel: 01730-263843 Email: Ellis@mrtdm.demon.co.uk
NY Sig	David Siegel	PO Box 2697; New York, NY10163-2697, USA. Email: NYSIGAPL@ACM.ORG
Oasis b.v.	Theo Zwart, Louis Rijkse	Lekstraat 4, 3433 ZB Nieuwegein, Holland. Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: info@oasis.nl or rijkse@oasis.nl
Object Oriented Ltd	Walter G. Fil	Am Grendel 2, CH-6004 Luzern, Switzerland. Tel: 41 41 418 70 70 Fax: 41 41 418 70 77 Email: info@object-oriented.com
Omega Computing Inc	Alan Graham, Andrew Chou	3 Columbus Avenue, Edison, NJ 08817, USA. Tel: +1 (732) 985 9519 Email: alangraham@mindspring.com
Optima Systems Ltd	Paul Grosvenor	115 Brighton Road, Purley, Surrey CR8 4HE, UK. Tel: 0181-763 2490 Fax: 0181-763 2491
RadSys Technologies AB	Randolph Schrab	Lovsångarv. 18, S-756 52 Uppsala, Sweden. Tel: +46 18 32 41 53 Fax: +46 708 1996 11 Email: randolph.schrab@radsys.se
Renaissance Data Systems	Ed Shaw	P.O. Box 313, Newtown, CT 06470, USA. Tel: +1 (203) 270-9729 Email: rendata@aplbooks.cnchost.com or orders@aplbooks.cnchost.com
RE Time Tracker Oy	Richard Eller	Mkonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
The Rochester Group Inc.	Robert Miller	600 Park Avenue, Rochester, NY 14607-2926, USA. Tel: +1 (716) 271-1110. Fax: +1 (716) 271-1230
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00100-Roma Trullo, Italy Email: marsac@vnet.ibm.com
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapl@oc.aol.com
Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 6th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 UK Address: Claridge House, 29 Barnes High St, London SW13 9LW Tel: 0181 8768656 Fax: 0181 8768650 Email: ashepp@compuserve.com
Snake Island Research Inc	Bob Bemecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bemecky@eocg.toronto.edu
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA. Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Soliton Associates	Laurie Howard	Soliton Associates Ltd, Groot Blakenberg 53, 1082 AC Amsterdam, Netherlands Tel: +31 20 646 4475 Fax: +31 20 644 1206 Email: sales@soliton.com
SovAPL Russian Chapter of SIGAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region 249020, Russia Tel: +7(08439)31463 Fax: +1 (530) 504 8194 Email: askom@obninsk.com
Strand Software Inc	Anne Faust	19235 Covington Court, Shorewood MN 55331 USA Tel: +1 (612) 470-7345 Email: annfaust@aol.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rex@rexswain.com
SwedAPL	Christer Uthleim	Novator Consulting Group AB, Svärdvägen 11C, S-182 33 Danderyd Sweden. Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CService: 100341,404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: sl@ifi.unizh.ch
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Toronto SIG	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada Email: info@torontocapl.org
Stephen Wynn		8 Clarence Gardens, Brighton, Sussex BN1 2EG, UK. Tel: 01273-327238 Email: centre@cwcom.net
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

FTP SITES

IBM APL2
Waterloo Archive
APL-to-ASCII

ps.boulder.ibm.com/ps/products/apl2/
archive.uwaterloo.ca/ftparch/languages/apl
archive.uwaterloo.ca/languages/apl/workspaces/aplascii

WORLD WIDE WEB SITES

ACM SigAPL	www.acm.org/sigapl/
Adapta Software	www.adapta.de/
Adatum Software	www.adatum.com/
AFAPL	<u>www.ensmp.fr/~scherer/angle/</u> (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.demon.co.uk/apl385/
APL Journal, Germany	www.rhombos.de/rb/apljourn.htm
AUSCAN	www.interlog.com/~rjp/auscan/
Eke van Batenburg	www.bio.LeidenUniv.nl/~Batenburg/index.html
Capital PC User Group	http://cpcug.org/
Causeway	www.causeway.co.uk/
CODEWORK	www.inrete.it/cdwk/eng/homee.html
COSY (Bob Armstrong)	www.cosy.com/
Dinosoft Oy	www.dinosoft.fi/
DMOZ - Open Directory	http://dmoz.org/Computers/Programming/Languages/APL/
Dyadic Systems Ltd	www.dyadic.com/
DynArray	www.dynarray.com/
Eventra	www.eventra.com/
FinnAPL	www.pyr.fi/apl/
Godin London Inc	www.godin.com/
Hoekstra Systems	www.khamsin.demon.co.uk/about/hsl/noframe.html
IBM APL2	www.torolab.ibm.com/ap/apl/apl2.html
Infostroy	www.insight.dk/infostroy/
Insight Systems ApS	www.insight.dk/
Iverson Software Inc	www.jssoftware.com/
Japan APL Association	www.naska.co.jp/JAPLA/
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo.com/
Mackay Kinloch	www.users.globalnet.co.uk/~akinloch/akinloch.htm
MicroAPL Ltd	www.microapl.co.uk/
Oasis b.v.	www.oasis.nl/
Renaissance Data	www.aplbooks.cnchost.com/
RE Time Tracker Oy	www.rett.fi/
The Rochester Group Inc.	www.rochgrp.com/
Shepp & Associates	www.digitravel.com/
SigAPL	www.acm.org/sigapl/
Soliton	www.soliton.com/
Strand Software Inc.	www.jssoftware.com/
Rex Swain	www.rexswain.com/
Toronto SIG (for Toolkit)	www.torontoapl.org/
Jim Weigang	www.chilton.com/~jimw/

Zark Newsletter Extracts

Introduced by Jon Sandles

A few Vectors back we asked you to write FNTREE which displays a tree diagram of function calls. Following Anne Wilson's letter concerning the recommended solution to the FNTREE problem, and in particular the use of recursive calls, I asked her to send through a looping solution:

```

v RC+TEST
[1] RELATIONS+12 2p1 7 1 1 10 1 1 2 2 2 2 1 2 3 2 4 2 5 5 6 7 3 8 4 8 5
[2] RELATIONS+RELATIONS,[1]9 2p8 5 7 4 7 6 7 8 7 5 7 9 9 10 1 11 11 4
[3] RELATIONS+RELATIONS,[1]4 2p11 12 1 13 2 4 2 5
[4] NAMES+8 6p'EMPLS SELECTCINPUTIF MSG UNLESSADDEMPNINPUT'
[5] NAMES+NAMES,[1]5 6p'CATEMPRCAT DELEMO SQZEMPLSTEMP'
[6] RC+RELATIONS DOING NAMES

v
vDOINGv
v RC+RLTNS DOING NAMES;BOX;FIRST;I;J;L;LEV;NGI;PC;RCX;S;SH;SUB;T;TERM;
USED;V0;V1;XPND
[1] a Anne D Wilson June 1999
[2] a
[3] a First order nodes into correct printing order
[4] a Use their LEVEL to convey information
[5] RC+0
[6] USED+((1pRLTNS)p0 a To keep track of nodes already processed
[7] LEV+1p1 a Start with root node's data already there
[8] PC+1p1 a or wherever the node's parent is in the name list
[9] L+1 a Find nodes at level L+1
[10] @10:
[11] NGI+RLTNS[;1]e(LEV=L)/PC a NGI - indices of Next Generation
[12] NGI+NGI^USED
[13] USED+USEDvNGI a Only expand subtrees once
[14] NGI+NGI/1pNGI
[15] NGI+NGI[&PC;RLTNS[NGI;1]]
[16] I+((pPC),PC;RLTNS[NGI;1]) o I-I[&I] o XPND+I^1+0,I
[17] LEV+XPND\LEV o LEV+LEV+((-XPND)*L+1 a Set up level numbers
[18] PC+XPND\PC o PC+PC+((-XPND)\RLTNS[NGI;2]
a Must slot in the correct order
[19] L+L+1
[20] -((pPC)<1+1pRLTNS)/@10 a Loop until all input processed
[21] a
[22] TERM+LEV+1+LEV,0 a Terminal nodes
[23] FIRST+(PC;PC)=1pPC a First occurrence of nodes
[24] a
[25] a Picture - Boxes + lines
[26] BOX+4 14
[27] RC+4p(((pPC),BOX[2]-1+1pNAMES)p' | -| '),[2]NAMES[PC;]
[28] RCX+((pPC),BOX[2])pBOX[2]+' |-----|'
[29] I+(TERM^FIRST)/1pTERM o RC[I;BOX[2]+1 0]+((pI),2)p'+
[30] I+(FIRST^TERM)/1pTERM o RC[I;BOX[2]+1 0]+((pI),2)p'--'

```

```

[31] RC+((BOX[1]*pPC),BOX[2])pRCX,RC,RCX,((pPC),BOX[2])p' '
[32] I+pRC+(-BOX[2]*BOX[1]/LEV-1)q((BOX[2]*s/LEV)+BOX[2]p1)~[2]RC
[33] a Vertical lines
[34] SH+pS+((s/LEV),(pLEV))pLEV) o S+,(S-q(φSH)p1s/LEV)
[35] J+,φSHp(1=S)*(SUB++\0=S) a J has positions of children ie lev+1
[36] V1+J1:s/J a Youngest
[37] V1+SH+(V1≤pJ)/V1 a of many
[38] V1[2;]+(SH[2]+1)-V1[2;]
[39] V1+SH+V1
[40] V0+SH+SH[2]+SUB:SUB[V1]
[41] V1+1+(SH+V1-1)[2;]
[42] S+(1+BOX[1]*V0[2;]-1)+(1+pRC)*~1+BOX[2]*V0[1;]
[43] T+1+4*V1-V0[2;]
[44] S~T/S~~1+0,+~T o S+S+1pS
[45] RC+,qRC o RC[S]+~'~' o RC+0 2+q(φI)pRC

```

v

In this following extract we build on FNTREE to produce a genuinely useful utility.

LIMBERING UP

Identifiers Within Functions

(The purpose of this column is to work some flab off your APL midsection. Like muscles, your APL skills can atrophy if not exercised with adequate frequency and variety. This column presents a task for you to perform. Set aside a few minutes from your busy schedule and work the task. Mail in your solution and stay tuned for the results.)

In last issues's LIMBERING UP column, we introduced a utility function name *FNTREE* that displays a tree diagram of the subfunctions (and their subfunctions) called by a specified function in the workspace. The right argument of *FNTREE* is the name of the function to be diagrammed:

```
FNTREE 'EMPLS'
```

The result is a character matrix of the tree diagram. (See the UTILITY CORNER column in this issue for an illustration.) Last issue, you were assigned the task of writing the logic to construct the tree diagram. This issue, you'll be given the task of analysing a function to spot the names of the subfunctions called by that function. Next issue, we'll ask you to put it all together and finally write the *FNTREE* function.

The problem of determining the subfunctions call by a specified function is not as simple as it might first appear.

Consider this function:

```

      v IND←CHOICES SELECT PROMPT;R
[1]   A Ask for choice:
[2]   ASK:R←CINPUT PROMPT
[3]   A Search choices for 1st char of response:
[4]   IND←CHOICES\1+R
[5]   A Ask again if not a valid choice:
[6]   →ASK IF(IND>pCHOICES)MSG'INVALID CHOICE.'
      v

```

What are the subfunctions of *SELECT*?

The subfunctions are *CINPUT*, *IF*, and *MSG*. How do you know?

Your eyes scan the lines of the function, starting with line 1. Any name within comments is ignored (e.g. *Ask for choice*). Any name within quotes is ignored (e.g. *INVALID CHOICE*). Any label is ignored (e.g. *ASK:*). Any name that is local is ignored (e.g. *IND* or *PROMPT*). Incidentally, a name is considered local if it's the result variable (e.g. *IND*), an argument variable (e.g. *CHOICES* or *PROMPT*), a localised variable (e.g. *R*), or a label (e.g. *ASK*).

Any name remaining after all of this ignoring *may* be a subfunction. It may also be a global variable. To find out for sure, you need to look at the names of the other functions in the workspace to see if there's a match.

Sounds simple? Not really.

Here's your task: Write a function named *ANALYSE* that accepts a character vector function name and analyses the identifiers within the function specified. (The term "Identifier" is the fancy word for any variable name, function name, or label name.) In particular, the *ANALYSE* function should create six global variables, named *FNS*, *RESULT*, *ARGS*, *LOCALS*, *LABELS*, and *IDS*.

No, we haven't forgotten our resolve to avoid using global variables. The *ANALYSE* function is not a utility function we're going to save for posterity. It's just a temporary repository for algorithms you'll be needing next issue. Don't worry. Be happy.

Each of the six global variables are character matrices with one name per row. Short names are left-justified, i.e. padded to the right with blanks.

The variables contain the following names:

<i>FNS</i>	the names of the functions in the workspace
<i>RESULT</i>	the name of the result variable (0 rows if none)
<i>ARGS</i>	the names of the argument variables (0, 1 or 2 rows)
<i>LOCALS</i>	the names of the variables localised in the header, i.e. preceded by ' ; '
<i>LABELS</i>	the names of the labels, i.e. the names beginning a function line and followed immediately by ' : '
<i>IDS</i>	the names of all identifiers within the body of the function (all lines but the header), ignoring names within comments or quotes; you may include or ignore label names, at your whim.

All six character matrices must have the same number of columns, namely as many as the longest name in *FNS*. Any longer name should be omitted, since it couldn't possibly be the name of a subfunction. Names that begin with the $\square$ symbol, such as $\square IO$ or $\square PW$, should be omitted. You don't want to include system variables or system functions in the list of subfunctions.

Here's an illustration of the *ANALYSE* function:

```

)FNS
ANALYSE  SELECT
)VARS
                                     No variables
ANALYSE  SELECT
)VARS
ARGS  FNS  IDS  LABELS  LOCALS  RESULT
      ARGS
CHOICES
PROMPT
      FNS
ANALYSE
SELECT
      IDS
ASK
R
CINPUT
PROMPT
IND
CHOICES
R
ASK
IF
IND
CHOICES
MSG
      LABELS
ASK

```

```

LOCALS
R
RESULT
IND

```

To write the *ANALYSE* function, you need to know about a few system functions, namely *PCR* and *NL*. The system function *PCR* (Canonical Representation) takes a character vector function name as its right argument and returns a character matrix of the function's contents, less line numbers, dels (v), and cosmetic indenting:

```

PCR'SELECT'
IND←CHOICES SELECT PROMPT;R
A Ask for choice:
ASK:R←CINPUT PROMPT
A Search choices for 1st char of response:
IND←CHOICES\1+R
A Ask again if not a valid choice:
→ASK IF(IND>pCHOICES)MSG'INVALID CHOICE.'

```

Reprinted with kind permission from Zark APL Tutor News, a quarterly publication of Zark Incorporated, 23 Ketchbrook Lane, Ellington, CT06029, USA

Vector Back Numbers

Back numbers of Vector are available from:

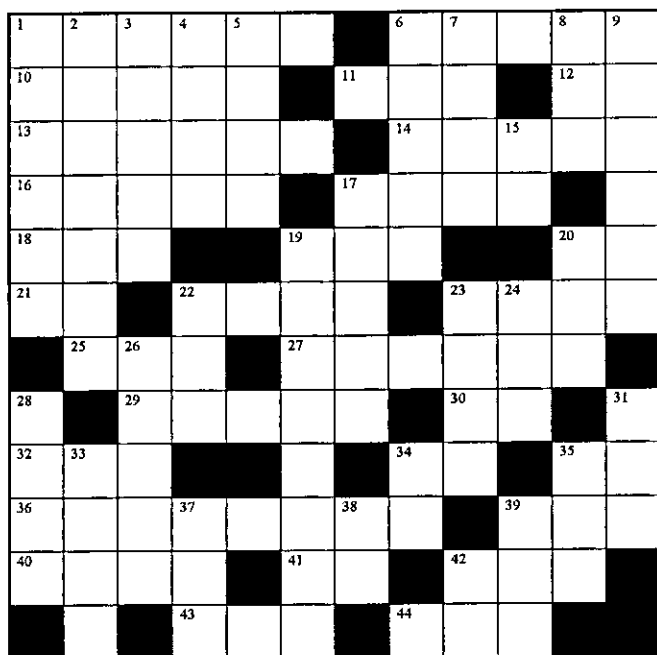
British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

Solution to Crossword from Vector 16.1

1	(	A	<	0	)	^	.	v	A	>	1	0			
11	M	+	+	\	S		12	6	/	B		13	1	€	
14	^	.	>	0			15	B	^	.	=	V			
16	.	5	x	p			17	2	≠	S		18	B	≠	
	=			19	A	x	21	1	+	K		22	S		-
23	V	x	V		26	p	p	A		27	⊗	M	⌈	1	
29	)	S	Y	30	M	B	O	L	S		32	p	/	+	
	/		33	v	/	~	Z		34	A	Z	(	÷	-	
36	1	€	B	≠	0		37	÷	V	9	+	V	9		
	†		38	I	M		39	N	,	E	9	≠	M	9	
40	p	X		42	/	(	p			44		M		.	
45	M	v	.	≠	(	1	+	p	M	)	†	V			

Crossword



Across

1. Are all elements of the vector A negative or greater than 10?
11. The running balance, given the starting balance M and the vector S of deposits and withdrawals.
12. Expand the matrix B by repeating each column six times
13. Any
14. All positive
15. $B \approx V$ for vectors B and V
16. Half the length of
17. $\sim 2 \approx S$
18. B isn't
19. The value of A in one year at annual interest rate K (e.g. 0.07)
23. $V \star 2$
26. The number of dimensions of A
27. The natural log of M, but avoid a negative result or an error message

29. How many variable, function or label names have been used?
32. $3\rho'\rho', \&2\ 2\rho' /\backslash \downarrow +'$
33. Are any of Z false?
34. $(20\rho0\ 0\ 1\ 0) / '(MATX+Z)*-(TAB\div K)\times^{-}3'$
36. Does the vector B contain any nonzeros? (Compare to 20 Down)
37. $1\div V9\times 2$
38. A good variable name for a matrix of whole numbers
39. Extract the rows from matrix M9 corresponding to 1s in bit vector E9, and precede each row by the scalar N
40. The length of the vector X
42. $2\phi^{-}3+7\rho'\rho /\backslash) ('$
44. The magnitude of M
45. Flag the rows of the matrix M that do not match the vector V (padded with blanks or truncated to conform with M)

Down

1. The indices in the matrix M of the rows that match the vector V (but omit the ninth symbol from the expression)
2. The first step to round A
3. $1\phi\phi'>+<\times'$
4. 0 if A is numeric, blank if character.
5. What)SAVE,)SI, and)SYMBOLS have in common
6. $\div 5\div 3$
7. Are any of the elements of the vector B not the constant K?
8. The product of Booleans AB and S
9. $0=B-11$
10. Does the sorted vector V, which does not contain $\bar{99}$, contain duplicates?
17. The integer AL, incremented twice
20. Does the vector B contain any nonzeros (Compare to 36 Across)
21. The first element in the array OZ
22. The denominator, with redundant parentheses, of the expression that scales the values of the matrix M so each column adds to 1 (given $SM+\rho M$).
24. $S>0$, for nonnegative S
25. Return bit vector BI if $VY=0$, or all 1s if $VY=1$
28. The largest reciprocal in the vector VM
30. $6\rho\phi' (A\neq B)/M' -' (AB)'$
31. What to do often when power failures are common
35. The remainder of $M\div Z99$
37. $' \div \times + - ' [? \square IO] , , ' , '$
39. N/1 for scalar N
41. The ____cist eliminates APL demons
43. The square root of one-third the sum of A and B (The first two symbols)

J-ottings 22: ... and now something for our very youngest readers

by Norman Thomson

The other day I came on my four year old grandson at play with J — no, not checking out Gene McDonnell, that would be having too many of the good things in life too early — what I mean is that while playing with large plastic letters of the alphabet, I found him busy fitting and refitting J into a square frame by turning it round through right angles, picking it up, turning it over and so on. So we got together and while he reflected with J, I too reflected with J. The grandfatherly view of J, by the way, is

```
]J=.4 4 $'  *  ** * **'
*
*
*  *
**
```

We started by turning J around its horizontal middle line:

```
(X=:|. )J
**
*  *
*
*
```

x means reflect in the x axis. In J vocabulary |. is called *reverse*, which is just what happened, that is a stack of four character vectors was reversed.

Next we turned J about the North-west/South-east axis:

```
(Q=:|:)J
*
*
*
***
```

Now what would happen if we did an X and then a Q, (or as Grandad says a Q with an X) ?

```
(S=: Q&X) J
```

```
*
*
*
***
```

... or for that matter a Q and then an X :

```
(R=: X&Q) J
```

```
***
*
*
*
```

(At this point I noticed that grandson had turned J within the frame by 90 degrees, first clockwise, and then anti-clockwise.) Only a few more tricks left:

```
((Y=: X&.Q) `(P=: Q&.X))/. J
```

```
*
*
* *
**
***
*
*
*
```

Just two things left now to complete the repertoire. First a half turn, for which either R or S could be applied twice (e.g. $H=: R\wedge 2$), or X and Q can be continued with as atomic items

```
(H=: X&. Q&X) J
```

```
**
* *
*
*
```

... and secondly a "do nothing" operation $I=: [\text{ or } I=:]$.

There are many alternative ways of defining these eight verbs. For example, using the rank conjunction, reverse "at rank 1" means at vector level so that each member of the stack of four vectors is separately reversed instead of the stack as

a whole, which results in a Y reflection. The full set of eight transformations are given in the table below with some of the many possible variants which include the equivalence of & and @ in this context.

	alternative	meaning	algebra
I=: [	NB.]	identity	
H=: X&.Q&X	NB. X"1@X	half-turn	H=YX=XY
X=: .	NB.	refln in 0x	X=QS=YH
Y=: X&.Q	NB. X"1	refln in 0y	Y=QR=XH
S=: Q&X	NB. X"1@Q	rotn pi%2 clockwise	S=QX=YQ
R=: X&Q	NB. Q@X"1	rotn pi%2 a-clockwise	R=XQ=QY
P=: Q&.X	NB. X&R	rotn in x=y	P=XS=YR
Q=: :	NB.	rotn in x=-y	Q=YS=XR

The column headed "algebra" contains some equivalences in which the conjunction & has been elided. A full composition table for the conjunction & applied to the eight verbs is

	I	R	H	S	X	P	Y	Q
I	I	R	H	S	X	P	Y	Q
R	R	H	S	I	P	Y	Q	X
H	H	S	I	R	Y	Q	X	P
S	S	I	R	H	Q	X	P	Y
X	X	Q	Y	P	I	S	H	R
P	P	X	Q	Y	R	I	S	H
Y	Y	P	X	Q	H	R	I	S
Q	Q	Y	P	X	S	H	R	I

At this point I left grandson experimenting with other fascinating objects such as A and K, while I perused the above composition table for inherent structures.

First it is an example of something which mathematicians call a *group*, or more fully a group of order eight since the body of the table contains just the basic eight distinct elements (this is called "closure"). But let's cut the technicalities and look for structures within the structure. To start with, the first four rows and columns themselves form a group of order four, which is structurally identical (isomorphic) to the addition table for addition in arithmetic modulo 4 (clock arithmetic to those of you youngsters who aren't in bed yet!). This table is:

c=:	+/~@1.	NB.	Cyclic group of order y.
c 4			
0	1	2	3
1	2	3	0
2	3	0	1
3	0	1	2

which can be transcribed into subsets of the 8 by 8 table by

(c 4){ each 'IRHS'; 'XPYQ'

IRHS	XPYQ
RHSI	PYQX
HSIR	YQXP
SIRH	QXPY

The first of these is a subgroup of the full table but the second is not a subgroup because it has no identity element (that is an element with the property that $Ia = aI = a$ for all elements of the subgroup). The two bottom quadrants have row shifts which go in the opposite direction to c 4. A general algorithm for this uses the "minus" table for i. n, two hooks, and modulo n arithmetic:

ac=:|(+/-@i.) NB. Anti-cyclic table of order y.

ac 4

0 3 2 1

1 0 3 2

2 1 0 3

3 2 1 0

(ac 4){'XPYQ' gives the bottom left hand quadrant of the table, and (ac 4){'IRHS' gives the bottom right hand quadrant. These two expressions can be merged as (4+ac 4){'IRHSXPYQ', and the indices in the string 'IRHSXPYQ' of the cells in the four quadrants are, in clockwise order, (c 4), (4+c 4), (ac 4) and (4+ac 4). These define the non-symmetric group which is known as the dihedral group of order 4. An adverb based on a hook which adds the "tally" of a matrix to itself ("tally" for a matrix is simply the number of rows) leads to the following general definition of dihedral groups:

a2n=: (+#)@

di=: (c, .c a2n), ((ac a2n), .ac) NB. adverb : adding 2 to the power n

di 4

0 1 2 3 4 5 6 7

1 2 3 0 5 6 7 4

2 3 0 1 6 7 4 5

3 0 1 2 7 4 5 6

4 7 6 5 0 3 2 1

5 4 7 6 1 0 3 2

6 5 4 7 2 1 0 3

7 6 5 4 3 2 1 0

and (di 4){'IRHSXPYQ' generates the 8 by 8 composition table given above.

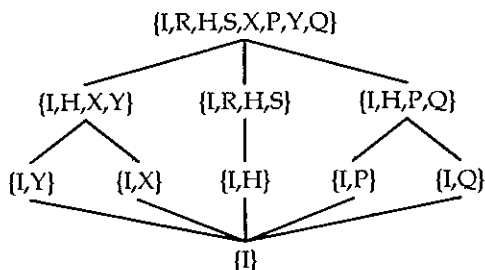
The table of I, H, X and Y rotations only is simply the dihedral group of order 2 (D2).

```

      (di 2) {'IHX Y'
IHX Y
HIY X
XYI H
YXH I

```

and the set of all possible subgroups is exhibited in the following lattice diagram in which a connecting line indicates that the lower node is a subgroup of the higher one.



In general the dihedral group D_n describes the symmetries of an n -sided regular polygon. For example

```

di 3
0 1 2 3 4 5
1 2 0 5 3 4
2 0 1 4 5 3
3 4 5 0 1 2
4 5 3 2 0 1
5 3 4 1 2 0

```

gives the non-symmetrical group of symmetries of the equilateral triangle. 0 1 and 2 correspond to rotations by 0 , $\pi/3$ and $2\pi/3$, and 3, 4 and 5 to reflections in the three perpendicular bisectors of the sides.

Relationships exist between the grade operations, $/:$ and $\backslash:$ applied to permutation vectors and the transformations IHXYRSPQ.

To demonstrate this here are two verbs which convert permutation vectors to matrices and vice versa:

```
pmfrom=:=/~ i.@#      NB. Transforms permutation vector to matrix
pmfrom 1 3 2 0
0 0 0 1
1 0 0 0
0 0 1 0
0 1 0 0

pvfrom=:+/. *~i.@#    NB. Transforms permutation matrix to vector
pvfrom pmfrom 1 3 2 0
1 3 2 0
```

A dihedral composition table for grade verbs can be constructed with names for the operations on permutation vectors matching exactly the one for the analogous matrix transformations, that is if pv is a permutation vector then $q\ pv$ is equivalent to $pvfrom\ q\ pmfrom\ pv$, and similarly for the other seven verb pairs. It turns out that it is s and q which are primitives under this scheme, and also for permutation vectors, y is equivalent to $|\ .$

		with grade verbs	some alternatives
i=: [	NB.		
h=: s^:2	NB.	$\backslash:\&\backslash:$	$y&x$
x=: q&s	NB.	$/:\&\backslash:$	$x&h$
y=: s&q	NB.	$\backslash:\&/:$	$ \ .$
s=: \:	NB.	$\backslash:$	$y&q$
r=: s&.q	NB.	$\backslash:\&\backslash:\&\backslash:$	$q&y$
p=: q&.s	NB.	$/:\&\backslash:\&\backslash:$	$s&y$
q=: /:	NB.	$/:$	$y&s$

I trust that I have managed to affirm my belief that *Vector* should be committed to including something for even the youngest reader. And now it's off to Mothercare to buy a set of these plastic letters

Array Based Naïve Bayesian Machine Learning Program: A J Implementation

by Andrew Charman-Williams (A.CharmanWilliams@utas.edu.au)

Description of Problem

Machine Learning

The data to be processed within a machine learning program consists, normally, of several tables of values. This potentially lends itself to array based processing rather than the more common incremental approach. The naïve Bayesian approach used has been found to be quite effective within certain domains. The particular implementation handles nominal domains with the nominal values of attributes and classes assumed to be consecutive integers from 0.

Naïve Bayesian Learning

The naïve Bayesian approach to machine learning is based on the extension to classical probability made by Thomas Bayes in the 18th Century. The aim is to use the class with the highest probability given a particular instance. This is represented as $P(C_i|I)$ where C represents the possible classes and I is the instance being classified. Bayes theorem gives us the formula $P(C_i|I) = P(C_i)P(I|C_i)/P(I)$. As $P(I)$ is the same for all possible classes this does not need to be calculated as the relative size of probabilities is sufficient for classification purposes. In order to calculate $P(I|C_i)$ the naïve Bayes approach assumes independence of attributes. Although this simplification may be based on invalid assumptions for some datasets it simplifies the process for the calculation of $P(I|C_i)$. This allows us to use the product of the probabilities that I is of the class C_i given the value v_j in order to calculate the probability $P(I|C_i)$.

Thus the approach used calculates the probability of class C given instance I as being:

$$P(C_i|I) = P(C_i) \prod_j P(v_j|C_i)$$

This leaves the problem of obtaining the values for $P(C_i)$ and $P(v_j|C_i)$. In order to calculate these this particular implementation uses a Laplace measure to estimate the probabilities with a numerator correction of 1 and a denominator correction as

the number of alternative values. Thus to determine the probability of the class we use:

$$P(C) = \text{number of records of class } C + 1 / \text{total records plus number of classes}$$

and for the probability of attribute j having a specific value x given class C we use:

$$P(v_j|C) = \text{number of items of class } C \text{ with attribute } j \text{ of value } x + 1 / \text{number of items with class } C + \text{number of alternative values of } j.$$

These estimated probabilities from the Laplace measures are inserted into the naïve Bayes equation in order to find $P(C_i|I)$. The instance is classified as the class with the highest probability rating.

The Laplace Measure

The Laplace measure is utilised in the calculation of the attribute and class probabilities. The Laplace measure consists of adding one to the numerator and the number of values to the denominator (in this case either the number of different classes or the number of attributes).

Overview of Algorithm

Learning

The learning phase consists of several steps:

- Read in the learning and description files and convert them to appropriate formats.
- Get overall information (number of records, number of attributes etc.) from learning and description files.
- Count the number of occurrences of each class in the learning data.
- Calculate the base probabilities of the occurrence of each class using the Laplace measure as detailed above. NB These are only relative probabilities as noted earlier.
- Count the number of occurrences of each value of each attribute for each class.
- Calculate the probabilities of the occurrence of each value of each attribute given each class using the Laplace measure.

Testing

The steps for the checking of the test data are:

- For each record of the test data the probabilities for the values of each attribute are obtained for each possible class.
- The Bayesian equation is applied to each record to find the class with the greatest relative probability.
- Each record is classified according to the best value from the equation.
- The classification of each record is compared with the actual class value recorded in the test data and the subsequent accuracy percentage is evaluated.

Classification

The classification procedure is the same as the initial steps of the testing procedure. The only difference in this phase is that no comparison is made with actual class values (as they do not exist in the classification data).

A J Implementation

Assumptions & File Format Requirements

Like most Machine Learning implementations fairly strict assumptions are made regarding the format and existence of the necessary data.

Filebase

The filebase (the base name and location of the files) are given in the command line when running the program. This filebase must be exactly the same for all required files. The filebase should include information regarding which drive and folder in a format suited to the operating system. (e.g. 'Macintosh HD:waves' for MacOS or 'a:\waves' for DOS/Windows)

Description file (filebase.des)

This file consists of one line for each attribute and a final line for the class. Each line contains the numbers (consecutive integers from 0) of the possible values for each attribute or class. These are separated by a comma. Unlike other similar machine learning programs the lines are not to begin with a name or type of attribute/class.

Learning data (filebase.dat)

The learning data file consists of each record on one line with the values separated by commas. The order of these should match the order in the description file. The last item on each line is assumed to be the class value.

Test data (filebase.tst)

The format for the test data is exactly the same as that of the learning data.

Classification data (filebase.cls)

The format for the classification data is the same as that for the learning and test data except that it will not contain a class value (therefore each line will have one less value than the other two files).

The learning, testing and classification data can contain differing numbers of records. The maximum number of records is limited by memory requirements due to the potential size of arrays with large number of attributes, classes and records.

J Implementation**Basic and Common functions**

Firstly some basic functions that are used at various places in the program are defined.

NB. Check readme file for required files

NB. and main function calls

NB. Basic or common functions

`read` is used to load the four sets of data.

`sum` gives the sum over the array at rank 1.

`get` is used to strip the frets from the incoming file and open the data as a string table (this is then converted at different times to an table of integers).

`from` is used to extract values from a table or array at rank 1.

`grd` is used to obtain the index (as 0) of the highest probability from the possible classes.

```
NB. Basic loading function
read=:1!:1@<
NB. Sum array at rank 1
sum =: +/ "1
NB. strip frets and open as string array
get =: > @ <:._2
NB. From at rank 1
from=: {"1
NB. Get grades of array (index of sorted array)
grd =: ( i. ~/:~)
```

The Learning Phase

The first of the major sections of the program is to learn from the learning data. This section consists of four subsections.

NB. The first major section is the learning from data

The first subsection consists of loading the learning data, converting it to a numeric array and extracting some basic information from the array.

lrnfile adds the appropriate extension (.dat) to the filebase.

lrndata loads the learning data using the read function, strips the frets using the get function, opens it to a string table and then converts the string table to a numeric table.

swapped swaps the dimensions of the table in preparation for further processing.

class extracts the line containing the class information from the learning array.

atts then removes the class data from the learning array.

recno obtains the number of records (instances) in the learning data.

inclrndata increments the values in the learning array. This is necessary because when the array is converted to a 3 dimensional array, separated on classes, the array is padded by zeros. This avoids the problem of class 0 matching on non-existent records.

lrnclsdata extracts the class data from the incremented learning array ready for later matching.

```
NB. Read learning data, get basic information
NB. and prepare for processing
```

```

NB. Add file extension to filebase
lrnfile=: ,&'dat'
NB. Load learning data, strip frets and convert to array
lrndata=: ". @ get @ read @ lrnfile
NB. swap dimensions of array of learning data
swapped=: |: @ lrndata
NB. extract class data from learning array
class=: {: @ swapped
NB. remove class data from learning array
NB. leaving attribute data for processing
atts=: }: @ swapped
NB. Number of records in learning data
recno=: # @ lrndata
NB. Increment data to remove problem of zero values
inclrndata=:>: @ lrndata
NB. Extract incremented class data from learning data
lrnclsdata=: attno from inclrndata

```

The second subsection consists of loading the description data, converting it to a numeric array and extracting some basic information from the array.

`descfile` adds the appropriate extension (.des) to the filebase.

`getdesc` loads the description data using the `read` function, strips the frets and opens it to a string array. Unlike the learning data this can not be immediately converted to a numeric array as the class information needs to be removed first to prevent possible corruption of values extracted from the data.

`classdesc` extracts the class description information from the description table.

`classno` obtains the number of different classes.

`attno` obtains the number of different attributes.

`attvals` removes the class information and converts the attribute information to a numeric table.

`maxvals` obtains the largest number of values for an attribute.

```

NB. Read description data, get basic information
NB. and prepare for processing

```

```

NB. Add file extension for description file
descfile=: , & '.des'
NB. load description file into array
getdesc=: get @ read @ descfile
NB. extract class description from description array
classdesc=: ". @ {: @ getdesc
NB. Get number of different classes
classno=:# @ classdesc
NB. get number of different attributes
attno=: <: @ # @ getdesc
NB. get maximum number of values in attributes
attvals=: ". @ }:@ getdesc
maxvals=:# "2 @ |: @ attvals

```

The third subsection processes the learning data with the information gained from the learning and data files in order to obtain the base class probabilities.

```

NB. Process learning and description data
NB. to get class probabilities

```

classct obtains a list of the number of occurrences of each class in the learning data.

classprob1 increments the count of the number of occurrences of each class. This is the numerator correction of the Laplace measure.

classprob2 adds the number of different classes to the number of instances in the learning data. This is the denominator correction of the Laplace measure.

classprob calculates the base (relative) probabilities of the occurrence of each class from the learning data.

```

NB. Get count of number of occurrences of each class
NB. in learning data
classct=:sum @ (classdesc =/ class)
NB. Calculate base class probabilities from learning data
classprob1=: >: @ classct
classprob2=: recno + classno
classprob=: classprob1 % classprob2

```

The fourth subsection processes the learning data with the information gained from the learning and data files in order to obtain the attribute probabilities.

```

NB. Process learning and description data
NB. to get attribute probabilities

```

attvalno finds the placement of highest value for each of the attributes by grading down and matching on zero (the highest element).

attvalno2 takes the list from attvalno and, by multiplying against the attribute value information, obtains the highest value for each attribute.

attvalno3 sums over attvalno2 to convert it back from a table to a list and then increments to obtain the number of different values for each attribute.

lrcnclsatts matches the class values for each instance.

lrcnclsatts2 uses the matching from lrcnclsatts to convert the table of attribute data to a 3 dimensional array (of class, instances and attributes) divided according to class values and removes the class data leaving only attribute data.

attprep gives a binary match of each element against the maximum number of different attribute values.

attcounts sums the information from attprep giving a 3 dimensional array (of value, class and attribute) containing the number of occurrences of each value for each attribute and class.

attprobs1 provides the sum of the number of values for each attribute with the number of occurrences of each class. This is the denominator correction of the Laplace measure.

learn, which is the first of the primary function calls, calculates the relative Bayesian probabilities, using an increment for the Laplace numerator correction. The array of attribute probabilities needs to have its dimensions rearranged to allow for calculation at the appropriate rank.

```
NB. Get number of values for each attribute
attvalno=:0&= @\: "1 @ attvals
attvalno2=: attvals * attvalno
attvalno3=: >: "0 @ sum @ attvalno2
```

```
NB. Divide attributes according to class
lrcnclsatts=: >:@ classdesc =/ lrcnclsdata
lrcnclsatts2=: inclrndata }:"1 @ #~ lrcnclsatts
```

```
NB.Count number of occurrences of each attribute
NB. within each class
attprep=: >:@i.@maxvals =/ lrcnclsatts2
attcounts=:+/"2 @ attprep
```

```

NB. Calculate Bayesian probabilities using Laplace measure
attprobs1=:attvalno3 +/ classsct
learn=: (>:@ attcounts) %"2 (|: @ attprobs1)

```

The Testing phase

The second major section of the program is to check the accuracy of classification of the test data. This section consists of three subsections.

NB. The second major section is the testing of data.

The first subsection consists of loading the test data, converting it to a numeric table and extracting the class data from the table.

```

NB. Read testing data, get basic information
NB. and prepare for processing

```

testfile adds the appropriate extension (.tst) to the filebase.

tstdata loads the test data using the read function, strips the frets using the get function, opens it to a string table and then converts the string table to a numeric table.

testclass extracts the class data from the table in preparation for checking the accuracy of classification.

tstdata2 removes the class data from the table leaving only attribute data.

tstdata3 swaps the dimensions of the table ready for processing.

attprobs3 rearranges the dimensions of the array of attribute probabilities ready for extracting the matching probabilities.

```

NB. Add file extension to filebase
testfile=: ,&' .tst'
NB. Load test data, strip frets and convert to array
tstdata =: ". @ get @ read @ testfile
NB. Extract class data from array of test data
testclass=: {: "1 @ tstdata
NB. Remove class data from array of test data
tstdata2=: }: "1@ tstdata
NB. swap dimensions of array of test data
tstdata3=: |: @tstdata2
NB. swap dimensions of array of attribute probabilities
attprobs3=: |: @ learn

```

The second subsection consists of extracting the matching probabilities and classifying each instance according to the most likely class.

NB. Process testing data with learning probability data
NB. to get attribute probabilities and then classification

tstdtamtch extracts the probabilities from the attribute probability array that match the values of each instance.

tstdatamtchprod gives the naïve element of the Bayesian equation by giving the product of the probabilities of a particular value for each possible class.

tstdtaprobs finishes the Bayesian equation by the product of the class probabilities with the instance probability calculated in tstdatamtchprod. This results in a table (class and instances) of relative probabilities.

tstdtaprobsgrd grades the relative probabilities per class.

tstprobsbst matches the highest relative probability with the appropriate class.

tstclassify1 converts the matches of most probable class to the appropriate class values.

test, the second of the primary function calls, converts the table from tstclassify1 back to a list by summing the table. This gives a list of the class that each instance has been classified as by the program.

```
NB. extract matching probabilities
tstdatamtch=: tstdat3 from attprobs3
NB. Sum of matching attribute probabilities
tstdatamtchprod=: */ @ tstdatamtch
NB. Apply Bayesian equation
tstdtaprobs =: classprob * tstdatamtchprod
NB. Grade probabilities for each class and record
tstdtaprobsgrd=: grd "1 @ |: @ tstdtaprobs
NB. Get highest (most likely) class for each record
tstdtaprobsbst =: (<: @ classno) ="0 tstdtaprobsgrd
tstclassify1=: classdesc * "1 tstdtaprobsbst
test=: sum @ tstclassify1
```

The third subsection checks the accuracy of the classification of the test data.

NB. Check accuracy of classification of test data

check1 obtains the match of the classification of the instances with the classes recorded with the instances in the test data.

check2 obtains the number of instances where the classification matches the recorded classes.

tstno obtains the number of instances in the test data.

accuracy, the third major function call, obtains the percentage accuracy of the classification of test data against the recorded values.

```
NB. Number of records correctly classified
check1=: testclass = test
check2 =: +/ @ check1
NB. Number of records in test data
tstno =: # @ testclass
NB. Get percentage accuracy
accuracy=: check2 100 & * @ % tstno
```

The Classification Phase

The final major section of the program is the classification of new instances from the classification data file. This is the same as the testing phase except that no checking of accuracy is done. This section consists of two subsections.

NB. The final major section is the classification of new data

The first subsection consists of loading the classification data and converting it to a numeric table.

```
NB. Read classification data, get basic information
NB. and prepare for processing
```

clsfile adds the appropriate extension (.cls) to the filebase.

clsdata loads the classification data using the read function, strips the frets using the get function, opens it to a string table and then converts the string table to a numeric table.

clsdata2 swaps the dimensions of the table ready for processing.

attprobs3 rearranges the dimensions of the array of attribute probabilities ready for extracting the matching probabilities.

```

NB. Add file extension to filebase
clsfile=: ,&'.'cls'
NB. Load classification data, strip frets and convert to array
clsdata =: ". @ get @ read @ clsfile
NB. Swap dimensions of array
clsdata2=: |:clsdata

```

The second subsection consists of extracting the matching probabilities and classifying each instance according to the most likely class.

```

NB. Process classification data with learning probability data
NB. to get attribute probabilities and then classification

```

clsdatamatch extracts the probabilities from the attribute probability array that match the values of each instance.

clsdatamatchprod gives the naïve element of the Bayesian equation by giving the product of the probabilities of a particular value for each possible class.

clsdtaprobs finishes the Bayesian equation by the product of the class probabilities with the instance probability calculated in clsdatamatchprod. This results in a table (class and instances) of relative probabilities.

clsdtaprobsgrd grades the relative probabilities per class.

clsprobsbst matches the highest relative probability with the appropriate class.

clsclassify1 converts the matches of most probable class to the appropriate class values.

classify, the last of the primary function calls, converts the table from clsclassify1 back to a list by summing the table. This gives a list of the class that each instance has been classified as by the program.

```

NB. extract matching probabilities
clsdatamatch=: clsdata2 from attprobs3
NB. Sum of matching attribute probabilities
clsdatamatchprod=: */ @ clsdatamatch
NB. Apply Bayesian equation
clsdtaprobs =: classprob * clsdatamatchprod
NB. Grade probabilities for each class and record
clsdtaprobsgrd=: grd "1 @ |: @ clsdtaprobs
NB. Get highest (most likely) class for each record
clsdtaprobsbst =: (<: @ classno) ="0 clsdtaprobsgrd
clsclassify1=: classdesc * "1 clsdtaprobsbst
classify=: sum @ clsclassify1

```

Primary Calls & Usage

The following are the primary calls and their function:

```
learn 'filebase'
```

This call will learn the data and will display the attribute probabilities calculated from the learning data. Requires filebase.dat and filebase.des.

```
test 'filebase'
```

This call will learn and test the data and will display the classification of the test data. Requires filebase.dat, filebase.des and filebase.tst.

```
accuracy 'filebase'
```

This call will learn and test the data and will display the accuracy of classification of the test data. Requires filebase.dat, filebase.des and filebase.tst.

```
classify 'filebase'
```

This call will learn, test and classify the data and will display the classification of the test data. Requires filebase.dat, filebase.des, filebase.tst and filebase.cls.

In addition to the four primary calls, some other calls may prove useful in certain situations. One function that may prove of particular interest is:

```
classprobs 'filebase'
```

This call will learn the data and display the class probabilities calculated from the learning data. Requires filebase.dat and filebase.des.

Limitations & Evaluation

The performance of this program is obviously dependent on the computer being used and the size and complexity of the data files. The two sets of files used for testing consisted of: wave 6 21 attributes, each of seven values, with three possible class values and 200 instances in each of the three data files; and led 6 seven binary attributes, with ten possible class values and 200 instances in each of the three data files. On a PowerMac G3 running J version 4 the performance was acceptable for the two sets of files tested (approximately 2 - 3 seconds for any of the calls). On a 68040 based Macintosh running J version 4 the performance was considerably less acceptable (30 - 40 seconds for any of the calls).

The performance of the program may be less suited for particularly complex domains due to the size of some of the intermediary arrays. Refining of the implementation may improve some of these performance concerns.

The program does allow for a good degree of flexibility in its usage. As well as the four primary function calls some of the other function calls (which would not normally be directly available in an implementation of this algorithm) could prove potentially useful in the analysis of the classification accuracy of data.

Further Work

A variety of further development work would be worthwhile on this program.

- Refining the program with performance considerations specifically in mind.
- Modifying the program to handle numeric rather than nominal attributes, which would require an approach based on the probability density function for the normal distribution. Particularly in the case of allowing for a mixture of nominal and numeric domains at least a slightly more incremental, or alternatively recursive, approach would be necessary. This is due to the necessity of processing the attributes separately for at least part of the time. An overall array based approach (with the necessary incremental or recursive modifications) would be possible with the use of gerunds.

Bibliography

Langley, P. (1996) *Elements of Machine Learning*, Morgan Kaufmann, San Francisco.

Acknowledgement

Neville Holmes, School of Computing, University of Tasmania.

Andrew Charman-Williams
School of Computing
University of Tasmania

RECENT MEETINGS APL99 AT SCRANTON

This section of Vector documents APL gatherings worldwide. If you would like to act as a Vector "Roving Reporter", please contact any member of the working group to check what is required. Vector will pay all agreed reasonable expenses incurred in attending and reporting the meeting.

Thanks to Ray Cannon for the notes and pictures from Scranton.

APL 99 – On Track to the 21st Century

A personal review of the Annual Conference of ACM/SIGAPL held at Scranton, Pennsylvania, USA, August 10th to 14th 1999, by Ray Cannon.

Preamble

There is no way that I can condense a four plus day experience, involving over one hundred and thirty APLers from 13 countries, 15 papers, 4 vendor forums, several evening seminars, satisfactorily into the space available within Vector, so here is a flavour derived from some hastily taken notes, and a blurred memory.

Disclaimer. All quotes (attributed and un-attributed) are subject to errors, and are at best my hazy recollection of what I thought I heard someone say. I will therefore apologise in advance for any misquotes, or mis-attributed, and further, to save time (and space in the next edition of Vector), here retract any/every thing that follows.

Scranton: "The Historic Jewel of the Pocono Mountain Region" and the original "Electric City"



The City of Scranton is located to the North East of Pennsylvania, a 3-hour bus ride from New York City. For over 100 years, it supplied the US with its most important source of energy (anthracite coal), and was a steel and locomotive producer.

With the demise of these former industries, Scranton is no longer a growing city, nor can it still boast of having the highest per capita income in the USA. However, it's not suffering from "inner city decline", and possesses a friendly "small town America" feel along with some very beautiful houses and civic buildings. Scranton seemed to be

very good at "recycling" its buildings, that is changing their use without tearing them down.

As part of our APL99 handout, there was a "Restaurant Guide" which included 9 "Downtown" venues, and 4 others within walking distance from the conference. The number one restaurant was billed as "Coney Island – Texas Weiners (a Depression Era Hot Dog Stand). Alan Graham and I had a quick and enjoyable lunch there. I also enjoyed breakfast at "Abe's Delicatessen" where I had "Kosher bacon" with my eggs. (That should be good for at least one free Weiner and a bagel if I am ever back in Scranton. :-))

Scranton University Campus is located between downtown Scranton, the railway line and "The Hill" (where all the former railroad and mining barons built their houses and mansions). The University is not large (taking up about seven by two "blocks"), but appeared well equipped, and made a very pleasant venue.



Electric City? Well Scranton was the very first city to electrify its streets and to have electric powered trams. Sadly the trams have all deserted the streets and can now only be found in the Tram Museum.

The Reception

On the Wednesday evening, Dyadic Systems Ltd. and The Carlisle Group sponsored a delightful reception up on "The Hill". The location was actually the veranda of Paul Mansour's beautiful house. Yes I did say the veranda; it was large enough to fit the whole conference without too much overflow onto the steps leading up to the house. It was here that I was first introduced to Alan Graham's Zero – see *What's New* below.

The Banquet

The Banquet was held on Thursday night at the Lackawanna Station. (Let me explain, Scranton is the main city in Lackawanna County. Its railway station was built in the grand style and has recently been refurbished to its former glory and converted into a very opulent hotel, the "Lackawanna Station Hotel".)

In addition to the initial reception in the splendour of the marble-lined main hall, we were treated to a 3(4?) course dinner.



Then we were entertained by the excellent "Steamtunes Barbershop Quartet" (one of whose members is Steve Mansour, the APL99 Conference Chairman).

This was followed by the presentations of the SigAPL Outstanding Achievement Award, to Roy Sykes (1998) and Bill Rutiser (1999).



APL Berlin 2000

We were then invited to "APL Berlin 2000" which is being hosted at the Technical University of Berlin between 24th-27th July 2000, with Dieter Lattermann as the Conference Chairman.

Finally Walt Niehoff spoke to us about one of the passions of his life, the "Switch Back Gravity Railroad", an 18-mile roller-coaster.

The Tours

Three tours were organised as part of the conference.

A walking tour of Scranton

This tour took us from the Lackawanna Station to the Reception (see above). Highlights included the Courthouse Square, the Public Library, the Woolworth Building and Scranton's unique "Hill section".

Lackawanna Coal Mine Tour

Our journey took us 300 feet beneath the earth where we experienced the cold, dark caverns where "ordinary men (and boys) laboured under extraordinary conditions."

Steamtown National Historic Site/Steam train excursion

A nostalgic trip through beautiful countryside to Moscow and back.



What's New: 2 new APLs

In the Unix world, when a new implementation of an existing application is released, it is often given a bland name like YACC, short for Yet Another Compiler Compiler. Hence Soliton's newest APL might have been called YAAI (Yet Another APL Interpreter) but instead they announced it as SAX for Linux. (I still think "SEX for Linux" would have a wider appeal.)

How long before APL2000 and Dyadic respond with their own APL for Linux?

The other new addition to the APL family announced at APL99 was Alan Graham's "0" (that's a zero not "Oh"). It is still "vapour ware" but a demonstration version, running as a Dyalog workspace, was shown at APL99, and should be available from the Vector web site by the time this edition of Vector is distributed.

(Vector hopes to have full reviews of both of these APLs in the next edition.)

Featured Speaker: Professor Shiu-Kai Chin

Professor Shiu-Kai Chin of Syracuse University gave an excellent talk on "High-Confidence Design for Security".

Quotes:

"The only time I was able to get a program to work first time, ... was when I built a theorem prover, I did it in a modular fashion, and I used APL. I have never duplicated that experience in my life, and that was 25 years ago! Of course I was naive at the time thinking 'This is great, just think how much better it will get!'"

"The half life of knowledge engineering is about 5 years."

He briefly described some of the methods that establish the confidence that a system (both in design and implementation) will behave securely.

A HCS (High Confidence System) must withstand both internal and external threats, both natural and intentional (e.g. malicious).

Properties of security include:

Privacy – Who can read.

Integrity – Detect corruption of information.

Authentication – Know the identity of the principle or source of the information.

Access control – read / write / delete...

Non-repudiation – Can't deny a transaction.

Availability of service – A system must be available for use when its users require it.

Mechanisms of security:

Encryption – Hiding information in a reversible way.

Hash functions – Map arbitrary size information into a fixed size. One way only.

Digital signature

Certification – You know who I am from my driving licence because the "state" knows who I am and issued me with the licence, and you trust the state.

Authorisation – The ability to grant (or pass on) permission to do something.

Chin described several forms of encryption (Symmetric, Asymmetric, Public and Private Key) and went on to show some of the problems in implementing them

over the Internet. For example, how do you associate Keys with Principles? "Who do you trust?" "How do you know?"

After the talk, I asked Professor Chin if he personally trusted the Internet enough to buy things over it with his credit card. His reply was very interesting. Basically "Yes", but he first checked with his credit card company on his ability to deny a transaction: does the burden of proof lie on the supplier, or with the purchaser. This may vary with credit card companies, but in Professor Chin's case, it lay on the supplier. This was later borne out when 6 months after ordering, paying for and receiving a book over the internet (NOT through Amazon.com), he was then re-invoiced for a second copy of the same book. He had no problems from his credit card company when he denied the second transaction.

Featured Speaker: Professor Doug Lea

Professor Doug Lea of The State University of New York at Oswego. His talk was billed as "Object-Oriented Concurrent, Parallel, and Distributed Programming in Java", but the talk given was "Exploiting Java".

Rather than me misrepresenting Doug Lea's interesting talk, can I point you to the excellent slides he illustrated his talk with, available (as at 5 September '99) from

<http://gee.cs.oswego.edu/dl/cpjslides/exploiting.pdf>

(They require an Acrobat Reader version 3.0 or higher. Version 2.1 will *not* display them.)

I will however include a few of his many "one-liners".

"If you put a gun to my head, and gave me like a couple of days, I could probably figure out how to get the standard deviation of a vector of numbers in APL. I'm assuming that if I put a gun to your head, you could probably write a Java program that did something."

"When in doubt, make a thread, then think twice, then 3 times about such discussions."

"No Microsoft products were involved in the production of any of the software..."

"Your toaster-oven with a Java VM in it is a multi-threaded device."

Talking about a QUAKE server with 5000 open socket connections:

"Having 5000 threads handling 5000 connections, probably just won't work. It can be done, but it can't be done very fast."

During questions, Doug was asked: "What overhead was incurred, when you applied these multi-tasking techniques of 'divide and conquer' between multi cpu on a uni-processor device?"

His answer I found quite astounding, "Can be NONE". This, he explained, applied to a uni-processor with a *lot* of memory. The divide and conquer technique produces very good "memory locality", which in turn actually reduces system overheads far more than the cpu-cycles incurred by the extra code.

Plenary Session – The ZARK Library of Utility Functions

by Gary Bergquist



Although originally submitted and accepted as a paper, this was upgraded into a Plenary session and a Panel Response (with the audience as the "panel"). Gary then proceeded to give a talk on "The 14 Myths of APL" rather than present the paper (published in the proceedings).

Gary started out by declaring that he is a self-confessed APL fanatic.

The 14 myths of APL:

1. Ken Iverson invented APL

Was Calculus invented by Newton or was it always there and just discovered by him?

Gary then listed 10 things that defined APL for him:

- i) Immediate execution mode;
- ii) Scalar extension;

- iii) Dynamic allocation of memory;
- iv) Simplicity of function: e.g. Order of execution (Right to Left rather than Multiplication before Addition);
- v) Function syntax in user functions;
- vi) Operators;
- vii) Consistent treatment of "empties";
- viii) Localisation of variables;
- ix) Reliability after errors and breakpoints;
- x) User defined functions have the same syntax as primitives, hence you can extend the language.

"I believe when the Aliens come down to land, and exit their little spacecraft, they will have calculus, they will know that $E=MC^2$, and they will have APL, They might not call it APL, but we don't either."

2. APL is Dying

"APL is not dying, calculus is not dying, you can't kill something that was never born. APL has always been there, APL will always be there."

3. APL is Easy

4. APL is Hard

APL is not easy or hard. APL is ABSTRACT. If you find abstraction easy then APL is easy, if not it will be hard.

"I tried to teach my wife APL, 'I could've learnt it' she said, 'but I really have to wash my hair tonight'. Some people can not learn APL, and it is because it's so abstract."

5. APL is Unreadable

"You know what, Russian is unreadable, I don't know Russian, so it's unreadable. If you don't know it, it's unreadable."

6. APL is Unmaintainable

Because APL is unreadable, it's a write-only language, hence is unmaintainable.

7. Programming is a Skill not a Profession

Programming is a profession.

"You can't be an actuary and a programmer, you can't be an engineer and a programmer, you can't be a doctor and a lawyer, well you can but I don't want you operating on me, and I don't want you representing me in court!"

8. The job of a programmer is to make sure that a computer system behaves properly and gives the right answers

That's just a quarter of it. The job of a programmer is to build an application that you can live with and can be maintained.

APL systems written by non-professionals may not be maintainable. It works, it may give the right answer and has some value, but when you need to change it, its value is zero.

9. Each year computers become more important

Computers are becoming cheaper and less important. The *use* of computers is becoming more important. Programmer time is important.

10. You can program 10 times faster in APL than any other language

"This is only partly true. There are other factors involved."

The Bergquist Equation of Productivity:

$$L \times E \times U = P$$

P is Productivity. Zero productivity is being catatonic. 1000 is producing it in the time it takes to specify it. (For example, a court stenographer stops typing as the speaker pauses for breath.)

L is Language factor, 0.1 for Cobol, 1 for C, 10 for APL.

(APL gets a 10 only if you use it right, if you use it like Fortran you only get a 2; the average for APL is about 8.)

E is Experience. This is not knowing the language. This is you have learnt from your mistakes. For a 10 "if you take your shirt off, your back's covered in scars, you have made every mistake there is, you have designed the application wrong, and been burned, tried it again and been burned. You have learnt what is wrong."

U is Utilities. Utilities are extensions to the language.

"If you don't have these utilities you are dead. The more utilities you have the more productive you can be."

"APL is the beautiful glue that ties all the utilities together."

The average C programmer has a good set of utilities so gets a 6. The average (non-professional) APLer gets 2. Gary gave himself a 5 for his utilities.

	L	E	U	P
Average APL	8	4	2	64
C	1	7	6	42
Gary	9	8	5	360

APL is more productive than other languages, but not 10 times.

The easiest way for a professional APL programmer to improve his/her productivity, is to improve the utilities. Hence the ZARK Utility library.

Gary feels that he could sell us his utilities, but that won't improve his productivity, the market's not big enough, and APLers are all cheap anyhow.

What Gary really wants are *our* utilities. (See ZARK Utility Library notes below.)

11. Avoided re-inventing the wheel

"If we did not re-invent the wheel this would be a Cobol conference, we would have leeches all over our bodies, we would be using 8-track tapes."

You should always be re-examining the wheel.

12. You can be a programmer or a manager but not both

Gary got us to say the creed:

"I am a programmer, I am a professional, like any other professional, and I demand the proper respect. Like any doctor, lawyer, architect or engineer I work for myself. Any manager I have, will be an office manager, existing only to serve me, not to evaluate me."

13. APL is a good tool for certain applications

"This is a myth. APL is the perfect tool for *all* applications, all you need is to find the applications, which is not easy."

Go out and find an application, offer to maintain it for the owner, and convert it to APL, for a percentage.

APL has got about 10 years before other languages start catching up with it.

14. We need to promote the success stories of APL

This is a myth. We need to be thankful that APL is such a well kept secret.

We need to improve our productivity, and need to clobber the competition.

ZarkLib – The ZARK Utility Library

Gary is offering his current utility library to anyone who can pay for it by supplying him with a utility to be included in the library. This new (or modified) utility must improve the library. (The library is defined to be improved if other APLers want the library more because of the change.)

You have to add to the library to get the latest copy. Gary envisages that programmers will keep coming back for the latest version, and pay for that version by adding yet more utilities.

This should induce a cascade effect of improvements as is common with a “positive (but regulated) feed-back loop”. The better it gets, the more programmers want it, the more they have to add to it, the better it gets.

Please contact Gary (gabergquist@snet.net) for a copy of the ZarkLib Licence Agreement.

Tutorials

Paul Mansour (The Carlisle Group)

“Windows, Models and Management MDI, SDI, work spaces, work books, and namespaces.”

The morning after the Reception at Paul Mansour’s house, he hosted this tutorial. Being a bit hung-over at the time, my notes are sketchy so I’ll just give you a few (mis)quotes:

“If you over-design a system you have lost.”

“Be the master of YOUR domain, let the end users be the masters of their domain.”

"God invented Economists to give Weather Forecasters a good name."

Paul also introduced us to a book called "A Pattern Language" by Christopher Alexander, Oxford University Press. 1977 (YAMAPL – Yet another meaning of APL.) A recommended read for all APLers. Available from Amazon.com where it has been reviewed by 19 people with a star rating of 5. Note: This book is about architecture, *not* APL.

John Scholes (Dyadic Systems Ltd.)

"Dynamic Functions Master Class"

John described dynamic functions as "an alternative low calorie wrapper for APL". They are faster, sometimes spectacularly so, but never slower.

(For some examples download the FINAPL idiom list, which has been updated in the form of dynamic functions, from Dyadic's site at www.dyadic.com. It now contains about 800 idioms including a 30-character expression for the "Meaning of Life".)

John, as usual, both taught and entertained us. In addition to introducing dynamic functions (DFns) to us, he demonstrated how to implement most of the control structures, and covered Dynamic Operators, and explained how recursive calls to DFns can take advantage of "tail end recursion".

For example, the `:For i:In iii .... :EndFor` structure becomes `{.....} iii` and is very fast as it is special cased.

While talking about their speed, John mentioned a "3.6 mega bloop machine", which I feel is like a 3 mega flop machine but not quite as disastrous :-).

Vendor Forums

Soliton Associates Ltd., Dyadic Systems Ltd., IBM, and APL2000 all hosted vendor forums.

Soliton

Soliton have produced a more portable SAX, and using this new code base have produced new versions for IBM, RISC and LINUX.

SAX for Linux will be royalty-free for personal use and cost \$2500 +\$1000pa for a production system. (Please check these prices with Soliton.) Beta version available October 1999.

They also have a Beta version of a Java API.

Dyadic

Peter Donnelly gave the "Not the Dyadic Vendor Forum", as there were:

No Announcements; No Promises; and No Ducks.

What it did contain was an overview of the current version 8.2.2 (upgrade from 8.2.1 via the Internet if you subscribe to DSS) as described at the BAA vendor forum earlier this year.

We were then given a glimpse of version 9.0.0's new function editor which allows MDI (multiple document interface) tiling.

IBM

(In my notes, I have a sketch of a large shark swimming just below the surface of the water through which a pair of webbed (duck?) feet are sticking....)



Nancy Wheeler told us about all the recent releases (including the TCP/IP update for APL2 TSO/CMS).

She also talked about the APL2 for Windows "RUNTIME" . The developer's package runs under NT, producing a DLL which then runs under Windows 95/98/NT.

Dave Liebttag then showed us some of the things he has been working on, in particular "Processor 12" - Files as arrays. This processor (available 4th quarter 1999) has 3 flavours: APL files as arbitrary arrays, Flat files (vector of text vectors +<cr><lf>) and "card image" flat files with fixed length records.

APL2000

"COGNOS bought APL2000 because of the speed of the applications"

Eric Baelen gave a similar presentation to the one he gave at the BAA vendor forum. He also told us about the web site <http://www.openhere.com> with many excellent APL links.

Again the ducks were targeted (and finally wiped out) by Eric's video game.

(Eric has this computer game, written in APL2000, that allows the player to defend his bunnies by hitting the ducks originating in the UK who are bent on conquering the world.)

Workshops

I was unable to attend all the workshops I would have liked, partly due to my late booking and partly due to multiple streaming. Several of the more popular workshops were repeated, which helped a lot.

John Daintree – Dyalog APL and Active Data Objects (ADO)



ADO are an object oriented approach to data access and provide a consistent interface via OLDDDB to ODBC, SQL Servers etc. They allow you fast access to other data (such as email or text files). Microsoft's ADO has been around for 12 months or so. ADO ships with Windows 2000 and is available as an add-on for Windows 95/98 and NT4.

John gave a good introduction to ADO, but I got lost in the workshop, which required us first to build up an empty MS Access database for a "Music Collection". Since many of us were not MS Access users this wasted a lot of time (particularly if your spelling is not up to scratch).

I felt John should have provided us with the database already built; we were not here to learn how to use MS Access.

Once we had created the database we then had to fill it from Dyalog APL via ADO. This I failed to do because I had taken too long to build the database, and had saved it in the wrong place, so the instructions on the workshop notes did not match what I had on the computer.

Tip to future workshop presenters: keep to the core of the workshop, do not allow the attendees to waste time processing non-core activities.

All was not lost though, John's workshop notes are excellent and I will be able to try it again from home, once I have downloaded ADO for Win98.

Papers

At conferences, I often do not attend many papers, I feel that I can always read them in the proceedings, and that I might miss something else that is not so well documented. However I did attend two:

"When Bears are Blue and Bulls are Red" presented by Linda Alvord and Tama Traberman.

A very interesting paper which used two techniques to provide additional information from a graph at a glance. They used data from the NY stock exchange (where a rising price is a "Bull" market and a fall a "Bear" market).

Linda and Tama, in standard curve fitting exercises, plotted the approximating curve with either an icon of a bull (well it looked like a moose to me) or a bear, depending on the angle of the tangent at the point being plotted. In addition they used colour intensity to indicate the angle of the slope.

So when the price is falling rapidly you end up with a dark blue bear, when it is rising rapidly a bright red bull, and somewhere in the middle you get either a "polar bear" or a "little white bull".

This technique of highlighting both the the direction and intensity of a trend in a market is very appealing to me.

"Sparse Arrays in J" presented by Roger K. W. Hui.



Roger's paper showed how he had implemented a method of representing sparse data as a new data type in J.

The technique he uses is to convert the sparse array into a 2-element vector. The first element is an integer matrix, one row for each non-zero element of the array, and a column for each dimension in the array. The values of a row of this matrix define the indices of the non-zero element.

The second element is a vector containing the values of the non-zero elements. Sparse arrays may contain data of type boolean, integer, floating point, complex, literal, or boxed.

The internal structure is more complex than this, actually consisting of 6 parts, the shape, the axes of which are a vector of the sorted sparse (indexed) axes, the sparse element (usually zero) which is also the fill character, the indices, the values, and various bit flags.

Although not a J user, I found the talk appealing since I have seen a similar technique for compressing sparse data used at British Airways within an APL database system called CHIEF (designed by Dave Crossley? in the 1980s), and I have also used operators for processing sparse data. However, having a built-in sparse data structure in J must be much faster and more convenient.

Postscript



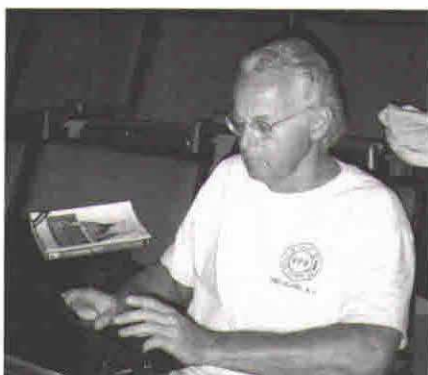
Steve, George and Lisa Mansour

As a veteran of over a dozen international conferences, I find the personal side the most important. Meeting friends becomes more important than the papers and talks. In this respect, Scranton was as good as it gets. However, I missed quite a few old friends. "APL conference Eclipsed by the Sun" was one title I had thought of calling this article, but that would have been totally unfair. I did see many old friends, and perhaps more importantly, I made several new friends. Old and new, I hope to see them all at "APL Berlin 2000" (July 24-27). Why not join me?

A Flavour of APL99



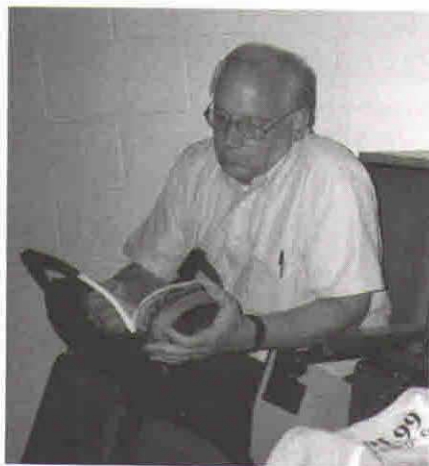
Jim Lucas with
Alan Graham



Chris Lincoln with the
Llama Book (PERL for
APLers?)



Alan Sykes, bravely
standing in for Causeway



Jim Brown, engrossed in
Vector

GENERAL ARTICLES

This section of Vector is intended for general readers who have a working knowledge of APL or J. Authors are encouraged to submit articles which illustrate effective APL applications.

The Dance of the Quirk

by Sylvia Camacho

Email: 100612.1057 @ compuserve.com

The Christmas 1997/1998 Royal Institution Lectures were given by Ian Stewart, Professor of Mathematics. For *Vector* readers not based in the U.K., I should say that these lectures are broadcast every year over 5 days during the Christmas holiday. They are superbly presented by BBC TV incorporating many demonstrations with much audience participation to a packed lecture theatre of eager children. During one of these lectures my attention was caught by Professor Stewart's demonstration of a cellular automaton of extreme simplicity and totally unexpected behaviour. Having asked a child to make a few moves using a manually operated model, Professor Stewart turned to a computer and demonstrated the result of running for over 10,000 generations. Naturally this whetted my appetite to set the model up for myself. This is the sort of plaything for which APL is an ideal tool. I hope *Vector* readers, if asked, 'What is APL good for?' will turn to this example and answer, 'Very good for having fun'.

The model has two parts: a two-dimensional array of cells and a 'Something' which in Ian Stewart's case was a plastic beetle but I have decided to call my virtual something a 'Quirk'. Chambers Dictionary defines 'quirk' as '... a trick or peculiarity of action ... to turn sharply ...'. The Quirk has a head and tail and it may point (and move in the direction it is pointing) in one of the four orthogonal directions of the matrix of cells, it moves one cell for each generation. After each move it rotates 90 degrees clockwise or anticlockwise. Each cell of the array may be in one of two states, let us say colour zero or colour one. All cells are zero to begin with and Quirk is placed somewhere about the middle of the array. I began my experiment with a 100×100 matrix but 76×76 will suffice. The rules for moving Quirk are:

- 1. Move Quirk one cell in the direction in which it is facing.
- 2. Turn Quirk 90 degrees anticlockwise if the cell to which it moves is colour zero or 90 degrees clockwise if it is colour one.
- 3. Change the colour of the cell which Quirk has just left.

One expects to see zero-coloured cells changing to colour one and some reverting to zero as Quirk twists its way around its Universe. How far from its starting point will Quirk wander in a given number of generations? I arranged for the

limits of its travel after each 100 generations to be reported as the result of my function DANCE.

I am using an old version of APL*PLUS in a DOS window under Windows 95. For this reason I use quadWPUT to display the array. This has the disadvantage that the screen is only 25x80 characters and I found I needed approximately 70x50 to reproduce Professor Stewart's display. My decidedly jury-rigged solution to this problem is contained in the function REHASH. Vector readers who command powerful GUI Tea-Clippers or East Indiamen will be able to explore the model without this inelegant lash-up. If any reader sets up a version with exciting bells and whistles (or should I say 'dressed overall') I am sure the Editor will be happy to publish it.

I suppose the best known cellular automaton is LIFE, invented by John Horton Conway long before we played with computers on our desks. He was challenged to devise rules which would give rise to a self-reproducing pattern and his invention acquired a cult following as people explored the characteristics of various starting patterns. (Some years ago Paul Chapman wrote a superb implementation of LIFE for the Archimedes and the PC but not unfortunately for SVGA monitors.) The reason that Professor Stewart's model took my eye is the extreme simplicity of his rules and the unexpectedness of the sudden change of behaviour after about 10,300 unremarkable generations. On seeing this the first thought that came into my head was 'How about that for a phase change?'. The next was 'a track-laying vehicle steered by its tracks', which I have always admired as a nice definition for a tank. However Quirk lays its track after it has moved forward, so it is only on the second time of moving to a cell that it is steered by its track previously laid. On the other hand, of course, we could change the rules so that Quirk changes the colour of the cell immediately before moving into it.

There are two aspects of this automaton that I think are notable. The first is that the change to a repeating pattern dancing inexorably towards the edge of the universe can be described as 'emergent' behaviour. The second is that the progress of Quirk is the result of an interaction with the environment and thus is an example of feedback. It leads me to wonder what change of the rules would be needed to handle more than one Quirk and how about modifying Quirk twists after two visits to the same cell - perhaps marking its passage with a cycle of colour changes.

For those of you with the energy to key it in here are my two functions and a picture of the screen after 11,000 generations. Have fun!

```

V RES=DANCE GEN;[]IO;AREA;C;COL;FIN;NWSE;OBJ;R
[1]  a display the Dance of the Quirk: SMC 2 March 1999
[2]  []IO+0 o RES+ 0 4 p0
[3]  U+ 76 76 p0 o AREA+ 35 22 35 22 a U is global
[4]  a controls movement of Quirk North, West, South, East
[5]  NWSE+ 2 4 p -1 0 1 0 0 -1 0 1
[6]  R+35 o C+22 o FIN+0
[7]  L1:=(FIN=GEN)+END
[8]  a record minimum and maximum row/column visited by Quirk
[9]  AREA+(AREA[0 1][R,C),AREA[2 3][R,C
[10] a change the colour of the cell currently occupied by Quirk
[11] U[R;C]←U[R;C]
[12] a move Quirk one cell
[13] R+R+NWSE[0;0] o C+C+NWSE[1;0]
[14] a go to END if edge of Universe reached
[15] →((R<0)∨(R=76)∨(C<0)∨C=76)+END
[16] a record the colour of the new cell and rotate Quirk accordingly
[17] COL←U[R;C] o NWSE←(1×COL)φNWSE o NWSE←(0=COL)φNWSE
[18] OBJ←REHASHqU a jury-rig character matrix to fit the screen
[19] FIN←FIN+1
[20] →(0=100|FIN)+L3 a record area covered after every 100 moves
[21] RES←RES+AREA
[22] L3: 0 0 24 80 []WPUT 24 80 +('Move Number ',*FIN)COVER OBJ o →L1
[23] END:
V
V RES=PLAYGOD GEN;[]IO;A;AREA;B;C;COL;FIN;NWSE;OBJ;R
[1]  a provoke repetitive pattern by pre-setting Universe: SMC 2 March 1999
[2]  []IO+0 o RES+ 0 4 p0 o A+35 o B+25
[3]  U+ 76 76 p0 o AREA+ 35 25 35 25
[4]  a play God by giving Quirk a favourable environment
[5]  U[A+0;B+3]+1 o U[A+1;B+ 2 3 4]+1 o U[A+2;B+ 5 6]+1
[6]  U[A+3;B+6]+1 o U[A+4;B+ 6 7]+1 o U[A+5;B+ 7 8]+1 o U[A+6;B+7]+1
[7]  NWSE+ 2 4 p -1 0 1 0 0 -1 0 1
[8]  R+40 o C+33 o FIN+0 a any of these starting points will serve
[9]  a R+39 o C+32 o FIN+0
[10] a R+39 o C+31 o FIN+0
[11] a R+38 o C+31 o FIN+0
[12] a R+38 o C+27 o FIN+0
[13] a R+36 o C+27 o FIN+0
[14] L1:=(FIN=GEN)+END
[15] L2:AREA+(AREA[0 1][R,C),AREA[2 3][R,C
[16] U[R;C]←U[R;C] o R+R+NWSE[0;0] o C+C+NWSE[1;0]
[17] →((R<0)∨(R=76)∨(C<0)∨C=76)+END
[18] COL←U[R;C] o NWSE←(1×COL)φNWSE o NWSE←(0=COL)φNWSE
[19] OBJ←REHASHqU o FIN←FIN+1
[20] →(0=50|FIN)+L3
[21] RES←RES+AREA
[22] L3: 0 0 24 80 []WPUT 24 80 +('Move Number ',*FIN)COVER OBJ o →L1
[23] END:
V

```

```

v R←REHASH U;⊠IO;N;ROW
[1] a returns decode of global Universe as block characters: SMC 2 March 199
[2] a argument U is a flip of global Universe so that rows convert to column
[3] a take rows of U in pairs & convert to ⊠AV block chars
[4] ⊠IO←0
[5] ROW←0
[6] U← 48 76 +U a screen limits
[7] N← 24 76 p0 a convert in pairs to give 24 screen rows
[8] L1:←(24=ROW)+END
[9] N[ROW;]←2⊠U[0,1;]
[10] ROW←ROW+1 o U← 2 0 +U o +L1
[11] END:R←(⊠AV[32 220 223 219])[N] a decoded version of U
v

```

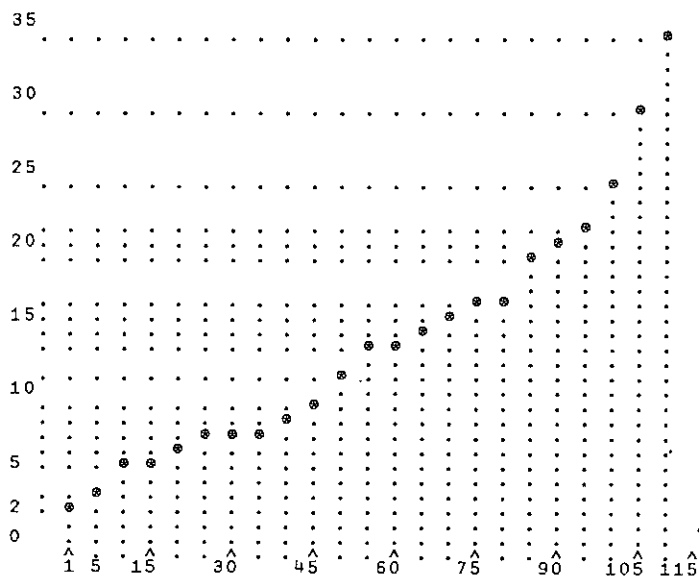
Move Number 11000



The function COVER is included in my article in Vector 15.3, 'Forty Years On'.

This graph is what I made of the result from DANCE 11000 but those of you with four-mast GUIs with no barnacles and a spinnaker set will be able to make a plot ship-shape and Bristol fashion.

Area Visited
Square Cells (100s)



Quirk Moves (100s)

Sharp APL vs. APL2 coding style

by Dan Baronet (danb@dsuper.net)

Introduction

Following Soliton's announcement to unleash their APL into the Linux community I decided to review the differences between Sharp APL and APL2 style APLs (IBM's APL2, Dyalog, APL+Win). I have divided the report into several articles, each with a different topic.

This article is the first of the series. It should be read before any of the others.

Not having a current version of IBM's APL2 on hand I have been making my observations based on Dyalog's APL using $\square m1 = 2$, which, like APL+Win, follows APL2 closely.

When reading this text keep in mind the date it was written (September 1999) as languages change and may render this information incorrect in the future.

Conventions

In the text *italicized* words have a special meaning and often precede their definition.

SAPL refers to Sharp APL, including SAM which is SAPL under MVS and SAX which is SAPL under Unix. SAX is the one also running under Linux and compared to APL2 here.

Variables and workspace names are enclosed in 'quotes'. Functions and files are enclosed in <angle brackets>.

The term *atom* refers to a numeric or character scalar only. The term *item* refers to any scalar.

Whenever possible I will show examples in SAPL and APL2 side by side.

A bit of history

Sharp APL (SAPL) used to be a mainframe product only until a PC version appeared in 1984 (release V17 of SAM and rel V20 later on). It remained pretty

much so as the PC version never took off. The PC version ran under DOS and lacked many features. Because it was emulating the S/370 it was also very slow.

SAPL really is a product as opposed to a language. There are various components including a batch scheduler, a file system, many shared variable processors, sorting utilities, etc. More importantly, SAPL tasks may spawn and **control** other APL tasks with system functions, something no other APL has ever achieved. Very large secure systems can be (and have been) written in SAPL.

By contrast, in IBM's APL2¹, everything but the language is external and only available through auxiliary processors (APs). There are no quad functions, for example, to perform file operations.

There are pros and cons for using each APL and the purpose of this article is not to take sides but rather to show the differences between them as objectively as possible.

SAX came to life in the early 90's and is a more flexible version of SAM. It is based on "the Dictionary of APL" by Ken Iverson. It extends SAM V21. It more closely resembles J. (Think of J on APL char set.) The Linux version is the same as the Unix version.

At some point there was a shift in philosophy using the first versus the last axis in function operations (ex: / vs. ⌵) and some newer functions were defined in terms of the latter. APL2 has remained pretty much a "last axis favoured" language.

Enclosed Arrays

In SAPL the term "enclosed array" is used. In APL2 the term used is "nested array". They are, practically, the same thing. The term "boxed array" (SAX) is also sometimes seen but won't be used here.

Let's assume we have the following variables on hand:

```
a←'text'◊ b←1 2 3 ◊ c←2 3p a ◊ d←2 4 3p133
```

The simplest way to create an enclosed item is to use the expression

SAX

```
x←<anyvalue
```

APL2

```
x←<anyvalue
```

¹ Dyalog and APL+Win do include system functions for APL files operations.

In SAPL there is no maximum level (depth) of enclosure. Thus $\ll\ll a$ is 'a' enclosed 3 times. In IBM's APL2 there is a limit.

Items must be *opened* before applying a function to them. If $x \leftarrow \ll 2 \ 3 \ 4$ then

$$+ / > x$$

$$+ / > x$$

will result in the sum of the values in 'x'.

To create a list (vector) of enclosed items we can use the usual *catenate* function (,):

$$x \leftarrow (\ll a), (\ll b), \ll c$$

$$x \leftarrow (\ll a), (\ll b), \ll c$$

This will ALWAYS work in all interpreters I know. But there is an easier (incompatible) way to do this:

$$x \leftarrow a > b > c$$

$$x \leftarrow a \ b \ c$$

Which brings us to the way things are presented to the interpreter.

Notation

APL2 uses what is known as *strand* notation. Using this notation, arrays are created by putting items together side by side, without using catenate (',') as in the example above. Sometimes it is handier than using ','. SAX does not support that notation and requires the use of functions ',' to catenate items together or '>' to *link* them if they are to be enclosed.

Link

Link is a specific SAPL function to 'enclose and catenate' with a twist. To understand its use let's look at the monadic case first:

Monadic 'link'

This is another function specific to SAPL. The formal definition is: 'enclose the argument IF it does NOT contain already enclosed item(s)'. Sort of a conditional enclose. In fact *conditional enclose* is its real name. See its usage:

$$x \leftarrow > var$$

$$\pm (1 < | = x \leftarrow var^2) / 'x \leftarrow c var'$$

² The | is necessary only in Dyalog where $\pm$ can be negative if the structure doesn't have a constant depth.

Determining if an array is enclosed

An application of this is to determine whether we are dealing with an enclosed array or not:

$$x \Rightarrow x \qquad 1 < | \Rightarrow x$$

will return 1 if 'x' is enclosed. There is no *depth* function in SAPL.

Dyadic 'link'

$a \Rightarrow b$ will enclose 'a' and catenate it to the enclosure of 'b' if 'b' does NOT contain enclosed elements. Formally,

$a \Rightarrow b$ is the same as $(\langle a \rangle) \Rightarrow b$

Why not use $(\langle a \rangle) \langle b \rangle$? The reason is to avoid *enclosing* a list repeatedly. Imagine what would happen if it were to always enclose the right argument in the expression $a \Rightarrow b \Rightarrow c$. This would result in $(\langle a \rangle) \langle (\langle b \rangle) \langle c \rangle$, a two-element list. This is not what we want. A consequence of this is that one must explicitly code the enclosure of the last item IF the last value MUST be enclosed, as in $x + a \Rightarrow b \Rightarrow m$ (with $m \neq 2$ $3 \rho \langle a \rangle$), to get a three-element list.

On the other hand, if we want to, say, catenate $\langle a \rangle$ before each row of 'm' then $a \Rightarrow m$ will do whereas $(\langle a \rangle) \cdot m$ is required in APL2.

Heterogeneous arrays

Both APLs support arrays containing a mixture of items of different data types, namely characters, numbers and enclosures although they must be explicitly catenated in SAX:

$x+1, 's'$	$x+1 \ 's'$
$x+1, 's', \langle 'sa' \rangle$	$x+1 \ 's' \ 'sa'$
$x+1, 's', 'sa' \Rightarrow 2 \ 3$	$x+1 \ 's' \ 'sa' (2 \ 3)$

Disclosing multiple items at a time

Entire arrays can be disclosed in one fell swoop with only space and maximum rank (127) restrictions:

$$2 \ 3 \Rightarrow \rho > 1 \Rightarrow 'dsa' \qquad 2 \ 3 \Rightarrow \rho > 1 \ 'dsa'$$

In Dyalog there is a "same rank" restriction³. Not in SAX:

³ I do not know what IBM's APL2 and APL+Win do here.

4 2 4 4 p>a>b>c>d .

(RANK ERROR in Dyalog and APL+PLUS II)

Disclosing an array results in another array of the shape of the array disclosed, followed by the largest of the shapes of ALL items disclosed with proper fill elements to fill where necessary.

Atom enclosure/disclosure

A major difference between SAPL and APL2 is in the handling of atoms (simple scalars) upon enclosing them. In SAPL enclosing ANYTHING adds a level of enclosure. Not so in APL2 where an atom can NEVER be enclosed. Thus:

$\sim 2 = < 2$

$2 = < 2$

This has several implications as we shall see in a later article.

The disclosure of an atom is permissible in both APLs. Thus

$2 = > 2$

$2 = 2$

Operations on arrays

SAPL offers a few operators to work with (enclosed) arrays. I will discuss only the basics in this article. The next article will cover the subject in more depth (no pun intended).

With, Under, Each (``)

Although *Under* is the official name under SAX, people often refer to it as *With*. APL2 uses the term *Each*.

With *'/'* this is the most used operator in ALL APLs. It is probably best explained starting with an example:

$p''>x$

$p''x$

Here we wish to return the *shape* of each item by first disclosing (each) cell of *'x'*, getting its *shape* and re-enclosing it, ending with a result of the same shape as the original argument.

In SAPL *''* is dyadic, allowing for a different right function. The people at APL2 probably felt that since *disclose/enclose* is used most of the time it was better to make *''* monadic and include these functions implicitly whereas at I.P.Sharp (where SAPL came from) people tried to be more general.

Some interesting combinations can be made with 2 functions and $\cdot$ although 99% of the time *disclose* ($>$) is used.

An example is multiplication:

$$A \times B \quad \text{is} \quad * (\odot A) + (\odot B) \quad \text{or} \quad A + \cdot \odot B$$

The real APL2 definition is: "for each cell:

- disclose the cell
- apply the (ambivalent) function of $\cdot$
- enclose the cell"

The real SAPL definition is: "for each cell:

- apply the right function of $\cdot$ (usually $>$)
- apply the left (ambivalent) function of $\cdot$
- apply the inverse of the right function of $\cdot$ (usually $<$)"

Or, given $f \cdot g \ y$, for each cell: " $g^{-1} f \ g \ y[j]$ ".

This, of course, implies that someone in the interpreter department goes over all the suitable right hand argument functions and determines an inverse for each one of them. And it was done. For example, $<$ and $>$ are inverses of each other. $\&$ is its own inverse.

Dyadic *With* reviewed

This is the same operator except that f (the left function) is used dyadically⁴. In more complicated terms, $x \ f \cdot g \ y$ is, for each corresponding cell:

$$"g^{-1} (g \ x[j]) \ f \ (g \ y[j])".$$

Where ' x ' is the left argument.

Examples:

$$\begin{matrix} p \ \rho \cdot > \ q \\ t1, \cdot > t2 \end{matrix}$$

$$\begin{matrix} p \ \rho \cdot \cdot \ q & \text{(reshape each)} \\ t1, \cdot \cdot \ t2 & \text{(catenate each)} \end{matrix}$$

⁴ this adverb can't be found in the English dictionary. Yet.

Scalar functions

In APL2, the use of the *each* operator is assumed for scalar functions (like $+$) and derived functions, enclosed or not. There is no need to specify it. Ex:

$$x + \cdot y$$

$$x + y$$

Moreover, the function is applied recursively so long as the lengths are respected at each (nested) level. This is not possible in SAX.

The reason is that since *disclose* is implied and that an *atom* can NEVER be enclosed in APL2 then the *each* specification CAN be omitted. Structural functions like *take* MUST use *each* to remove ambiguities.

This is also why $+ / x$ and $+ / \cdot x$ mean two different things in APL2. In SAX those would be coded as $+ \cdot > / x$ and $+ / \cdot > x$ respectively.

On ($\ddot{o}$)

This operator is similar to *with* except that the result of each cell is not modified by the inverse of *g*.

$\cdot$ and $\ddot{o}$ are called composition operators. The only difference between them is in the absence of inverse for the right argument function of $\ddot{o}$. For example, using $f \ddot{o} > y$ is the same as doing

$$> f \cdot > y$$

$$> f \cdot y$$

This may seem trivial but I will show in a subsequent article, along with another operator, the implications of using this operator.

Other differences

SAPL supports the use of alpha (α) and omega (ω) as APL identifiers for variables, functions and labels. These were introduced when the α / ω "direct definition" notation was popular (late seventies). The wisdom of their usage in code is debatable but they are available. Also SAPL literature often refers to the left argument as alpha and the right argument as omega. Something to keep in mind.

Complex numbers

These are not available in SAX yet.

Control structures

These are not APL2 but are worth a mention anyway. They do not exist in SAX. In a subsequent article I will show how to emulate them in a control version system that Soliton provides.

Defined operators

SAPL does not allow you to define your own operators.

Error trapping

Similar to Dyalog APL. Even the event numbers are almost the same. A notable difference: the ability to trap value errors "in line" and resume after solving the error. This allows you to easily do "demand paging" of objects.

File operations

In SAPL those are more like APL+Win. System functions don't start with an 'F' (⌈FREAD, not ⌈FREAD). All file operations return a result, that is *val* *replace* *tc* returns 0 0 0. This allows someone to use them with operators.

Format by example

SAX allows the usage of a character left argument to the format primitive *⍕*. This is the same as IBM's APL2 (not found in Dyalog or APL+Win).

Multiple assignments

It is possible, in both APLs, to "assign with disclose" several values to several variables by using quotes around the variables:

```
'a b'+2 5>'cxz'          (a b)+(2 5)'cxz'5
```

There is a peculiarity using this technique where the number of variables is 1:

```
'a b c'+1>1 2>1 2 3      ('c' is 1 2 3)
'b c'+1 2>1 2 3          (again, 'c' is 1 2 3)
'c'+>1 2 3               (now 'c' is <1 2 3, NOT 1 2 3)
```

This behaviour is found on ALL APLs (also in J!).

⁵ In Dyalog the parentheses are not necessary around the variables leading to potential conflicts.

Personally, I think this is inconsistent. If there are no quotes (or parentheses) it should be a direct assignment. If there are some it should ALWAYS remove one level of nesting before assigning the value(s).

From

From is an alternative to bracket indexing. The beauty with *from* is that since it IS a function it does not suffer bracket syntax problems and can be used with operators.

From selects elements along the first dimension. Used with list arguments it selects individual items:

```
n{list                      n>"c list
```

Pick (dyadic $\triangleright$ in APL2) and *First* (monadic $\dagger$) are not available under SAX. You can still use $\triangleright$ with brackets [x] or use *from* (!):

```
x $\dagger$ atom{list              x $\dagger$ atom $\triangleright$ list
```

Note that *from* is $\square io$ independent. It uses an *offset* instead of an origin dependent position (index). *from* also allows you to select items from the end of the array by supplying negative offsets. Ex:

```
't' = 3{a                      't' = (3 +  $\square io$ ) $\triangleright$ a
't' ^ . = 0 3{a                't' ^ . = a[1 4] (in  $\square io$  1)
'x' = ~2{a                     'x' = a[ $\square io$  + ( $\rho a$ ) + ~2]
```

Prototyping

In APL2 prototypes are used to fill holes created by $\backslash$ or $\dagger$. Ex: $2 \ 2 \triangleright \rho \dagger 0 \rho < 2 \ 2 \rho 5$. SAPL has no prototype. If 'x' is an *item*, the expression '0\ x' always returns either 0 for a number, blank for a character or < 10 for an enclosed object.

Reduce N-wise

IBM's APL2 allows you to reduce along an axis in sections of length N ($N f/ x$). Not in SAPL.

Stop, Trace

Those are set using $\square stop$ and $\square trace$ system functions as in Dyalog APL. IBM's APL2 uses 't δ ' and 's δ ' special name prefixes to do the same.

Take and drop

In SAX these functions work on the leading axis of an array without having to specify them all. Ex ($\rho \rho m$):

$$\begin{array}{ll} 2+m & (2, 1+\rho m)+m \\ 2+m & ((1 \uparrow \rho \rho m)+2)+m \end{array}$$

IBM's APL2 allows you to use *take* with one axis as in

$$3+m \qquad 3+[1]m$$

Scattered indexing

When the argument has a rank higher than 1 then indices should be enclosed:

$$\begin{array}{ll} (<1 \ 2)\{d & (<1 \ 2)> d \\ (1 \ 2>0 \ 1)\{d & (1 \ 2)(0 \ 1)>d \quad (\square \text{io } 0) \end{array}$$

But you can pick an entire row (or first dimension elements) by supplying the offset only:

$$2 \ 3 \approx \rho \ 1 \ 3 \{ \ 4 \ 3 \ \rho \ 1 \ 2$$

Swap

This operator merely swaps the arguments to a function. Dyalog uses $\approx$ for that. For ex:

$$2=7-\approx 9$$

Unique features in SAX

Here is a list of features not found in any other APL.

Index (iota underbar)

This function returns the index of the first occurrence using epsilon underbar (ϵ) as primary function:

$$2 \ 3 \approx (4 \ 5 \rho \ 1 \ 2 \ 0) \underline{1} \ 2 \ 2 \ \rho \ 8 \ 9 \ 1 \ 3 \ 1 \ 4 \ (\square \text{io } 1)$$

Jot

$\circ$ is a constant whose value is < 10 . This is the constant used for *overtake* and $\backslash$ with enclosed or heterogeneous arrays.

Nub (monadic take)

This one is not unique to SAX (Dyalog's equivalent is `v`) but it handles higher rank arrays:

```
5 2 = + 9 p 5 2
(+m[7p:3;]) = m+3 4 p 'johnmarypaul'
```

Nubin (monadic =), nubsieve (monadic ≠)

`=x` is the equivalent of $(+x) \circ . = x$ for vectors

`≠` returns a boolean indicating where each element is first encountered. This is the equivalent of $((v \vee v) = \vee v)$ for vectors. Works on matrices too such that

```
1 0 1 0 1 0 = ≠ 2 ≠ m
```

Output redirection

It is possible to send all input/output to a file using `⎕out 0, tie`.

Packages

Packages are unique to SAPL and are very useful. A series of system functions can be used with them, all of which start with `⎕p` as in `⎕pack`, `⎕pdef`, etc. For example, to define variable 'x' with objects (variables, functions) 'a' and <f> (for example to be stored on file) you would do

```
x←⎕pack 'a f'
```

Think of them as static namespaces (found only in Dyalog). You cannot "enter" a package and run something from within it. For example, if you wish to use its contents in a local environment you must first localize the names. For example, this function <execin> will execute an expression using a package's contents:

```
vr←exp execin pack;f
[1]⊢⎕fx>('f',,',';',⎕pnames pack)⊃'⎕pdef pack ∘ r+',exp
v
```

Pass, Stop

These two functions are probably the simplest ever conceived yet very useful. *Stop*:

```
⎕dl 5                                0 0p⎕dl 5
x←2-y+5                              y←5 ∘ x+2
```

Pass:

```
⊖x+1 2 3
```

```
⊖+x+1 2 3
```

This last function is more useful than it looks. For example, if space is important and you do not want the interpreter to keep a copy of the argument to a function you could do

```
myfn (⊖ex 'x')⊖x
```

This effectively erases the variable AFTER passing it as an argument to <myfn>. The interpreter is then able to free up the memory space, should the need arise later on (for example after you reassign the argument within <myfn>). I don't know of any other **general** way to do this with any other interpreter.

I-beam functions

These are a leftover from the 60's. If you think you know what they are you probably weren't there. They are kept for compatibility reasons with SAM (many are no-ops) and will not be explained here.

Less

This one is not unique to SAPL (most APLs handle the vector case) but it accepts higher rank arrays:

```
12 7 ⍱1 7 12 5 ~ 16
'john' = ,m~m[2 3;]
```

Raze

This is an interesting monadic function (in terms of *raison d'être*). Its formal definition is

```
⌈"⌈>⌈
```

In other words, "working from the first dimension: catenate on the first dimension all items".

My guess is that this came from a requirement to create in SAPL a function reminiscent of APL2's monadic ϵ which puts everything in ravel order regardless of structural depth⁶. Before *raze* programmers would often need to get text (or numbers) from enclosed lists together, sort of $> , ''> /$ but SAM wouldn't allow it

⁶The design of *raze* had nothing to do with monadic epsilon in APL2. It is much superior, as it preserves lower-level structure. Some consider the design of monadic epsilon to be largely useless in real applications.

resulting in expressions like “, *x” (plus `⎕fi` if numeric) being used resulting in high CPU costs. Hence *raze* was created. Plus it can be useful.

Shared variables

`⎕svs` returns the status of a variable

`⎕sc` is used to wait on ANY shared variable event. It can also be assigned a “maximum wait time”.

`⎕svn` is used to uniquely identify your task when multi-tasking.

System commands

SAX commands are similar to other APLs.

`)copy` allows the copying of system variables.

`)load` and `)clear` allow you to specify initial `⎕wa`. Ex:

`)clear 200000000` will give you a workspace of 200M if you have the room!

System functions

Workspace reporting

Various workspace elements (name, stack) can be found using `⎕ws`.

Function definition

`⎕fd` can be used to manipulate function representations (similar to `⎕def`, `⎕vr` in APL+Win).

Of course ISO standard functions like `⎕cr` and `⎕fx` also exist.

System variables

`⎕ps` controls the formatting of enclosed arrays. It is a four-elements integer list of

- 1 where to position each item in its own box (-1=left, 0=centre, 1=right)
- 2 where to position each item in its own box (-1=up, 0=centre, 1=down)
- 3 horizontal spacing between items
- 4 vertical spacing between items

The default `-1 -1 0 1` fits most cases.

`⎕sp` is used to store values. It stays the same between `)loads`. Probably one of the most valuable system variables ever created.

10{\av is the character used as line delimiter. Not 13{\av. One should exercise caution with this.

Table (monadic comma bar, ¸)

This function restructures any structure into a matrix, something like

```
(2+(pt),1 1)pt
```

Table turns a scalar into a 1 by 1, a vector into a 1 column table, anything else into a table.

Task spawning

This is possible in SAX and will be the subject of another article.

Conclusion

SAPL and APL2 are quite different in many respects but each (!) has its own merits. For anyone wishing to delve into the APL/Linux world there is, right now, little choice but to examine SAPL. Personally I am comfortable with both. I found SAPL to be fast, reliable and, with the tools Soliton is providing, a serious product.

The next article will deal with special SAPL operators (hoof, rank, cut, numeric with,)),. Another one will deal with multi-task operations. If time permits, another one will talk about intrinsic functions and one of Soliton's products, *logos*.

Please direct questions/remarks to danb@dsuper.net.

Dan Baronet
Montreal
Canada

References

1. Bernecky, R: An Introduction to Function Rank in APL88 Conference Proceedings, ACM SIGAPL Quote Quad 18:2, 1987, pp. 39-43
2. Bernecky, R and Iverson, K E: Operators and Enclosed Arrays, (APL) Users Meeting 1980, I.P. Sharp Associates Limited

Time Space Tradeoffs in Vector Algorithms for APL Functions

Timothy A. Budd (*budd@cs.orst.edu*)

Abstract

When considering algorithms which will run on processors in which large vectors of values can be manipulated in time which is independent of the length of the vector, many issues of time versus space arise which are not important, or of less importance, on more conventional machines. This paper explores various alternative techniques which might be used in an implementation of APL on such a machine, and considers some of these time/space tradeoffs.

Keywords: *code generation, vector machines, APL, time/space tradeoff.*

Introduction

An observed characteristic of many APL expressions is what might be called a "bulging middle". That is, they may start out with relatively small inputs, which are then transformed (by an inner or outer product, for example), into large intermediate values. These intermediate results are then compacted, for example by reduction or compression, into relatively small outputs. The classic idiom for generating a boolean vector representing the primes less than n [Perlis & Rugaber 1979] is a good example:

```
2=+/0=(1n)°.|1n
```

Here the inputs are scalar and the output is a vector of length n , but in between an array of size n^2 is both constructed and reduced. Many APL programmers have had the annoying experience of writing a function which works fine on small test cases, but then runs out of memory on larger data because the system could not accommodate these intermediate results.

Some APL implementations [Budd 1988] have attempted to circumvent this problem by means of a lazy evaluation approach which produces elements on demand. Using these techniques the large intermediate results are never generated all in one place, but rather are produced element by element as needed. In this manner memory requirements can be kept to a minimum. However little or no time savings are produced using these algorithms, except in the cases such

as compression where portions of the computation which do not contribute to the result can be eliminated altogether.

This situation changes when one starts to consider algorithms which might be used to implement various APL functions on parallel architectures, such as vector machines. In this paper we will present examples illustrating some of the time/space tradeoffs which can occur. This is not to say there are not time/space tradeoffs which can be made in scalar implementations of APL. Indeed, for some functions, such as membership (dyadic ϵ) the alternatives are dramatic [Budd 1981]. However, for most functions, and certainly for the most commonly occurring functions, the choices are straightforward. This is not true for vector implementations.

The model we will use for describing the vector algorithms is that used by Hillis and Steele [Hillis & Steele 1986] to describe the Connection Machine [Hillis 1985]. Our notation for algorithms will be Algol-like, with the addition of an instruction:

for all k to n in parallel do s od

which will execute the statement s in parallel n times, each instance of s having access to a unique value k . The total time for this computation will be assumed to be the same as the time necessary for a single execution of s , assuming n does not exceed some maximum vector length (on the Connection Machine, this limit is the number of processors). For larger values of n , the vector is divided into equal components each of the maximum size, and an implicit loop is made around the statement s . Our notation differs from Hillis and Steele in that it explicitly provides the upper bound n .

Reduction and Outer Product

Saal and Weis reported [Saal & Weis 1975] that reduction and outer product were among the most commonly used APL functions. These two frequently also occur in combination. A somewhat contrived example, but similar to many real situations (such as the primes expression of the last section), will illustrate a few of the possible tradeoffs that can be made. Our example expression will simply take a scalar value n , produce an n^2 array using outer product, then reduce this twice to once more produce a scalar result.

```
+ / + / (1 n) * . + 1 n
```

The naïve and straightforward algorithm for this example would evaluate the expression from inside out, forming an entire intermediate value before

considering the next outermost function. In this example this is not only the most straightforward algorithm, it is also the most time efficient. All n^2 elements of the outer product can be generated in one step, as follows:

```
for all  $k$  to  $n*n$  in parallel do
     $t[k] := (((k-1) \bmod n) + 1) + (((k-1) \div n) + 1)$ 
od
```

The innermost reduction can then be performed using a variation of Hillis' and Steele's array summing algorithm [Hillis & Steele 1986], which works in $O(\log_2 n)$ time:

```
for  $i := 1$  to  $\text{ceil}(\log_2 n)$  do
    for all  $j$  to  $n*n$  in parallel do
        if  $((j-1) \div n) \bmod 2^i = 0$  then
            if  $((j-1) \div n) + 2^{i-1} < n$  then
                 $t[j] := t[j] + t[j + (2^{i-1} * n)]$ 
            od
        od
    od
```

And the outermost reduction in a similar fashion. Thus we can produce the answer using $O(\log_2 n)$ time and $O(n^2)$ space.

Supposing n is large, and we would prefer not to generate the n^2 intermediate values. Using demand-driven techniques [Budd 1988], we could combine elements of the outer product with the reductions. Since the nested reduction is computing column-sums, one approach would be to use a slightly less efficient algorithm for the reduction, but combine the summing of the reduction with the generation of rows for the outer product:

```
for all  $i$  to  $n$  in parallel do
     $t[i] := 0$ 
od
for  $i := 1$  to  $n$  do
    for all  $j$  to  $n$  in parallel do
         $t[j] := t[j] + (i + j)$ 
    od
od
```

Here we use $O(n)$ time, but only $O(n)$ space. The outermost reduction can be accomplished, as before, using an $O(\log_2 n)$ summation to produce the final scalar result.

But even this can be viewed as being space inefficient, since the final output is simply a scalar. The final possibility is to ignore vector instructions altogether, producing instead single values from the outer product on demand.

```
t := 0
for i := 1 to n do
  for j := 1 to n do
    t := t + (i + j)
  od
od
```

So we have a sequence of possible algorithms to implement the functionality of this expression:

<i>algorithm</i>	<i>time</i>	<i>space</i>
simple algorithm	$\log_2 n$	n^2
combined reduction and outer product	n	n
fully demand driven reductions	n^2	1

Which is the "correct" approach? The answer, of course, is that under various circumstances any one of them might be the correct technique to use, depending upon the amount of memory available and how quickly the user wants the answer produced. One problem is that while it is easy to optimize for time (the naïve first algorithm, which simply produces intermediate results from the inside out), it is much more difficult for a system to make compromises with time and space, since these involve the interaction of adjacent functions.

It should be noted that matrix multiplication (inner product in APL terms) is in many ways similar to the reduction of an outer product. It should come as no surprise, therefore, that there are several possible choices in matrix multiplication algorithms of a vector nature, from $O(\log_2 n)$ time and $O(n^3)$ space, to $O(n)$ time and $O(n^2)$ space, for complete results. If only partial results are desired (for demand driven evaluation), there exist $O(n)$ time and $O(n)$ space algorithms, and a $O(n^2)$ time and $O(1)$ space version.

Why would an implementation choose to use a less time efficient algorithm? In our model, if the size of a vector operation exceeds the maximum permitted size the vector is simply divided into equal portions and a loop is made around the separate pieces. If n^2 is close to this maximum length, then a $O(\log_2 n)$ operation on a vector of size n^3 will still take $O(n \log_2 n)$ steps. Thus one of the more space efficient algorithms would be likely preferable, despite their asymptotically worse performance.

A Searching Example

Our second example again exhibits a time/space tradeoff, and also illustrates some of the non-obvious algorithms which can be considered when vector instructions are available that work in constant time.

The APL function "index of" (dyadic ι) takes two arguments, the first of which must be a vector. It returns a result of the same size and shape as the right argument, only containing, for each element in the right argument, the position of the first occurrence of the element in the vector left argument. The length of the vector left argument plus one is returned for values which do not occur. For example $27\ 32\ 32\ 17\ \iota\ 12\ 17\ 27\ 32\ 19$ yields $5\ 4\ 1\ 2\ 5$.

To describe the possible algorithms which might be used to implement this function, let us suppose the left argument, call it a , is a vector of length n and the right argument, b , a vector of length m . Furthermore assume we want to place the answer into a vector of length m called c .

The naïve algorithm is simply a double loop, which iterates over the values of b , checking each against the successive values of a .

```
for i := 1 to n do
  c[i] := m + 1
  for j := m downto 1 do
    if a[i] = b[j] then
      c[i] := j
  od
od
```

Using this algorithm takes $O(n * m)$ operations to produce the n result values.

It has long been known [Bernecky 1973], that execution of this function can be speeded considerably by preprocessing the left argument, producing an array of length n containing the positions of the elements in sorted order. (This is, in fact, what is produced by the APL sorting function *grade-up*). For example if a is the vector $17\ 32\ 12\ 97$, the temporary variable containing the sorted positions would be $3\ 1\ 2\ 4$, indicating the fact that the smallest value occurs in position 3, the next largest in position 1, and so in. Using this temporary variable and three additional variables one can then use a binary search to find the position of the arguments from the right argument. Let us assume this temporary sorted value has already been generated and placed in the variable t .

The algorithm to produce the n output values is therefore as follows:

```

for  $i := 1$  to  $m$  do
   $high := m$ 
   $low := 1$ 
  for  $j := 1$  to  $\text{ceil}(\log_2 n)$  do
     $mid := (high + low) \text{ div } 2$ 
    if  $a[t[mid]] < b[i]$  then  $low := mid$ 
    else  $high := mid$ 
  od
  if  $b[i] = a[t[mid]]$  then  $c[i] := t[mid]$ 
  else  $c[i] := m + 1$ 
od

```

Using this technique the time necessary to produce the results has been reduced to $O(n \log_2 n)$ for sorting the left argument, and $O(m \log_2 n)$ for performing the m binary searches, at the cost of an additional $O(n)$ space that would not otherwise be necessary (the variable t). Simply by changing the implementation of the membership algorithm in this fashion, execution time of one example expression was observed to drop from 34.2 seconds down to 2.0 seconds [Budd 1981].

When one considers the possibility of implementing the membership function on a vector machine, it is tempting to assume that the preprocessing of the left argument is no longer necessary. Instead, in one vector step it is possible to discover whether any position matches a given element in the left argument; for example as in the following (which assumes no elements in the left argument are repeated):

```

for all  $i$  to  $n$  in parallel do
   $c[i] := m + 1$ 
od
for  $i := 1$  to  $n$  do
  for all  $j$  to  $m$  in parallel do
    if  $a[i] = b[j]$  then  $c[i] := j$ 
  od
od

```

The answer is produced in $O(n)$ steps and requires no additional space. In working with vector algorithms, however, one quickly learns to be suspicious of linear time algorithms. And indeed, by once more preprocessing the left argument (which now can be done in $O(\log_2 m)$ vector steps [Hillis & Steele 1986]), we can

perform all n binary searches in parallel, producing the answer in $O(\log_2 n)$ time. (But with a cost of four size n temporary variables.)

```

for all  $i$  to  $n$  in parallel do
     $high[i] := m$ 
     $low[i] := 1$ 
od
for  $j := 1$  to  $\text{ceil}(\log_2 n)$  do
    for all  $i$  to  $n$  in parallel do
         $mid[i] := (high[i] + low[i]) \text{ div } 2$ 
        if  $a[t[mid[i]]] < b[i]$  then  $low[i] := mid[i]$ 
        else  $high[i] := mid[i]$ 
    od
od
for all  $i$  to  $n$  in parallel do
    if  $b[i] = a[t[mid[i]]]$  then  $c[i] := t[mid[i]]$ 
    else  $c[i] := m + 1$ 
od

```

Conclusions

In moving from an environment in which values are produced one at a time on a scalar processor to one in which entire vectors can be manipulated in constant time, not only do many of our conventional views and opinions about algorithms become invalid, but choices and alternatives arise which are not present, or not as critical, in the scalar world. In particular there are time versus space tradeoffs which become much more pronounced in the vector environment than they were in the scalar case. Of course, from a theoretical point of view we should not be surprised at all that we can trade polynomial time and constant space (as in the reduction example) for logarithmic time and polynomial space [Parberry 1986]. The question is not whether such tradeoffs are possible, but under what circumstances the choice should be made.

In this paper we have presented a number of alternative schemes for implementing several operations in the language APL. Further work will be necessary to develop heuristics to help the language implementor in selecting the most appropriate technique.

Timothy A. Budd
 Department of Computer Science
 Oregon State University
 Corvallis, Oregon
 97331

September 20 1994

References

- [Bernecky 1973]
Bernecky, B., "Speeding Up Dyadic Iota and Dyadic Epsilon", In *Proceeding of the APL Congress 73*, (Copenhagen, Denmark, Aug 22-24). Elsevier North-Holland, New York, 1973, pp. 479-482.
- [Budd 1981]
Budd, Timothy A., "An APL Compiler", Technical Report 81-17, University of Arizona, October 1981.
- [Budd 1988]
Budd, Timothy A., *An APL Compiler*, Springer-Verlag, 1988.
- [Hillis 1985]
Hillis, W.D., *The Connection Machine*, MIT Press, Cambridge, Mass. 1985.
- [Hillis & Steele 1986]
Hillis, W. D. and Steele, G. L., "Data Parallel Algorithms", *Communications of the ACM*, Vol 29(12):1170-1183 (December 1986).
- [Parberry 1986]
Parberry, L., "Parallel Speedup of Sequential Machines: A Defense of the Parallel Computation Thesis", *Sigact News*, Vol 18(1), 1986, pp. 54-67.
- [Perlis & Rugaber 1979]
Perlis, Alan J. and Rugaber, Spencer, "Programming with Idioms in APL", *APL Quote Quad*, Vol 9(4):232-235 (June 1979).
- [Saal & Weis 1975]
Saal, H. and Weis, Z., "Some Properties of APL programs", *APL '75 Conference Proceedings*, (Pisa, Italy, June 11-13). ACM, New York, 1975, pp. 292-297.

This article was originally written in 1994. We thank Professor Budd for permission to print it in Vector - Ed.

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hacker's Corner: Making Waves in Win32	Adrian Smith	110
APL+Internet Explorer	Alexander Balako	113
Data Domains	Phil Chastney	121
A Very Simple Structure Compiler for J	Martin Neitzel	129
From Horadam to Atanosov	Joseph De Kerf	135

We apologise to readers looking for "At Play With J" which has been held over to Vector 16.3.

Hackers' Corner: Making Waves in WIN32

by Adrian Smith (adrian@cgsi.demon.co.uk)

The one piece of luggage I invariably forget to pack is some kind of travelling alarm clock. This habit came home to roost in Austria on our eclipse-spotting outing, when we needed to be up early to catch the morning coach for a cross-border raid on the Czech Republic. Of course I had a computer with me, but Delia (being nearly 4 years old) has no sound card, so I regret to say I wrote my wake-up call in APL*PLUS/PC using `□SOUND`. However this reminded me that several people had been having trouble getting WAV files to play properly on Windows95, and that I had never updated my little workspace that plays sounds from memory. I rather expected this to be easy, but good ol' Microsoft had gone out of their corporate way to make things awkward, as usual

Playing WAV Files in Windows95

Here is a simple translation of the old 16-bit code ...

```

▽ {r}+{flg}Play snd;sps
[1]  A Play a .WAV file (synchronous by default)
[2]  A flg+÷/0(synch) 1(asynch) 2(no deflt) 8(repeat) 16(nostop)
[3]  snd←snd, (~'. 'esnd)/'.WAV'
[4]  'sps'□NA'U winmm.C32|sndPlaySoundA <OT U'
[5]  ⚡(0=□NC'flg')/'flg+0'
[6]  r←sps snd flg
▽

```

Of course they changed the name of the DLL and you need to add an 'A' to the function name, but so far so good. It makes a noise. However when you call it with a 1 as the optional left argument, silence. This is a real nuisance if you want sound effects in games, as you really want the noise to keep playing after control has returned to APL. Stef provided the answer to this one ...

```

▽ {r}+{flg}Play snd
[1]  A Play a .WAV file (asynchronous by default)
[2]  A flg+÷/0(synch) 1(asynch) 2(no deflt) 8(repeat) 16(nostop)
[3]  snd←snd, (~'. 'esnd)/'.WAV'
[4]  'sps'□NA'U winmm.C32|sndPlaySoundA <OT U'
[5]  ⚡(0=□NC'flg')/'flg+0'
[6]  r←sps snd flg
▽

```

Spot the difference! That's right, the boot's bigger [1]. Well actually the header line is smaller, by 4 characters - when the `□NA` call goes out of scope the sound dies, so you *must not localise it*. First problem solved, but Adrian just a little peeved.

Playing Sounds from Memory

This is the real he-man stuff. The 16-bit code made heavy use of `HMEMCPY` to move bytes into a block of allocated, locked memory. Attempts to make the obvious translation to 32-bit calls met with nothing but `VALUE ERROR` as I tried `kernel32`, `user32` and so on. Having nearly given up, I dug into the Windows documentation on the CD and found no trace of `HMEMCPY` in the Win32 API. However I did get some hits in `Windows.H`, but as a `#DEFINE` to something like `RtlCopyMemory`. OK, let's look for this ... once again no joy so back to the header file to discover that a little further down, this is itself defined as `RtlMoveMemory`, with exactly the same syntax as before. Actually, it copies, and the significance of the `Rtl` is that the rightmost bytes are copied first which can matter if your memory blocks overlap. So, to play a sound from memory:

```

▽ PlaySound sound;GlobalAlloc;GlobalLock;GlobalUnlock;
  GlobalFree;handle;pointer;sink;RtlMoveMemory;sndPlaySoundA
[1]  A Add header structure to sound and play from memory image.
[2]  sound←AddHdr sound
[3]  □NA'U kernel32.C32|GlobalAlloc U U'
[4]  □NA'U kernel32.C32|GlobalLock U'
[5]  □NA'U kernel32.C32|GlobalUnlock U'
[6]  □NA'U kernel32.C32|GlobalFree U'
[7]  □NA'kernel32.C32|RtlMoveMemory U <U1[] U'
[8]  □NA'U winmm.C32|sndPlaySoundA U U'
[9]  handle←GlobalAlloc 0(psound)
[10] pointer←GlobalLock handle
[11] sink←RtlMoveMemory pointer sound(psound)
[12] A The <4> is the flag to play from memory at <pointer>
[13] sink←sndPlaySoundA pointer 4
[14] A Don't forget to free the memory!!
[15] sink←GlobalUnlock handle
[16] sink←GlobalFree handle
▽

```

Wow! Let's try it ...

```

▽ MakeTSF
[1]  A Make Tonic Sol Fah as variables
[2]  {doh re mi fah soh lah ti dohtop}+{100,-440*(2*÷12)*+÷0 2 2
1 2 2 2 1}''i8
▽

```

```

doh re mi
440 100 493.8833013 100 554.365262 100

▽ {r}←Square arg;□IO;sam;freq;ampl;dur;data;size;dieaway;vol
[1] freq vol←arg ◊ dur÷2[400+freq ◊ □IO÷0 ◊ sam÷14567
[2] size←[dur×sam
[3] dieaway←freq÷1000000
[4] ampl←1000×*-dieaway×size
[5] data←[ampl×sizep1002×(freq×dur)×(size)+size
[6] A Add random noise to leading edge (percussive effect)
[7] (200÷data)÷÷200p1000
[8] data←[data×vol÷0.1×[//|data
[9] data←(ampl>2)/data
[10] data←255[0[128+data
[11] r←data
▽

pSquare doh
13242

▽ Scale
[1] {PlaySound Square ω}“doh re mi fah soh lah ti dohtop
▽

▽ r←AddHdr snd;hdr;sam;size;□IO
[1] A Add header structure to sound, assuming standard sampling rate.
[2] A R I F F {size} W A V E f m t bl ... etc
[3] hdr←82 73 70 70 6 192 0 0 87 65 86 69 102 109 116 32 16 0 0 0 1 0 1 0 17 43
    0 0 17 43 0 0 1 0 8 0 100 97 116 97 226 191 0 0
[4] □IO÷0 ◊ sam÷14567 ◊ size÷psnd
[5]
[6] hdr[24 25 28 29]÷4p÷256 256rsam
[7] hdr[ε4 40+εi4]÷8p÷256 256 256 256rsize
[8]
[9] A Return completed structure, ready to be played ...
[10] r←hdr,snd
▽

```

OK, it sounds terrible, as the lower notes take a little longer to compute than the high ones! However for making happy beeping noises at the end of a long process - the sort of thing we always used □SOUND for - it works rather well.

References

- [1] Jeremy Clarkson, *BBC Top Gear*, on the new Mazda MX-5.

APL+Internet Explorer

by Alexandr Balako (*abalako @ infostroy.ru*)

Introduction

Using OLE Automations (OLE and ActiveX) extends the features of common applications. In this article I want to describe the possibility of using the "Microsoft Web Browser" ActiveX in APL. This OLE Automation is a part of Internet Explorer 4 (IE) or higher. The simple workspace is described too.

Object Model

Here I try to describe an object model of the "Microsoft Web Browser" (MWB). The MWB is a tool that allows access to different URLs (Uniform Resource Locators). It identifies the full path of a document, graphic, or other file on the Internet or an intranet. The MWB has a complex structure. That's why I show the objects, properties, methods and events of the part that is used in the simple WS.

Main object

Figure 1 shows the model of the main object:



Figure 1

Properties of the main object:

- **Document:** represents the content of the page that is currently navigated by the MWB;
- **LocationURL:** is a string that reports the path and name of the page;
- **LocationName:** is a string that reports the header of the page.

Methods that are used:

- **Navigate**: a command to read the page from a specified URL;
- **GoHome**: a command to navigate to the home URL (specified by the user as his home page);
- **GoBack, GoForward**: commands to navigate to the previous or next page;
- **GoSearch**: go to the search URL;
- **Refresh**: re-read the page from the current URL.

Just one Event was used:

- **DocumentComplete**: this event is reported by the MWB after reading the page.

After using the **Navigate** method it is possible to access a Document object. There are two types of Document that were used in the simple WS: HTML page and Folder. The Document as HTML page is shown in Figure 2 and as Folder in Figure 3.

Document (HTML)

When the MWB has navigated to an HTML (Hypertext Markup Language) page, the Document object contains a property 'body'. It can be used to access the page:

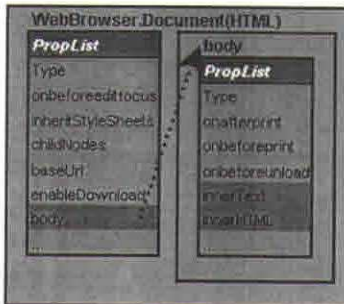


Figure 2

When the page contains frames it is possible to access the frame code through the frames property (I don't examine this case here).

The properties **InnerHTML** and **InnerText** produce strings with the HTML code and the text of the page respectively.

Document (Folder)

When the MWB has navigated to a folder, for example URL='C:\WINDOWS', the document is represented by other objects, as it does not contain a 'body' property. Instead it has a 'Folder' property that represents objects through the items method. Each Item is a folder or file or URL.

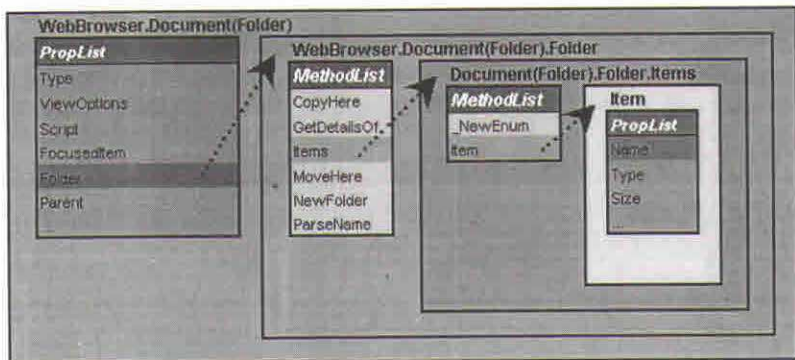


Figure 3

Now we are ready to start with an example.

Building a Simple Application

To build a demonstration WS we need set $\square ML \leftarrow 0$, $\square IO \leftarrow 1$ and create two functions. The first one is the main function that will run the simple application and is listed below:

```
[0] WebBrowser; OBJName; OBJPref; tmp; MicrosoftWebBrowser; F_MicrosoftWebBrowser;
[1] TRAP
[2] A This function Creates web browser
[3] OBJName='MicrosoftWebBrowser'
[4] OBJPref='F_MicrosoftWebBrowser.'
[5] A Create OCXClass
[6] OBJName WC'OCXClass' 'Microsoft Web Browser'
[7] A Create form
[8] (1+OBJPref)WC'Form' 'APL+Microsoft Web Browser'('Size' 480 640)
[9] ('Coord' 'Pixel')
[10] A Placing ActiveX to Form
[11] (OBJPref,OBJName)WC OBJName('Size' 430 640)('Posn' 50 0)
[12] ('Attach' 'Top' 'Left' 'Bottom' 'Right')
[13] A Detecting Event DocumentComplete from WebBrowser
[14] (OBJPref,OBJName)WS'Event' 'DocumentComplete' 'DocumentComplete'
[15] A Prepare decorate toolbar
```

```

[13] (OBJPref,'TB0')WC'ToolBar'('Size' 1 640)
[14] A Prepare toolbar for controls
[15] (OBJPref,'TB1')WC'ToolBar'('Size' 24 640)
[16] A Stop button
[17] tmp+*3 QNQ(OBJPref,'MicrosoftWebBrowser')'Stop''
[18] (OBJPref,'TB1.B1')WC'Button' 'Stop'('Event' 'Select' tmp)
[19] A Go Home Button
[20] tmp+*3 QNQ(OBJPref,'MicrosoftWebBrowser')'GoHome''
[21] (OBJPref,'TB1.B2')WC'Button' 'Go Home'('Event' 'Select' tmp)
[22] A Go back Button
[23] tmp+*3 QNQ(OBJPref,'MicrosoftWebBrowser')'GoBack''
[24] (OBJPref,'TB1.B3')WC'Button' 'Go Back'('Event' 'Select' tmp)
[25] A Go forward button
[26] tmp+*3 QNQ(OBJPref,'MicrosoftWebBrowser')'GoForward''
[27] (OBJPref,'TB1.B4')WC'Button' 'Go Forward'('Event' 'Select' tmp)
[28] A Refresh button
[29] tmp+*3 QNQ(OBJPref,'MicrosoftWebBrowser')'Refresh''
[30] (OBJPref,'TB1.B5')WC'Button' 'Refresh'('Event' 'Select' tmp)
[31] A Go Serch button
[32] tmp+*3 QNQ(OBJPref,'MicrosoftWebBrowser')'GoSerch''
[33] (OBJPref,'TB1.B6')WC'Button' 'Go Serch'('Event' 'Select' tmp)
[34] A Tool bar for edit field to type address
[35] (OBJPref,'TB2')WC'ToolBar'('Size' 24 640)
[36] A Button to react on Enter key
[37] tmp+*0 0p3 QNQ(OBJPref,'MicrosoftWebBrowser') 'Navigate'
      ((OBJPref,'TB2.ED')WG'Text') 0 0 0 0'
[38] (OBJPref,'TB2.B1')WC'Button' ''('Size' 0 0)('Default' 1)
      ('Event' 'Select' tmp)
[39] (OBJPref,'TB2.ED')WC'Edit'('Size'(24 630))
[40] QNQ(OBJPref,'TB1.B2')'Select'
[41] QTRAP+11 'E' ->LC'
[42] QDQ -1+OBJPref
[43] A End

```

The second one is the DocumentComplete function and it will detect the DocumentComplete event.

```
[0] DocumentComplete MSG;TMP;[]TRAP;IT;Ind;TMP1
[1] (OBJPref,'TB2.ED') []WS'Text'((OBJPref,OBJName)[]WG 'LocationURL')
[2] :If 9#[]NC OBJPref,'H'
[3] (OBJPref,'H')[]WC'FORM' 'HTML code'('Posn' 0 0)('Size' 200 300)
[4] (OBJPref,'H.E')[]WC'EDIT'('Style' 'Multi')('Posn' 0 0)('Size' 200 300)
('Vscroll' -1)
[5] :End
[6] :If 9#[]NC OBJPref,'T'
[7] (OBJPref,'T') []WC'FORM' 'Text of page'('Posn' 0 310)('Size' 200 300)
[8] (OBJPref,'T.E')[]WC'EDIT'('Style' 'Multi')('Posn' 0 0)('Size' 200 300)
('Vscroll' -1)
[9] :End
[10] :If 9#[]NC OBJPref,'F'
[11] (OBJPref,'F')[]WC'FORM' 'Items'('Posn' 0 620)('Size' 200 300)
[12] (OBJPref,'F.E')[]WC'EDIT'('Style' 'Multi')('Posn' 0 0)('Size' 200 300)
('Vscroll' -1)
[13] :End
[14] (OBJPref,'H.E')[]WS'Text' ''
[15] (OBJPref,'T.E')[]WS'Text' ''
[16] (OBJPref,'F.E')[]WS'Text' ''
[17] []TRAP+0 'E' '-0'
[18] 'TMP'[]WC(1=MSG)[]WG'Document'
[19] :If *[]NC'TMP.body'
[20] (OBJPref,'TB2.ED')[]WS'Text'((OBJPref,OBJName)[]WG 'LocationURL')
[21] 'TMP'[]WC'TMP'[]WG'body'
[22] (OBJPref,'H.E')[]WS'Text'('TMP'[]WG'innerHTML')
[23] (OBJPref,'T.E')[]WS'Text'('TMP'[]WG'innerText')
[24] :ElseIf *[]NC'TMP.Folder'
[25] 'TMP'[]WC'TMP'[]WG'Folder'
[26] 'TMP'[]WC TMP.Items
[27] IT+''
[28] :For Ind :In -1+i'TMP'[]WG'Count'
[29] :Trap 0
[30] 'TMP1'[]WC TMP.Item Ind
[31] IT,+<TMP1.Name
[32] :End
[33] :End
[34] IT+!IT
[35] (OBJPref,'F.E')[]WS'Text'(IT)
[36] :End
```

Now try to start the WebBrowser in the WS. The first screen is shown in Figure 4.

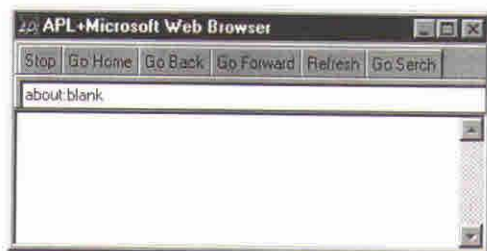


Figure 4

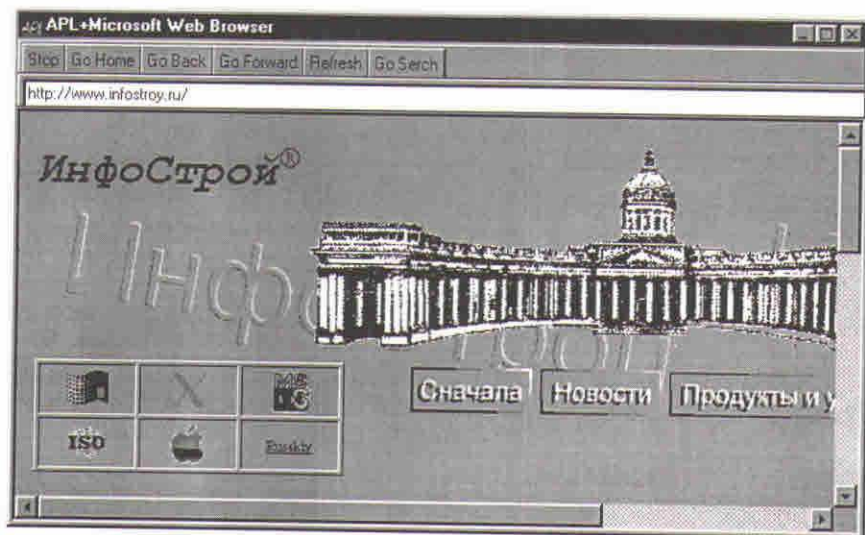


Figure 5

Try to type 'www.infostroy.ru' in the edit field and if you are connected to the Internet it shows the screen in Figure 5.

If you are not connected to the Internet type 'c:\WINDOWS' in the edit field and you will see the screen in Figure 6.

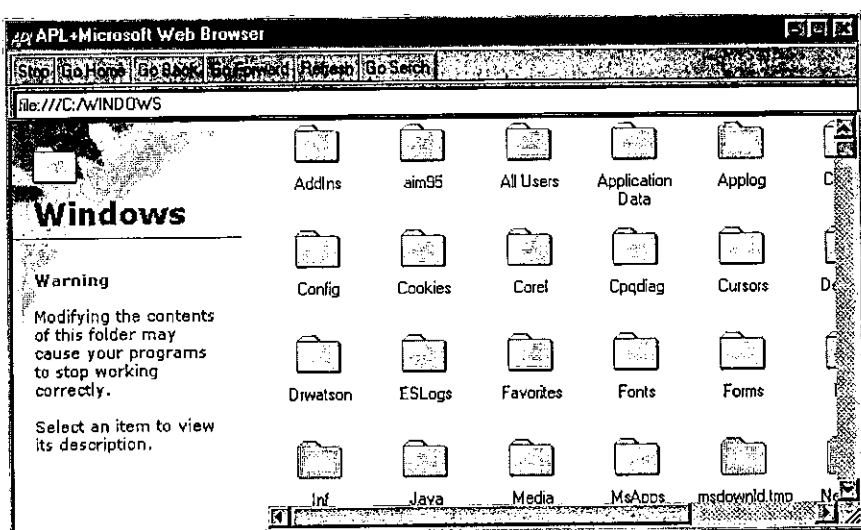


Figure 6

Accessing Page Contents

While you are trying to explore different pages, at the top of your screen 3 little forms are created (they are created by the 'DocumentComplete' function). The first is a form with HTML code (see lines 18-23 of 'DocumentComplete'):

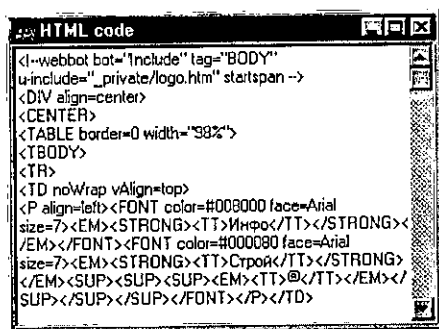


Figure 7

The second one contains the text of the page:

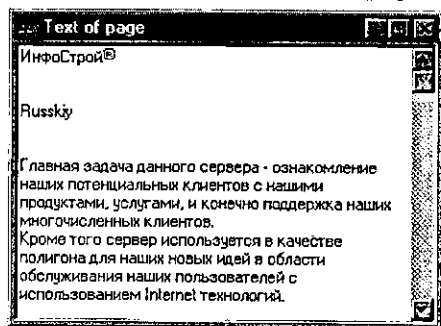


Figure 8

And the last form contains items if the page shows a folder (see lines 25-33 of 'DocumentComplete'):

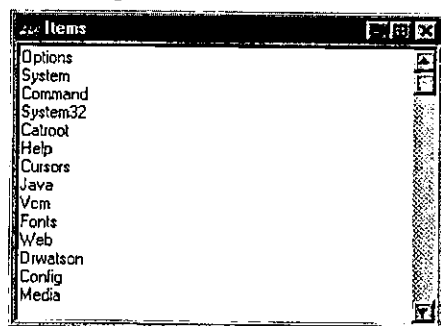


Figure 9

Play with the MWB

Now try to type in the edit field the path to an existing Office document. For example it could be 'c:\My Documents\Document1.doc' or 'c:\My Document\Book1.xls'. I think that you have never seen it before in APL. If you have Office installed it is possible to edit files there!!! This Part of the article was edited in the WebBrowser WS.

Conclusion

So we can power an application by Internet without creating an APL Java machine or APL HTML interpreter. It is possible to use this ActiveX as a tool to access Web sites, download files, copy, move and rename them without using low-level programming with the Windows APL.

Data Domains

by Phil Chastney

1. Towards a Definition

The domain of real numbers is denoted by $\mathbb{R}$.

It is unlikely that anyone would have a problem accepting this statement, although it is quite loaded. The use of the word "denoted" implies a Platonic (as opposed to a formalist) standpoint: even the use of the definite article as the first word reveals an assumption that (i) the domain of real numbers exists, and (ii) it is unique.

The really interesting word, however, is "domain". It is used in mediaeval history (where we were taught to spell it "demesne"), and also in permanent magnetism (a subject which still defies comprehension). The word crops up again in real analysis, where the "domain" and "range" of a function are matters of primary interest.

Data modelling makes much use of the term, and usage is increasing. Be warned, however, that the word is not used consistently. It has been used as a synonym for a column of a table in a relational database (i.e., all the actual values), for the set of actual values (i.e., after dropping out duplicates), and for the set of possible values which may occur in a column or columns. Sometimes the term is, implicitly or explicitly, restricted to "simple" domains (sometimes also called "primitive" or "atomic" domains), but most often, nowadays, it is used for "compound" or "structured" domains. (It used to be known as "designing the record layout"; nowadays it is known as "domain modelling" – only the name has been changed, to baffle the innocent.)

In formal semantics, a domain is something more than a set. Each element of a domain is deemed to be "tagged", so as to identify its original primitive domain. In set-theoretic terms, then, a domain is a set of ordered pairs – the first element being the tag, the second the "value". This models the situation inside the machine quite nicely, since the language processor will normally have some way of knowing the physical storage type of a variable, so that, for instance, 2 and 2.0 are distinguishable.

That serves the purposes of formal semantics very well, but for our purposes we shall use the simpler definitions:

- the **domain** of a variable is the set of all possible values that it is permitted to take;
- the **domain** of a function is the set of all inputs for which it is defined.

Implicit in the above is the restriction that we shall only be considering data domains.

2. The Primitive Data Domains

The primitive data domains are remarkably similar in all language.: in APL, they are as follows.

2.1 $\mathbb{H}$ - characters and literals

The memory of Hermann Hollerith lives on! To be precise, in APL, this domain contains the character scalars: character vectors (strings) are not a primitive domain. APL seems to be alone in taking this stance: most other languages define strings as the primitive domain. It might be argued that a language which comes equipped both with strings and utilities for extracting individual characters is, implicitly, using characters rather than strings as the primitive domain, but this argument is really only valid if the language in question actually recognises single characters as qualitatively different from strings – most do not.

2.2 $\mathbb{2}$ - the logical values, 0 and 1

The spirit of Paul Halmos lives on! The rather convenient device of using a bold or double-struck 2 to denote the set $\langle 0, 1 \rangle$ can be found in his Lectures on Boolean Algebras.

Some languages (e.g. C and APL) allow the logical values to be used in arithmetic, some (e.g. Basic and Pascal) do not.

2.3 $\mathbb{N}$ - the natural numbers

Surely everybody knows the natural numbers: 0, 1, 2, etc, etc, ad infinitum?

2.4 $\mathbb{Z}$ - the integers

The sequence of integers is 0, 1, -1, 2, -2, etc. etc. ad infinitum.

(We could not possibly define the sequence by starting at “negative infinity”, because (i) we have not yet established that this undefined “infinity” has a negative counterpart, and (ii) the sequence of natural numbers never actually *reaches* infinity, so we do not know how to count back from an infinite to a finite number.)

2.5 $\mathbb{R}$ - the real numbers

This is the familiar "real number line". Obviously, with a finite machine, there are upper and lower limits to the numbers we can actually represent, and with a finite word size we are further restricted to a small set of rational numbers between those limits. These are only physical limits, however, and do not affect our considerations here.

2.6 $\mathbb{C}$ - the complex numbers

Not all languages support complex numbers. No matter: they are not necessary for the purposes of this discussion.

3. Operations on Domains

3.1 membership

Perhaps the most fundamental operation we can perform using a domain is a test of membership:

$$X \in \mathbb{R}$$

or not, as the case may be:

$$X \notin \mathbb{R}$$

which have the usual semantics.

3.2 domain sum

New domains may be obtained by combining existing domains in various ways. The simplest of these is the **domain sum**, written

$$A \oplus B$$

which contains all the elements from domain A and all the elements from domain B. An element X is a member of the domain $A \oplus B$, if

$$(X \in A) \vee (X \in B) .$$

3.3 domain product

A more powerful way of combining domains is the **domain product** (or **cross product**, or **Cartesian product**), written

$$A \otimes B$$

whose elements are ordered pairs, the first element of each pair being drawn from domain A, the second element being drawn from domain B.

The domain product is a new construct: it is not merely iterated application of domain sum. It differs, in this respect, from the foundations of mathematics, where multiplication is introduced as a syntactically convenient way of representing repeated addition, and addition itself is only a syntactically convenient way of representing repeated application of the successor function.

There are many similarities between arithmetic addition and multiplication, on the one hand, and domain sum and domain product on the other: $A \oplus B$ and $B \oplus A$ are identical, for instance. $A \otimes B$ and $B \otimes A$, however, are not. The order in the elements of $A \otimes B$ means that the results of the two expressions are isomorphic, but not identical, so domain product is not commutative, so the two algebras are not identical. The symbols for the domain operations are (for the time being) enclosed in circles, to remind us of that fact.

3.4 tuples

The domain $\mathbb{R} \otimes \mathbb{R}$ consists of all possible ordered pairs of real numbers.

The domain $\mathbb{R} \otimes \mathbb{R} \otimes \mathbb{R}$ consists of all possible ordered triples of real numbers.

We shall use the notation $\mathbb{R}^*K$ (called **domain repetition**) as a syntactically convenient way to denote all possible k -tuples of real numbers: that is to say, all real vectors of length k .

(The enclosing circle has been dropped at this point because the circle-star combination has already been appropriated for other uses. The chances of confusion are (hopefully) reduced by the use of the asterisk, which floats above the line, as opposed to the APL exponentiation symbol, which is vertically centred.)

3.5 simple vectors

We can generalise the notation a little, by allowing the exponent to be a set, with the following definition:

$$A^*K = \sum_{k \in K} (A^*k) .$$

With this definition, we can see that

A^*N	$=$	(A^*0)	the (unique) empty vector
	$\oplus$	(A^*1)	all unit-length vectors
	$\oplus$	(A^*2)	all vectors of length 2
	$:$		
	$\oplus$	(A^*K)	all vectors of length k
	$:$		
		etc., etc., ...	

and A^*N therefore denotes the domain consisting of all vectors of whatever length, whose elements are drawn from A .

3.6 simple arrays

We can generalise further. Each of the terms in the above infinite series has a single-element exponent, and is therefore a vector. We can re-express the definition of vectors, as follows:

$\mathbb{R}^*N^*1$ all real-valued vectors

and extend the definition thus:

$\mathbb{R}^*N^*2$ all real-valued matrices

$\mathbb{R}^*N^*3$ all 3-dimensional real-valued objects

:

$\mathbb{R}^*N^*K$ all k-dimensional real-valued objects

:

etc, etc, and in particular:

$\mathbb{R}^*N^*0$ all real-valued scalars

which leads inevitably into

$\mathbb{R}^*N^*N$ all real-valued rectangular arrays .

4. Function Domains

4.1 Let us now consider functions – in particular, those functions which take an input and produce a result, without any side-effects. If the input is chosen from domain X and the result comes from domain A , we say that the function **maps** the domain A into domain X , which is written thus:

$$X \leftarrow A$$

and this expression is known as the **function mapping**. (A function mapping is still a domain, of course, but we already have the domain of the left argument and the domain of the right argument to consider, and we need something distinctive in the way of terminology here.)

4.2 A function such as “negation” will return a real result when supplied with a real argument.

The same truth can be variously expressed as:

“negation” maps reals into reals;

“negation” is a member of the set of functions which map reals into reals;

the function mapping of "negation" $\in \mathbb{R} \leftarrow \mathbb{R}$.

Some more primitive monadic functions and their mappings are as follows:

plus-reduce: $\mathbb{R} \leftarrow \mathbb{R} * \mathbb{N}$
 plus-scan: $(\mathbb{R} * \mathbb{N}) \leftarrow \mathbb{R} * \mathbb{N}$
 iota: $(\mathbb{N} * \mathbb{N}) \leftarrow \mathbb{N}$ (although $(\mathbb{N} * \mathbb{N}) \leftarrow \mathbb{N} \oplus \mathbb{N} * 1$ is normal practice)
 rho: $(\mathbb{N} * \mathbb{N}) \leftarrow (\mathbb{H} * \mathbb{N} * \mathbb{N}) \oplus (\mathbb{R} * \mathbb{N} * \mathbb{N})$
 ravel: $((\mathbb{H} * \mathbb{N}) \leftarrow \mathbb{H} * \mathbb{N} * \mathbb{N}) \oplus ((\mathbb{R} * \mathbb{N}) \leftarrow \mathbb{R} * \mathbb{N} * \mathbb{N})$

Note that the functions' mappings are expressed for their simplest non-trivial arguments.

4.3 The domain of monadic ravel (strictly speaking, the domain of its right argument) is

$$(\mathbb{H} * \mathbb{N} * \mathbb{N}) \oplus (\mathbb{R} * \mathbb{N} * \mathbb{N}),$$

while its range is

$$(\mathbb{H} * \mathbb{N}) \oplus (\mathbb{R} * \mathbb{N}).$$

We could therefore specify its function mapping as

$$((\mathbb{H} * \mathbb{N}) \oplus \mathbb{R} * \mathbb{N}) \leftarrow ((\mathbb{H} * \mathbb{N} * \mathbb{N}) \oplus (\mathbb{R} * \mathbb{N} * \mathbb{N}))$$

but this is less precise than the expression given in the preceding paragraph, because the earlier definition expresses the fact that the elements of the result of monadic ravel come from the same primitive domain as the elements of the right argument.

4.4 We might want to extend this idea, to make it is clear that an integer argument produces an integer result. The notation for this would be

$$((\mathbb{H} * \mathbb{N}) \leftarrow \mathbb{H} * \mathbb{N} * \mathbb{N}) \oplus ((\mathbb{N} * \mathbb{N}) \leftarrow \mathbb{N} * \mathbb{N} * \mathbb{N}) \oplus ((\mathbb{R} * \mathbb{N}) \leftarrow \mathbb{R} * \mathbb{N} * \mathbb{N})$$

and this could be extended to do the same for boolean arguments. It *could* be extended – but it won't be. The stance taken here is that *integer* vs. *real* is a question of internal representation, and representation is a matter of implementation not semantics. We ask only that the implementation can *distinguish* the elements of the each primitive domain: precisely *how* it distinguishes these domains is an implementation concern, and can therefore be airily dismissed as irrelevant to this discussion. The domain of an object is not necessarily the same as its physical type.

Noting that, because we are using "untagged" domains, $2 \subset \mathbb{N}$, $\mathbb{N} \subset \mathbb{Z}$, $\mathbb{Z} \subset \mathbb{R}$ and $\mathbb{R} \subset \mathbb{C}$, a corollary of this stance is that the interpreter is assumed to be capable of "promoting" a boolean to integer, say, where the context requires an integer, and

of "demoting" a real to an integer if the context requires an integer and the real value satisfies the appropriate integrality constraints.

4.5 More difficult to dismiss is the objection that the present syntax does not permit the expression of other important relationships, such as the fact that the result of plus-scan has the same shape as its argument, or that the length of the result of monadic iota is determined by its argument. This could be done: the conventional notation would be something like:

plus-scan: $\lambda k: (\mathbb{R}^*k) \Leftarrow \mathbb{R}^*k \in \mathbb{N}$

iota: $\lambda k: (\mathbb{N}^*k) \Leftarrow k \in \mathbb{N}$

but, for the time being, let us simply admit it as a shortcoming of the present notation and accept it as such.

4.6 Dyadic functions require extensions to the notation.

4.7 Conformability and scalar extension and suchlike are a separate matter.

5. Three Special Domains

5.1 $\emptyset$ - the empty domain

Clearly, the empty domain contains no elements. It is, however, still a domain: it is not a null entity.

5.2 D - the domain of domains

The domain of domains is denoted by D . An object is an element of D if and only if:

- (i) it is a primitive data domain, or
- (ii) it can be constructed from members of D using the operations of domain sum, domain product, domain repetition and function mapping.

Testing whether an object X is a domain then becomes a simple matter of testing whether

$$X \in D.$$

5.3 U - the universal domain

Nested arrays require extensions to the notation, but a given first generation APL implementation may have an overall data space described by

$$(\mathbb{H}^* \mathbb{N}^* \mathbb{N}) \oplus (\mathbb{R}^* \mathbb{N}^* \mathbb{N})$$

or

$$(H * N * N) \oplus (C * N * N)$$

or

$$(H \oplus R) * N * N.$$

It is convenient to represent the overall data space by U , known as the **universal data domain** (commonly, if inaccurately, abbreviated to the **universal domain**), a term which has the advantage of being implementation-independent.

The function mapping for monadic ρ can then be simplified (and generalised) to:

$$(N * N) \leftarrow U.$$

6. Summary

For very little work, we have developed a powerful and attractive notation, which helps us to describe some of the characteristics of functions, in a clear, unambiguous and concise manner, by describing the data domains of the functions' inputs and outputs.

References

- 7.1 R.D. Tennent, *Principles of Programming Languages*, Prentice Hall (1981)
The notation presented above is a very close copy of Tennent's Chapter 3, except that here it has been re-interpreted, simplified and generalised.
- 7.2 John McCarthy, *A basis for a mathematical theory of computation*, in P. Braffort and D. Hirschberg (ed.), *Computers and Formal Systems*, North Holland, Amsterdam (1963)
Tennent quotes McCarthy's paper as the earliest use of these constructions.
- 7.3 Dana Scott, *Towards a Mathematical Theory of Computation*, in Proc. 4th Annual Princeton Conference of Information Sciences and Systems. Dept. of Electrical Engineering, Princeton University (1970)
My own starting point, it made more sense after reading Tennent.

A Very Simple Structure Compiler for J

by Martin Neitzel (neitzel@gaertner.de)

Motivation

J programmers can use external libraries written in any language. Simple scalar values are easy to pass between J and the external function. The DLL interface knows about a handful of simple data types and does both a bit of conformance checking as well as all the necessary conversion work.

However, very often one has to deal with more complex data types such as C structures. This article describes one possible approach to analyse such structures conveniently.

The need to solve this problem came up when I wanted to write some J code to control a MIDI soundcard. More on this in another article [*in Vector 16.3 - Ed.*]. The first step was very easy:

```
<session trace>=  
  require 'dll'  
  midiOutGetNumDevs =:'mmsystem midiOutGetNumDevs n' & cd  
  midiOutGetNumDevs ''  
+--+  
|3|  
+--+
```

OK, so there are 3 MIDI devices on this machine, numbered 0, 1, and 2. But which is which? The Windows Multimedia API provides the function `midiOutGetDevCapsA` to identify them and get information on their capabilities. Its C prototype is:

```
int midiOutGetDevCapsA (int deviceNumber, MIDIOUTCAPSA *buf, int buflen);
```

After reading the `MIDIOUTCAPSA` type definition and arriving at a rough ballpark figure that 100 bytes would be on the safe side to hold the entire data, we could do the following to check out MIDI device number 0:

```
<session trace>+=
```

```
midIOutGetDevCapsA := 'winmm midIOutGetDevCapsA i i *c i' & cd
r=.midIOutGetDevCapsA 0 ; (100# ' '); 100
4 25 $ a. i. caps =. > 2 { r
46 0 9 0 4 4 0 0 69 83
83 32 77 80 85 45 52 48 49 0
168 139 0 0 132 235 109 0 108 12
0 0 7 0 0 0 128 15 0 0
1 0 0 0 0 0 255 255 0 0
0 0 32 32 32 32 32 32 32 32
32 32 32 32 32 32 32 32 32 32
32 32 32 32 32 32 32 32 32 32
32 32 32 32 32 32 32 32 32 32
32 32 32 32 32 32 32 32 32 32
```

Well, more complex data types such as structures or arrays require a bit more work, obviously. The J DLL interface and `memr/memw` treat such structures only as character vectors. It remains for us to deal with those bytes properly.

To do so, one has to take care of the exact offsets and field lengths, and after extraction of the relevant bytes, some conversion into a J data type is usually wanted. For example, the 4 bytes encoding a signed or unsigned integer should be properly converted to a single J number. Likewise, null-terminated C strings in fixed-size character arrays are only interesting up to the null character – nobody would usually be interested in the trailing garbage.

Rather than doing these things manually over and over again, it is much more convenient (and safer!) to let J do the extraction of a structure's fields. The following code demonstrates one approach based on type declarations specifying the conversion of a C structure to a boxed J table. In fact, we are not really focussed on the C language here. If you are concerned with, say, Delphi record types, this method will help you, too.

Two different kinds of declarations are used:

1. a global table of base types and associated converters
2. one table for each structure type declaring all the fields

Here is the original type definition of this example's structure. It is the C declaration from the Microsoft Windows' multimedia API describing MIDI output devices:

```
<c2j-data.ijs>=
```

```
NB. typedef struct tagMIDIOUTCAPSA {
NB.     WORD    wMid;           /* manufacturer ID */
NB.     WORD    wPid;           /* product ID */
NB.     MMVERSION vDriverVersion; /* version of the driver */
NB.     CHAR     szPname[MAXPNAMELEN]; /* product name (NULL terminated string) */
NB.     WORD     wTechnology;    /* type of device */
NB.     WORD     wVoices;        /* # of voices (internal synth only) */
NB.     WORD     wNotes;        /* max # of notes (internal synth only) */
NB.     WORD     wChannelMask;   /* channels used (internal synth only) */
NB.     DWORD    dwSupport;     /* functionality supported by driver */
NB. } MIDIOUTCAPSA
```

Adapting Foreign Structure Types to J

Different C base types should show up differently in J, too. Our type declarations will associate a converter expression with each C type. Here are some auxiliary converter verbs. They all take a series of bytes (*i.e.*, a J character list) as input.

```
<c2j-data.ijs>+=
```

```
char=: {. NB. character (singleton to scalar)
ctoi =: 3 : '256 #. a. i. |. y.' NB. C bytes to J integer.
NUL=.0{a. zerocut =: {.- i.&NUL NB. C string to J string.
```

Now our type declarations. The first, `TypeTable`, is an initial table of Type Converters for the base types as used in C. Each row of the table is a boxed triple of

1. the name of the type (a string)
2. the memory usage for a single value of this type in bytes (a number)
3. the name of a converter from memory bytes to a J value (suitable to be evoked later)

A full table would of course contain many more rows.

Structure Declarations define the fields of a structure we are interested in. They have the the following three columns:

1. the type name of the field (referring to a `TypeTable` entry)
2. repeat count (for arrays)
3. name of the field

Both tables are written as plain text nouns using tabs(!) and newlines to delimit rows and columns. The cutting of the fields happens on-the-fly, as does the text to number version of the middle fields.

```
<c2j-data.ijs>+*
```

NB. (I wish I knew a more elegant mapping than the complicated Amend here:)
 middlenum =. ".&.>@{1&{}@]`1:~ "1

NB. Type definitions for:

NB. short word, long word, character, null-terminated string, version type

```
TypeTable =: middlenum < ; _1 ; _2 noun define
```

```
  w      2      ctoi
  W      4      ctoi
  c      1      char
  s      1      zerocut
  v      4      ctoi
```

```
)
```

NB. Description corresponding to MIDIOUTCAPSA:

```
MOCAPS_SD =: middlenum < ; _1 ; _2 noun define
```

```
  w      1      wMid
  w      1      wPid
  v      1      vDriverVersion
  s     32      szPname
  w      1      wTechnology
  w      1      wVoices
  w      1      wNotes
  w      1      wChannelMask
  W      1      dwSupport
```

```
)
```

The definition MOCAPS_SD contains all it is necessary to know to define a structure, provided you also have the knowledge about the base type as encoded in TypeTable. The utility `expand_structure` will interpolate this information and pre-compute some additional information such as the individual offset and size of each field. An Expanded Structure Definition compiled in this way has seven columns altogether:

- basetype
- arraycnt
- name
- typelen
- conv
- length
- offset

```

<c2j-data.ijs>+=

expand_field =: monad define "1
    i=.({."1 TypeTable) i. {. y.
    y. , }. i { TypeTable
)

expand_structure =: monad define
    y. =. expand_field y.
    sizes =. */"1 > 1 3 { "1 y.
    offsets =. 0, }.: +/\ sizes
    y. ,. <"0 sizes ,. offsets
)

```

The cutter utility finally takes the memory and one or more rows of the Expanded Structure Description (one row for each field you are interested in). It extracts the data from the byte vector, passes it to the appropriate converter, and associates it with the field name. One would typically use the entire ESD and get a nice association table.

```

<c2j-data.ijs>+=

cutter =: dyad define "1 1
    'basetype arraycnt name typelen conv length offset' =. x.
    datachars =. (offset + i. length) { y.
    name ; conv- datachars
)

```

Now, we can have a proper look at our cryptic structure:

```

<session trace>+=

] MOCAPS_ESD =. expand_structure MOCAPS_SD
+-----+-----+-----+
|w|1|wMId|2|ctoi|2|10|
+-----+-----+-----+
|w|1|wPid|2|ctoi|2|12|
+-----+-----+-----+
|v|1|vDriverVersion|4|ctoi|4|14|
+-----+-----+-----+
|s|32|szPName|1|zerocut|32|8|
+-----+-----+-----+
|w|1|wTechnology|2|ctoi|2|140|
+-----+-----+-----+
|w|1|wVoices|2|ctoi|2|142|
+-----+-----+-----+
|w|1|wNotes|2|ctoi|2|144|
+-----+-----+-----+
|w|1|wChannelMask|2|ctoi|2|146|
+-----+-----+-----+
|W|1|dwSupport|4|ctoi|4|148|
+-----+-----+-----+

```

MOCAPS_ESD cutter caps

wM1d	46	
wP1d	9	
vDriverVersion	1028	
szPname	ESS MPU-401	
wTechnology	1	
wVoices	0	
wNotes	0	
wChannelMask	65535	
dwSupport	0	

Possible and Necessary Extensions

Enter derived types into the TypeTable, along the lines of

```
TypeTable:=TypeTable,'MOCAPS';(sizeof MOCAPS_ESD);'MOCAPS_ESD&cutter'
```

The `expand_structure` compiler should automatically take alignment into account as appropriate for the machine. If you have a structure declared as `struct {char c; int i;}`, the C standard will require that the memory layout has `c` and `i` in this order, but the hardware architecture may require one or even more bytes of padding between them to align `i` properly.

A marriage with the standard J fields.ijs utility.

Code to *write* structures/fields, probably based on just associating proper obverses with each converter.

Acknowledgements

I'm indebted to Eric Iverson and Uwe Faustmann for helping me with this text.

Martin Neitzel
Gärtner Datensysteme
Braunschweig, Germany

From Horadam to Atanassov

by Joseph De Kerf

Abstract

In a previous paper, the generalization of the Fibonacci sequence to what we called the Horadam sequences was investigated. In this paper we treat the so-called generalization introduced by Atanassov, in fact not a generalization but a new concept as the recursive formula is multiplicative instead of linear. APL defined functions for generating these sequences are given. Benchmarks confirm that explicit calculation is much more efficient in CPU time than the calculation based on recursion.

Horadam Sequences

In a previous paper [1] the generalization of the Fibonacci sequence to what we called Horadam sequences [2]:

$$\begin{aligned} H_1 &= p \\ H_2 &= q \\ &\dots \\ H_n &= rH_{n-2} + sH_{n-1} \quad (n > 2) \end{aligned}$$

was investigated. APL defined functions for calculating those sequences, by applying the recursion formula (*HOR1* and *HOR2*) or by explicit calculation from the auxiliary calculation (*HOR3*) were given. Benchmarks confirmed explicit calculation from the auxiliary calculation is more efficient in CPU time than the calculation based on recursion. In this paper we investigate the so-called generalization, introduced by K.T. Atanassov [3].

Atanassov Sequences

Atanassov sequences may be defined by the recursion formula:

$$\begin{aligned} T_1 &= p \\ T_2 &= q \\ &\dots \\ T_n &= T_{n-2} \times T_{n-1} \quad (n > 2) \\ &= p^{F_{n-2}} \times q^{F_{n-1}} \end{aligned}$$

with $F_1 = 1$

$F_2 = 1$

.....

$$F_n = F_{n-2} + F_{n-1} \quad (n > 2)$$

the sequence $F_1 \dots F_n$ being the Fibonacci sequence, but the domain of the parameters being the set of real numbers.

In fact, the Antanassov sequence is not a generalization of the Fibonacci or Horadam sequences: the recursion formula for the Fibonacci and Horadam sequences is linear, while the recursion formula for the Antanassov sequence is multiplicative.

Even for small values of p and q , the absolute value of T_n may increase very rapidly. With $p = 2$ and $q = 3$ for instance, we get for $n = 1(1)15$:

n	T_n
1	2
2	3
3	6
4	18
5	108
6	1944
7	209952
8	408146688
9	8.569121344E13
10	3.497458496E22
11	2.997014624E36
12	1.048193426E59
13	3.141451027E95
14	3.292848314E154
15	1.034432172E250

such that for $n=16$, the value of T_n would be 3.406228233E404, generating a domain error report when the implementation conforms to the ANSI/IEEE Standard 754-1985 [4].

This may be intercepted by calculating the logarithm of the absolute value of T_n instead of the value of T_n itself:

$$\log|T_1| = \log|p|$$

$$\log|T_2| = \log|q|$$

.....

$$\log|T_n| = \log|T_{n-2}| + \log|T_{n-1}| \quad (n > 2)$$

$$= F_{n-2} \log|p| + F_{n-1} \log|q|$$

giving a Horadam sequence H_n with parameters $\log|p|$, $\log|q|$, $r=1$ and $s=1$. With $p=2$ and $q=3$ for instance, taking the common logarithm, we get:

$$\begin{aligned}\log|T_1| &= 0.3010299957 \\ \log|T_2| &= 0.4771212547 \\ \log|T_3| &= 0.7781512504 \\ &\dots\dots\dots \\ \log|T_{1000}| &= 1.78152198E208\end{aligned}$$

The auxiliary equation of $H_n = \log|T_n| = \log|T_{n-2}| + \log|T_{n-1}|$, is $h^2 - h - 1 = 0$, such that [5]:

$$\begin{aligned}H_n = \log|T_n| &= ca^{n-1} + db^{n-1} \quad \text{with } \Delta = \sqrt{5} \\ a &= (1 + \Delta) / 2 \\ b &= (1 - \Delta) / 2 \\ c &= (\log|q| - b \cdot \log|p|) / \Delta \\ d &= (a \cdot \log|p| - \log|q|) / \Delta\end{aligned}$$

If $c = 0$, as $|b| < 1$, the sequence $H_n = \log|T_n|$ converges to 0. The calculated value of c however, may be slightly different from 0 and, as $|a| > 1$, the calculated sequence of $H_n = \log|T_n|$ converges to $\pm \infty$. Correction may be done by setting $c = 0$ when $\log|q| = b \cdot \log|p|$. If $d = 0$, no correction is needed as $|b| < 1$.

Examples:

$$\begin{aligned}p=10 & \quad q=10^b & (c=0) \\ p=10^a & \quad q=0.1 & (c=0)\end{aligned}$$

Defined Functions

APL defined functions for calculating the sequence $\log|T_1| \dots \log|T_n|$ based on implicit recursion (ATA1), programming a recursive loop (ATA2), the linear transformation formula $\log|T_n| = F_{n-2} \log|p| + F_{n-1} \log|q|$ (ATA3), and the explicit calculation from the auxiliary equation of $\log|T_n| = \log|T_{n-2}| + \log|T_{n-1}|$ (ATA4) are given below. In line 3 of ATA4, C is set to 0 when $P[2] = P[1] \times b$ (P containing the logarithms of the absolute values of the parameters p and q).

```
[0]  Z←P ATA1 N
[1]  →(N≤2)/0,Z←(2\N)ρ10∘|P
[2]  Z←P ATA1 N-1
[3]  Z←Z,+/-2+Z
```

```
[0] Z←P ATA2 N
[1] →(N≤2)/0, Z+(2[N]ρ10⊗|P
[2] END:→(N>ρZ+Z, +/ˆ2+Z)/END
```

```
[0] Z←P ATA3 N;F;R
[1] →(N≤2)/0, Z+(2[N]ρP+10⊗|P
[2] F←{0.5+(÷R)×(0.5×1+R+5×0.5)}×1N-1
[3] Z←Z, (P[1]×ˆ1+F)+P[2]×1÷F
```

```
[0] Z←P ATA4 N;A;B;C;D;Δ
[1] →(N≤2)/0, Z+(2[N]ρP+10⊗|P
[2] B←1-A+0.5×1+Δ+5×0.5
[3] C←(P[2]×P[1]×B)×(P[2]-P[1]×B)÷Δ
[4] D←((P[1]×A)-P[2])÷Δ
[5] Z←Z, (C×A×N)+D×B×N+1+1N-1
```

Common logarithms are used. The left argument is a two-element vector containing the parameters p and q . If $P[1]=0$ ($p=0$) and/or $P[2]=0$ ($q=0$), an error report is generated. The right argument N is the non-negative integer scalar or one-element vector specifying the number of terms of the sequence requested. The explicit result Z is a vector containing those terms. If N is 0, the empty numeric vector is returned. If N is a negative integer or not an integer, an error report is generated. Results are not rounded off as in general the logarithms of the absolute values of p and q are not integers (even if the domain of p and q is restricted to the set of integers).

Finally, if the value of $\log|T_n|$ is requested for only one specific value of n , when applying recursion, the entire sequence $\log|T_1| \dots \log|T_n|$ must be calculated. When applying explicit calculation from the auxiliary calculation however, $\log|T_n|$ must only be calculated for that specific value of n . A modified form ATA5 of the defined function ATA4, that only calculates $\log|T_n|$ for a given non-negative scalar or one-element vector is given below.

```
[0] Z←P ATA5 N;A;B;C;D;Δ
[1] →(N≤2)/0, Z+(P+10⊗|P)[2[N]
[2] B←1-A+0.5×1+Δ+5×0.5
[3] C←(P[2]×P[1]×B)×(P[2]-P[1]×B)÷Δ
[4] D←((P[1]×A)-P[2])÷Δ
[5] Z←(C×A×N-1)+D×B×N-1
```

Examples:

```
2 3 ATA1 3
0.3010299957 0.4771212547 0.7781512504
```

```

      2 3 ATA2 3
0.3010299957 0.4771212547 0.7781512504

```

```

      2 3 ATA3 3
0.3010299957 0.4771212547 0.7781512504

```

```

      2 3 ATA4 3
0.3010299957 0.4771212547 0.7781512504

```

```

      2 3 ATA5 3
0.7781512504

```

```

      2 3 ATA5 1000
1.78152198E208

```

Note: Applying recursion (*ATA1* and *ATA2*) as well as linear transformation of the Fibonacci sequence (*ATA3*), gives erroneous results for the special case described above ($c=0$). Results for the examples are compared with the exact solutions as returned by *ATA4* in the accompanying Appendix. It would be a challenge to incorporate the corrections needed such as is done in the defined functions *ATA4* and *ATA5* (if possible!).

Benchmarks

Benchmarks have been done for $N = 250(250)1000$ in order to compare the effectiveness in CPU time for the functions defined. Results are given below (in ms):

<i>N</i>	250	500	750	1000
<i>ATA1</i>	100.0	220.0	362.7	532.9
<i>ATA2</i>	52.2	121.7	211.6	317.7
<i>ATA3</i>	3.91	5.96	8.18	10.47
<i>ATA4</i>	3.64	5.93	8.36	10.85
<i>ATA5</i>	1.54	1.54	1.54	1.54

Depending on N , the CPU times are from 48% to 40% lower for *ATA2* than for *ATA1*. The CPU times for *ATA3* and *ATA4* are approximately equal and depending on N , the efficiency factor in CPU time versus *ATA2* is from 14 to 30. Depending on N , the efficiency factor in CPU time of *ATA5* versus *ATA2* is from 34 to 206. Figures conform *grasso modo* to those reported for the defined functions for calculating the Horadam sequence. Of course, those figures depend on the hardware and software used.

Functions defined conform to the standard ISO APL [6]. Programming, debugging, illustrations and benchmarks have been done on a MicroLine Pentium S-100, with mathematical co-processor, using Dyalog APL/W Version 7.1.2, under Windows 3.11 [7]. Comparison Tolerance was set to $1E^{-13}$, the default value being $1E^{-14}$. Benchmarks have been done using the system function *QdMONITOR*.

The Signum Function

Calculating $\log |T_n|$ instead of T_n , the signum of T_n remains unknown. Signum of T_n depends on the signum of the parameters p and q . An overview of possible combinations is given below:

p	q	signum sequence								
0	0	0	0	0	0	0	0	0	0	...
0	+	0	1	0	0	0	0	0	0	...
0	-	0	-1	0	0	0	0	0	0	...
+	0	1	0	0	0	0	0	0	0	...
-	0	-1	0	0	0	0	0	0	0	...
+	+	1	1	1	1	1	1	1	1	...
-	-	-1	-1	-1	-1	-1	-1	-1	-1	...
+	-	1	-1	-1	1	-1	-1	1	-1	...
-	+	-1	1	-1	-1	1	-1	-1	1	...

If the signum of p and/or the signum of q is 0, the subsequent signums are 0. If neither the signum of p nor the signum of q is 0, the signum sequence is periodic, the period being the three-element vector containing in that order, the signum of p , the signum of q , and the signum of pq .

APL defined functions returning the corresponding sequence of signums of the sequence $T_1 \dots T_n$ (*SIGNUM1*) and the signum of the n th term T_n (*SIGNUM2*) are for instance:

```
[0] Z←P SIGNUM1 N
[1] →(N≤2)/0,Z+(2|N) pP×P
[2] →(0=×/P)/0,Z+P,(N-2) p0
[3] Z←N pP,×/P
```

```
[0] Z←P SIGNUM2 N
[1] →(N≤2)/0,Z+(P×P)[2|N]
[2] →(0=×/P)/0,Z+0
[3] Z←(1 2 0=3|N)/P,×/P
```

Benchmarks have been done for $N=250(250)1000$, for $pq=0$ and for $pq \neq 0$ respectively. Results are (in ms):

N	250	500	750	1000
$SIGNUM1$ ($pq=0$)	1.23	1.40	1.57	1.75
$SIGNUM1$ ($pq \neq 0$)	1.09	1.12	1.15	1.18
$SIGNUM2$ ($pq=0$)	0.96	0.96	0.96	0.96
$SIGNUM2$ ($pq \neq 0$)	1.08	1.08	1.08	1.08

The difference in CPU time for $SIGNUM1$ and $SIGNUM2$ is rather small. So, in practice, it is more simple to implement $SIGNUM1$ as $SIGNUM$ and, if only the signum for the n th term is requested, to use the expression $-1 + P \text{ SIGNUM } N$.

References

- [1] J. De Kerf: *From Fibonacci to Horadam*. Vector Vol.15. No.3 January 1999, pp 122-130
- [2] A.F. Horadam: *Basic Properties of a Certain Generalized Sequence of Numbers*. The Fibonacci Quarterly, Vol.3 No.3 August 1985, pp 161-176
- [3] K.T. Antanassov: Remark on a New Direction for a Generalization of the Fibonacci Sequence. The Fibonacci Quarterly, Vol.33 No.3, June-July 1995, pp 249-250
- [4] ANSI/IEEE Standard 754-1985. *IEEE Standard for Binary Floating-Point Arithmetic*. Institute of Electrical and Electronics Engineers IEEE, New York 1985
- [5] A. Brousseau: *Linear Recursion and Fibonacci Sequences*. The Fibonacci Association, San Jose State College, San Jose, California, 1971
- [6] International Standard ISO 8485: 1989 (E), First Edition; 1989-11-01; *Programming Languages - APL*. International Organization for Standardization ISO, Geneva, Switzerland, 1989
- [7] *Dyalog APL/W Reference Manual -Version 7*. Dyadic Systems Ltd., Basingstoke, 1994

Appendix

Results for the special cases described in the text, returned by *ATA1/ATA2/ATA3*, are compared with the exact solutions returned by *ATA4*.

```

P+10 (10*0.5*1-5*0.5)
R1+-5+ ATA1 1002
R2+-5+ ATA2 1002
R3+-5+ ATA3 1002
R4+-5+ ATA4 1002
( -5+1002),R1,R2,R3,[1.5]R4
998 -5.573927286E191 -5.573927286E191 -1.140610154E192 -4.358355692E-209
999 -9.018803800E191 -9.018803800E191 -1.140610154E192 2.693611953E-209
1000 -1.459273109E192 -1.459273109E192 -2.281220309E192 -1.664743739E-209
1001 -2.361153489E192 -2.361153489E192 -4.562440618E192 1.028868213E-209
1002 -3.820426597E192 -3.820426597E192 9.124881235E192 -6.358755258E-210

P+(10*0.5*1+5*0.5) 0.1
R1+-5+ ATA1 1002
R2+-5+ ATA2 1002
R3+-5+ ATA3 1002
R4+-5+ ATA4 1002
( -5+1002),R1,R2,R3,[1.5]R4
998 2.622901190E192 2.622901190E192 2.281220309E192 -7.051967644E-209
999 4.243943274E192 4.243943274E192 4.562440618E192 4.358355692E-209
1000 6.866844464E192 6.866844464E192 4.562440618E192 -2.693611953E-209
1001 1.111078774E193 1.111078774E193 9.124881235E192 1.664743739E-209
1002 1.797763220E193 1.797763220E193 4.562440618E193 -1.028868213E-209

```

Results returned by *ATA1/ATA2/ATA3* converge to $\pm \infty$ and are erroneous.
Results returned by *ATA4* converge to 0 and are correct.

Joseph De Kerf
Rooienberg 72
B-2570 Duffel
Belgium

Index to Advertisers

Dyadic Systems Ltd	6
Strand Software	2
IPR Corporation Ltd	12
Mackay Kinloch	10
Vector Back Numbers	37

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (hardcopy with diskette as appropriate) to the Vector Working Group via:

Vector Administration, c/o Gill Smith
Brook House
Gilling East
YORK, YO62 4JJ
Tel: +44 (0) 1439-788385
Email: apl385@compuserve.com

Authors wishing to use Word for Windows should contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template. These may also be downloaded from the Vector web site at www.vector.org.uk

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1999/2000 Committee

Chairman	Anthony Camacho 0117-973 0036 100612.1057@compuserve.com	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Secretary	Ajay Askoolum 01737-771643 106173.3347@compuserve.com	42 Hanworth Road, Redhill, Surrey RH1 5HT
Treasurer	Nicholas Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ
Journal Editor	Stefano Lanzavecchia stf@apl.it	c/o Vector Administration, Brook House, Gilling East YORK YO62 4JJ
Projects and Publicity	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Webmaster	Ray Cannon 01252-874697 100430.740@compuserve.com	21 Woodbridge Road, Blackwater, Camberley, Surrey GU17 0BS
Activities	Jon Sandles 01904-612882 jon_sandles@csi.com	138 Burton Stone Lane, YORK YO30 6DF
Education	Dr Ian Clark 01388-605544 100021.3073@compuserve.com	Unit R21, Auckland New Business Centre, St. Helen Auckland, Bishop Auckland, Co. Durham DL14 9TX
Administration	Rowena Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ

Journal Working Group

Editor:	Stefano Lanzavecchia	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan Barman (01488-648575), Richard and Adam Weber (0121-3546550), Anthony & Sylvia Camacho, Ray Cannon (01252-874697), Marc Griffiths, Bob Hockstra (01483-771028), Jon Sandles (01904-612882)	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" — an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
The Mallings, Castlegate,
MALTON, North Yorks YO17 7DP
Tel: 01653-696760
Fax: 01653-697719
Email: causeway@csi.com
Web: www.causeway.co.uk

Compass Ltd
Compass House
60 Priestley Road
GUILDFORD, Surrey GU2 5YU
Tel: 01483-514500

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants, RG24 0AL
Tel: 01256-811125
Fax: 01256-811130
Email: sales@dyadic.com
Web: www.dyadic.com

HMW Trading Systems Ltd
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel: 0171-353 8900
Fax: 0171-353 3325
Email: 100020.2632@compuserve.com

Insight Systems ApS
Nordre Strandvej 119C
DK-3150 Hellebæk
Denmark
Tel: +45 49 76 20 20
Fax: +45 49 76 20 30
Email: info@insight.dk
Web: www.insight.dk

APL2000
Rapid Application Development
6610 Rockledge Drive
Suite 502
Bethesda MD 20817
USA
Email: sales@apl2000.com
Web: www.apl2000.com

Soliton Associates Ltd
Groot Blankenberg 53
1082 AC Amsterdam
Netherlands
Tel: +31 20 646 4475
Fax: +31 20 644 1206
Email: sales@soliton.com

Dutch APL Association
Postbus 1341
3430BH Nieuwegein
Netherlands
Tel: +31 347 342 337